



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



FACULTAD DE
INGENIERÍA

Semáforos Inteligentes

Un Enfoque con Aprendizaje por Refuerzo Multiagente

TESINA

Licenciatura en Ciencias de la Computación

AUTOR

Juan Martín Morales

DIRECTORA

Dra. Ana Carolina Olivera

CIUDAD DE MENDOZA, ARGENTINA

2026

Resumen

En este trabajo se aborda el problema de la congestión vehicular en áreas urbanas mediante un sistema de control semafórico basado en el Aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*) (MARL) cooperativo. Se analiza la evolución del parque automotor en Argentina, se describen las limitaciones del control semafórico tradicional y se formula el problema como una tarea de decisión secuencial en la que las intersecciones actúan como agentes. El objetivo es desarrollar y evaluar políticas coordinadas que reduzcan las demoras y mejoren las condiciones de operación de la red. La espera acumulada por carril se codifica mediante una transformación logarítmico-lineal acotada y discretizada, con mayor resolución para valores bajos de esa medida; así, valores próximos se representan con una misma categoría y los extremos quedan acotados por un umbral predefinido. También se estudia una adaptación de HAPPO, con actualización secuencial de las políticas, ante la heterogeneidad de las intersecciones. La propuesta se evalúa como parte del sistema completo mediante simulaciones microscópicas en escenarios sintéticos y en una red vial que representa el microcentro de la Ciudad de Mendoza, comparando algoritmos independientes y cooperativos con métricas de tránsito y de aprendizaje.

Abstract

This work addresses urban traffic congestion through a traffic-signal control system based on cooperative MARL. It analyzes the growth of Argentina’s vehicle fleet, describes the limitations of traditional signal control, and formulates the problem as a sequential decision task in which intersections act as agents. The objective is to develop and evaluate coordinated policies that reduce delays and improve network operating conditions. The accumulated waiting time per lane is encoded through a bounded, discretized log-linear transformation with finer resolution for low values of the measure; nearby values are mapped to the same category, and extremes are capped at a predefined threshold. The study also examines an adaptation of HAPPO, with sequential policy updates, for heterogeneous intersections. The approach is evaluated as part of the complete system through microscopic simulations of synthetic scenarios and an urban road network representing downtown Mendoza City, comparing independent and cooperative algorithms using traffic and learning metrics.

Agradecimientos

Página de agradecimientos

Índice

1	Introducción	1
1.1	Motivación	3
1.2	Organización del documento	4
2	Marco teórico	5
2.1	Ingeniería de tránsito y simulación	5
2.1.1	Terminología de tránsito	6
2.1.2	Fundamentos de la dinámica vehicular	8
2.1.3	Sistemas de control de tránsito	14
2.1.4	Simulación de tránsito	15
2.2	Aprendizaje por refuerzo (RL)	22
2.2.1	Procesos de decisión de Markov (MDP)	23
2.2.2	Métodos basados en valor y basados en política	25
2.2.3	Aprendizaje profundo y redes neuronales	29
2.2.4	Aprendizaje por refuerzo profundo (DRL)	32
2.3	Aprendizaje por refuerzo multiagente (MARL)	39
2.3.1	Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP	40
2.3.2	Desafíos del aprendizaje multiagente	41
2.3.3	Paradigmas de entrenamiento y ejecución	43
2.3.4	MADRL y líneas de base	44
2.3.5	MAPPO	45
2.3.6	HAPPO	48
2.3.7	Antecedentes recientes del control semafórico multiagente	53
3	Metodología y diseño experimental	59
3.1	Enfoque y diseño de investigación	59
3.2	Especificación del ecosistema tecnológico	63
3.3	Diseño y calibración de escenarios de tránsito en SUMO	64
3.3.1	Modelado geométrico y sanitización topológica	65
3.3.2	Escenarios sintéticos: 3×1 , 3×3 y $4 \times 4_H$	65

3.3.3	Escenario urbano de la Ciudad de Mendoza	66
3.4	Arquitectura de integración entorno-agentes	72
3.4.1	Ciclo episódico e interfaz de simulación	72
3.5	Instanciación del modelo Dec-POMDP	73
3.5.1	Observaciones y codificación logarítmico-lineal	74
3.5.2	Espacio de acciones y enmascaramiento dinámico	76
3.5.3	Función de recompensa compuesta y coordinación vecinal	77
3.6	Implementación algorítmica e hiperparámetros	79
3.6.1	Mecanismos de aprendizaje independiente y centralizado	79
3.7	Búsqueda de hiperparámetros (HPO)	81
3.7.1	Selección y congelamiento de las configuraciones finales	84
3.7.2	Infraestructura de cómputo y orquestación	85
3.8	Evaluación de rendimiento	86
4	Análisis y discusión de resultados	90
4.1	Resultados del ajuste de hiperparámetros	90
4.1.1	Diseño del proceso de ajuste	90
4.1.2	Resultados del ajuste y análisis de sensibilidad	90
4.1.3	Configuración seleccionada para la evaluación final	90
4.2	Resultados de los modelos finales	90
4.2.1	Criterios de evaluación y métricas de comparación	91
4.2.2	Desempeño en escenarios sintéticos: 3×1 , 3×3 , 4×4 heterogéneo	91
4.2.3	Desempeño en la red de carreteras de la Ciudad de Mendoza	91
4.3	Discusión e interpretación de resultados	91
5	Conclusiones	92
A	Arquitectura de software y patrones de diseño	93
A.1	Diseño orientado a objetos del entorno de simulación	93
A.2	Jerarquía de funciones de recompensa y patrón Strategy	95
A.3	Patrones de diseño implementados	96

Índice de figuras

1.1	Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.	1
2.1	Movimientos vehiculares, fases y cruces peatonales	7
2.2	Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.	8
2.3	Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).	11
2.4	Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de pelotones. Las curvas representan situaciones conceptuales y no una distribución empírica particular.	13
2.5	Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.	15
2.6	Correspondencia entre la circulación real y su representación en SUMO	16
2.7	Interfaz sumo-gui durante una simulación microscópica	18
2.8	Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.	20
2.9	Interacción agente-entorno en un proceso de decisión de Markov [SB18].	24
2.10	Esquema de un método basado en valor	26
2.11	Esquema de un método basado en política	27
2.12	Arquitectura actor-crítico aplicada al control semafórico	29
2.13	Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación.	30
2.14	Arquitectura conceptual de una red neuronal <i>feedforward</i> con dos capas ocultas.	31

2.15	Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de <i>bootstrapping</i> del blanco Diferencia temporal (<i>Temporal-Difference</i>) (TD). Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS ⁺ 15].	34
2.16	Red de política para un espacio discreto de acciones	36
2.17	Interacción agente-entorno en MARL cooperativo	41
2.18	Evolución de las políticas de dos agentes bajo WoLF-PHC en Piedra, Papel o Tijera	43
2.19	Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.	45
2.20	Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m} , que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.	51
3.1	Flujo metodológico de la investigación experimental. Los siete recuadros representan, de arriba hacia abajo, las etapas de modelado de redes, formulación del problema, desarrollo del entorno, integración algorítmica, optimización de hiperparámetros, ejecución distribuida y evaluación estadística; las flechas indican la dependencia secuencial entre ellas.	62
3.2	Codificación por carril empleada para construir la observación local. Cada entrada de espera se transforma y conserva su correspondencia espacial con el carril de origen.	76
3.3	Ejemplo de la mezcla logarítmica-lineal utilizada por Encoder . La curva se interpreta como una transformación de la espera acumulada, no como una trayectoria de entrenamiento.	76
A.1	Diagrama de clases del entorno de simulación y de sus relaciones principales.	94
A.2	Especificación UML del módulo de recompensas.	95

Índice de tablas

2.1	Variables de tránsito macroscópicas y microscópicas	10
2.2	Archivos de entrada y configuración empleados en una simulación de SUMO.	21
2.3	Comparación sintética de familias algorítmicas PPO en entornos multia- gente cooperativos.	53
3.1	Especificación de las tecnologías y herramientas utilizadas en la investigación.	64
3.2	Características verificadas de los escenarios sintéticos.	66
3.3	Señales de recompensa disponibles en el entorno.	78
3.4	Métricas utilizadas para presentar y controlar la evaluación final.	88
4.1	Métricas operacionales del escenario canónico v14 de la Ciudad de Mendoza bajo control de tiempo fijo.	91

Siglas

A2C *Advantage Actor-Critic* (método actor–crítico basado en una estimación de ventaja).

AFAC Asociación de Fábricas Argentinas de Componentes.

API Interfaz de programación de aplicaciones (*Application Programming Interface*).

ASHA Algoritmo de reducción sucesiva de configuraciones asíncrono (*Asynchronous Successive Halving Algorithm*).

BFS Búsqueda en anchura (*Breadth-First Search*).

CCAD Centro de Cómputo de Alto Desempeño.

CCAR Centro de Cómputo de Alto Rendimiento.

CI Tasa de completitud de los vehículos insertados (*Completion of Inserted Vehicles*).

CLIP Objetivo sustituto recortado (*Clipped surrogate objective*).

CO Monóxido de carbono.

CO₂ Dióxido de carbono.

CONICET Consejo Nacional de Investigaciones Científicas y Técnicas.

CR Tasa de completitud de viajes (*Completion Rate*).

CTDE Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*).

CVaR Valor en riesgo condicional (*Conditional Value at Risk*).

DAG Grafo acíclico dirigido (*Directed Acyclic Graph*).

DDPG *Deep Deterministic Policy Gradient* (gradiente determinista de política con redes profundas).

Dec-POMDP Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*).

DLR Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*).

DQN *Deep Q-Network* (red neuronal profunda que aproxima valores de acción).

DRL Aprendizaje por refuerzo profundo (*Deep Reinforcement Learning*).

E/S Entrada/salida (*Input/Output*).

FCEFN Facultad de Ciencias Exactas, Físicas y Naturales.

FHWA Administración Federal de Carreteras (*Federal Highway Administration*).

FLIR Tecnología infrarroja con barrido hacia adelante (*Forward Looking Infrared*).

GAE *Generalized Advantage Estimation* (estimación de la ventaja mediante errores temporales ponderados).

GEH Estadístico empírico de comparación de flujos viales de Geoffrey E. Havers.

GUI Interfaz gráfica de usuario (*Graphical User Interface*).

HAPPO *Heterogeneous-Agent Proximal Policy Optimization* (PPO con actualización secuencial de agentes heterogéneos).

HARL Aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*).

HATRPO *Heterogeneous-Agent Trust Region Policy Optimization* (optimización con región de confianza para agentes heterogéneos).

HBEFA Manual de factores de emisión para transporte por carretera (*Handbook Emission Factors for Road Transport*).

HC Hidrocarburos no combustionados.

HPC Computación de alto rendimiento (*High-Performance Computing*).

HPO Optimización de hiperparámetros (*Hyperparameter Optimization*).

IA Inteligencia artificial (*Artificial Intelligence*).

IDQN *Independent Deep Q-Network* (DQN independiente por agente).

IL Aprendizaje independiente (*Independent Learning*).

IPPO *Independent Proximal Policy Optimization* (PPO independiente por agente).

IQM Media intercuartílica (*Interquartile Mean*).

IR Tasa de inserción (*Insertion Rate*).

ITE Instituto de Ingenieros de Transporte (*Institute of Transportation Engineers*).

KL Divergencia de Kullback–Leibler.

MADDPG *Multi-Agent Deep Deterministic Policy Gradient* (extensión multiagente de DDPG).

MADRL Aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*).

MAPPO *Multi-Agent Proximal Policy Optimization* (PPO multiagente con información centralizada durante el entrenamiento).

MARL Aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*).

MDP Proceso de decisión de Markov (*Markov Decision Process*).

ML Aprendizaje automático (*Machine Learning*).

MLP Perceptrón multicapa (*Multi-Layer Perceptron*).

NFS Sistema de archivos de red (*Network File System*).

NOx Óxidos de nitrógeno.

OD Origen-destino (*Origin-Destination*).

OOD Fuera de distribución (*Out-of-Distribution*).

OSM OpenStreetMap.

PHEMlight Modelo microscópico de emisiones de vehículos livianos y pesados (*Passenger car and Heavy duty Emission Model light*).

POMDP Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*).

POSIX Interfaz portable de sistemas operativos (*Portable Operating System Interface*).

PPO *Proximal Policy Optimization* (optimización mediante un objetivo sustituto recordado).

QMIX *Q-Mixing Network* (red que combina valores de acción individuales).

ReLU Unidad lineal rectificadora (*Rectified Linear Unit*).

RL Aprendizaje por refuerzo (*Reinforcement Learning*).

SGD Descenso de gradiente estocástico (*Stochastic Gradient Descent*).

SL Aprendizaje supervisado (*Supervised Learning*).

SLURM Gestor simple de recursos para administración de Linux (*Simple Linux Utility for Resource Management*).

SUMO Simulación de Movilidad Urbana (*Simulation of Urban MObility*).

TAZ Zona de análisis de tránsito (*Traffic Analysis Zone*).

TCP Protocolo de control de transmisión (*Transmission Control Protocol*).

TD Diferencia temporal (*Temporal-Difference*).

TPE Estimador de densidad de Parzen estructurado por árbol (*Tree-structured Parzen Estimator*).

TraCI Interfaz de control del tránsito (*Traffic Control Interface*).

TRPO *Trust Region Policy Optimization* (optimización con una restricción de región de confianza).

UL Aprendizaje no supervisado (*Unsupervised Learning*).

UML Lenguaje unificado de modelado (*Unified Modeling Language*).

UTM Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*).

VDN *Value-Decomposition Networks* (redes de descomposición del valor conjunto).

VF Función de valor (*Value Function*).

WGS84 Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*).

WoLF-PHC *Win or Learn Fast Policy Hill-Climbing* (PHC con tasas distintas según el desempeño).

XML Lenguaje de marcado extensible (*Extensible Markup Language*).

Glosario

G_t (**retorno**) Suma acumulada, usualmente descontada, de las recompensas futuras a partir del instante t .

G_{wait} (**coeficiente de Gini de esperas**) Índice adimensional entre 0 y 1 que cuantifica el grado de desigualdad en la distribución espacial de los tiempos de espera medios entre las intersecciones semaforizadas de la red.

$M_{i_1:m}$ (**factor de reponderación HAPPO**) Factor acumulativo de razones de probabilidad que repondera la ventaja conjunta en la actualización secuencial de HAPPO para reflejar los cambios de política de los agentes predecesores.

$Q(s, a)$ (**valor de acción**) Retorno esperado de ejecutar la acción a en el estado s y continuar posteriormente de acuerdo con una política determinada.

R_t^{target} (**retorno objetivo**) Estimación de retorno utilizada como referencia para ajustar la predicción de valor del crítico en el instante t .

$S[\pi_\theta]$ (**entropía de la política**) Medida de la dispersión de la distribución de acciones de la política; su inclusión en el objetivo favorece la exploración.

$V(s)$ (**valor de estado**) Retorno esperado al comenzar en el estado s y seguir una política determinada, promediando las acciones que dicha política selecciona.

W_{net} (**demora acumulada global**) Métrica operacional agregada que cuantifica la suma de los tiempos de espera de todos los vehículos detenidos en la red durante la ventana de evaluación, expresada en vehículo·segundo (veh s).

α (**tasa de aprendizaje**) Paso de actualización que regula la magnitud del ajuste de los parámetros o valores en cada iteración de aprendizaje. En arquitecturas actor-crítico se especializa mediante subíndices, tales como α_θ para la red del actor y α_ϕ (o η) para el crítico.

δ_t (**error de diferencia temporal**) Diferencia entre la recompensa más el valor descontado del estado siguiente y la estimación de valor del estado actual.

- ϵ_{clip} (**margen de recorte**) Hiperparámetro de PPO que delimita el intervalo utilizado para recortar la razón entre la probabilidad nueva y la anterior de una acción.
- ϵ_{exp} (**exploración**) Probabilidad o coeficiente que controla la selección de acciones exploratorias frente a las acciones favorecidas por la política o la función de valor.
- η (**tasa del optimizador**) Tamaño de paso empleado por el optimizador en la actualización de parámetros mediante descenso de gradiente.
- γ (**factor de descuento**) Parámetro en el intervalo $[0, 1)$ que pondera la contribución de las recompensas futuras al retorno esperado.
- $\hat{A}_t$ (**ventaja estimada**) Estimación del valor de la función de ventaja en el paso temporal t (habitualmente calculada mediante GAE), que cuantifica cuánto supera o desmejora el rendimiento de la acción ejecutada respecto del valor basal esperado del estado.
- λ (**suavizado de la ventaja**) Hiperparámetro que pondera errores de diferencia temporal de distintos horizontes al estimar la ventaja y equilibra sesgo y varianza.
- $\mathcal{L}(\theta)$ (**función de pérdida**) Función escalar que cuantifica el error del modelo y que se minimiza para ajustar los parámetros θ .
- ϕ (**parámetros del crítico**) Vector de parámetros entrenables de la red que estima la función de valor del crítico.
- π_θ (**política parametrizada**) Distribución de probabilidad sobre las acciones condicionada por el estado u observación y controlada por el vector de parámetros entrenables θ .
- ρ_{coop} (**ponderación cooperativa**) Coeficiente en el intervalo $[0, 0.95]$ que pondera la combinación convexa entre la señal de recompensa local de una intersección y el promedio de su vecindario topológico.
- θ (**parámetros de la política**) Vector de parámetros entrenables que determina la distribución de acciones de la política del actor.
- a_i, A_t (**acción**) Decisión ejecutada por un agente. En formulaciones temporales, A_t denota la variable aleatoria de acción en el paso t ; en entornos multiagente, a_i representa la acción local del agente i y $\mathbf{a} = (a_1, \dots, a_n)$ la acción conjunta. En el control semafórico corresponde a la fase asignada a la intersección.
- c_1 (**coeficiente de valor**) Peso asignado al término de pérdida de la función de valor en el objetivo combinado de PPO.

c_2 (**coeficiente de entropía**) Peso asignado al término de entropía que incentiva la exploración en el objetivo combinado de PPO.

o_i (**observación local**) Información disponible para el agente i durante la ejecución descentralizada. Se obtiene a partir de los sensores o variables observables de la intersección que controla.

r_t (**recompensa**) Señal escalar recibida en el paso temporal t después de ejecutar una acción. Se utiliza para orientar la actualización de la política o de la función de valor.

s (**estado global**) Representación conjunta de las variables relevantes del entorno en un instante determinado. En el modelo multiagente puede incluir la información de todas las intersecciones y sus corrientes vehiculares.

Batch (lote) Conjunto de muestras o transiciones procesadas conjuntamente durante una operación de entrenamiento..

Binding Mecanismo que conecta una biblioteca escrita en un lenguaje de programación con otro lenguaje para poder utilizar sus funciones.

Bootstrap Procedimiento de remuestreo con reemplazo utilizado para aproximar la incertidumbre de un estimador a partir de las observaciones disponibles.

Buffer Estructura que almacena temporalmente datos para su posterior procesamiento o remuestreo en algoritmos de aprendizaje.

Epoch (época) Recorrido completo de optimización sobre el conjunto de datos recolectado para una iteración; puede reutilizarse durante varios *epochs*.

Logit Puntuación real sin normalizar que una red de política produce para una acción antes de convertirla en probabilidad.

Minibatch (minilote) Subconjunto reducido de muestras extraído de un *batch* utilizado para estimar el gradiente y actualizar los parámetros de un modelo durante el entrenamiento.

Replay buffer Estructura que almacena experiencias previas para remuestrearlas durante el entrenamiento de un método basado en valor.

Rollout Secuencia ordenada de transiciones generada mientras una política interactúa con el entorno.

Softmax Función que transforma un conjunto de puntuaciones reales en probabilidades no negativas cuya suma es uno.

Teleport (SUMO) Mecanismo específico de SUMO que retira temporalmente un vehículo bloqueado y lo reinserta posteriormente para evitar que la simulación quede detenida.

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semafóricas programadas en una intersección particular.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Episodio Secuencia de interacción que comienza al reiniciar el entorno y termina cuando se alcanza una condición terminal o el horizonte máximo definido.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso a uno o más movimientos o corrientes vehiculares compatibles entre sí para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Iteración de entrenamiento Ciclo completo en el que se recolectan datos con una política y se realizan una o más actualizaciones de sus parámetros.

Marco de trabajo (*framework*) Conjunto reutilizable de interfaces, componentes y reglas que facilita la construcción y la integración de un sistema de software.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

Serialización Conversión de un objeto o mensaje a una representación que puede almacenarse o transmitirse y luego reconstruirse.

Telemetría Conjunto de mediciones que el simulador registra durante la ejecución, como velocidades, posiciones, detenciones y tiempos de espera.

Transición (Aprendizaje por Refuerzo) Registro de un paso de interacción que contiene el estado u observación actual, la acción ejecutada, la recompensa recibida, el estado siguiente y, cuando corresponde, un indicador de terminación.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

Introducción

En las últimas décadas, el crecimiento del parque automotor y la urbanización sostenida han convertido a la congestión vehicular en uno de los principales desafíos para la movilidad urbana. Según los reportes de flota vehicular de Argentina elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y tomando 2012 como línea base, el volumen de vehículos aumentó un 21,7% en 2017. Esta tendencia continuó durante la última década y alcanzó un incremento acumulado del 44,4% en 2025 (véase la figura 1.1) [Aso18, Aso26].

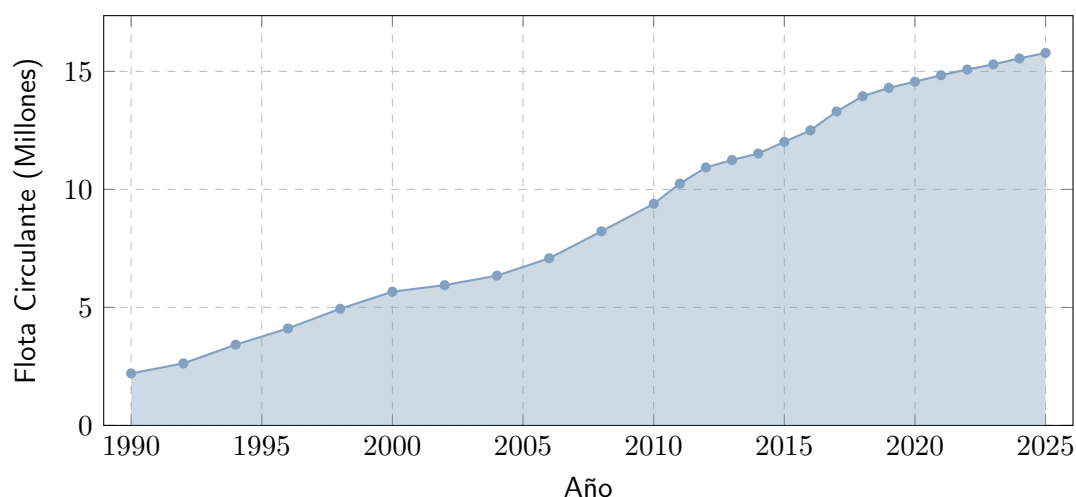


Figura 1.1: Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.

Como refleja la figura 1.1, la flota vehicular activa en Argentina experimentó una expansión sostenida entre 1990 y 2025, pasando de aproximadamente 2,2 millones a 15,78 millones de unidades operativas [Aso18, Aso26]. En los reportes de AFAC, la flota circulante comprende automóviles y vehículos comerciales livianos y pesados en condiciones de circular, excluyendo acoplados y remolques; representa una estimación del parque activo bajo esa definición, no el total histórico de unidades patentadas ni la cantidad de viajes realizados. Este incremento acumulado ejerce una presión creciente sobre redes viales consolidadas que poseen una capacidad física fija, agudizando la formación de colas y demoras en las intersecciones urbanas críticas.

Las congestiones vehiculares deterioran las condiciones de operación de la red e incrementan las demoras y las detenciones de los vehículos [CyMR18]. En el modelado microscópico, las emisiones de gases como el monóxido de carbono (CO), el dióxido de carbono (CO₂), los óxidos de nitrógeno (NO_x) y los hidrocarburos no combustionados (HC) se estiman a partir de variables cinemáticas, entre ellas la velocidad, la aceleración y el tiempo en ralentí [DLR26]. Este problema es especialmente relevante en ciudades con alta densidad vehicular. En este trabajo se utilizan escenarios sintéticos y un escenario urbano restringido por datos: el microcentro de la Ciudad de Mendoza, delimitado por las avenidas San Martín, Colón, Belgrano y Las Heras.

Frente a la congestión, una alternativa que no requiere modificaciones importantes de la infraestructura consiste en optimizar la gestión del tránsito mediante sistemas de control semafórico adaptativos. Los enfoques tradicionales pueden presentar limitaciones para responder a variaciones en el flujo vehicular [PW16]. En este contexto, los avances en Inteligencia artificial (*Artificial Intelligence*) (IA) y en particular el Aprendizaje por refuerzo (*Reinforcement Learning*) (RL) permiten modelar los semáforos como agentes que aprenden políticas de control a partir de la interacción con el entorno. Las acciones corresponden a cambios de fase y la recompensa puede definirse, por ejemplo, como el negativo del tiempo de espera acumulado. Diversos estudios han mostrado la viabilidad de este enfoque, desde algoritmos tabulares aplicados a una sola intersección [Wie00] hasta modelos de redes coordinadas basados en redes neuronales profundas y control multiagente [W⁺19].

Sin embargo, la aplicación simultánea de este enfoque en múltiples intersecciones introduce desafíos adicionales. Un esquema centralizado puede resultar poco escalable, pues el espacio de acciones conjunto crece multiplicativamente con el número de intersecciones [ACS24]. En una formulación de MARL con Aprendizaje independiente (*Independent Learning*) (IL), cada semáforo optimiza su propia intersección sin considerar explícitamente el efecto de sus decisiones sobre el resto de la red [AS22]. Además, como todos los agentes aprenden y modifican sus políticas al mismo tiempo, cada uno observa un entorno no estacionario. Esto dificulta la convergencia de los algoritmos de agente único [FNF⁺17].

Por esta razón, este trabajo aborda el problema desde el marco del MARL cooperativo. Cada controlador aprende a gestionar su propia intersección, pero sus decisiones también buscan evitar que las colas y las demoras se trasladen hacia las intersecciones vecinas. En lugar de asignar una única recompensa idéntica a todos los agentes, el sistema entrega a cada uno una señal que refleja tanto la evolución de la espera local como la de su entorno inmediato. Durante el entrenamiento de las variantes centralizadas, el crítico incorpora además información conjunta de la red. De este modo, cada controlador recibe una señal vinculada con su área de influencia, sin perder de vista el funcionamiento de la red vial en su conjunto.

En este contexto, el objetivo general de este trabajo es desarrollar y evaluar un sistema de control semafórico basado en aprendizaje por refuerzo multiagente para mejorar la operación del tránsito en redes urbanas. En particular, la investigación compara el desempeño de algoritmos de entrenamiento centralizado con ejecución descentralizada (MAPPO y HAPPO) con esquemas de aprendizaje independiente (IPPO e IDQN) bajo condiciones controladas, con el fin de evaluar estrategias de coordinación cooperativa y de tratamiento de la heterogeneidad entre intersecciones. La evaluación empírica se lleva a cabo mediante simulaciones microscópicas en escenarios sintéticos de complejidad creciente y en la red vial modelada del microcentro de la Ciudad de Mendoza, utilizando el control de tiempo fijo como ancla operacional de referencia.

La propuesta incorpora una codificación de la espera acumulada por carril mediante una transformación logarítmico-lineal en I niveles. La resolución no uniforme distingue con mayor detalle valores bajos de esa medida y acota los valores representados a partir de un umbral. La elección de cómo representar el tránsito influye en el problema de aprendizaje y cuenta con antecedentes en control semafórico [GR18]. En este trabajo, la codificación se utiliza en todos los controladores; determinar si acelera el aprendizaje requeriría compararla con otras representaciones.

También se estudia una adaptación de HAPPO para intersecciones con distintas geometrías, fases disponibles y demandas. A diferencia de la actualización de actores utilizada en MAPPO, esta adaptación los actualiza secuencialmente y repondera las ventajas individuales según los cambios de política precedentes [KCW⁺22]. El crítico centralizado incorpora información conjunta durante el entrenamiento. La comparación con MAPPO, IPPO y IDQN examina el desempeño de procedimientos completos en esos escenarios, sin atribuir de antemano una ventaja a HAPPO ni aislar el efecto de su actualización secuencial.

1.1 Motivación

La motivación de este trabajo se centra en estudiar cómo coordinar controladores que aprenden en una misma red vial. El control adaptativo basado en aprendizaje por refuerzo permite ajustar decisiones a las condiciones observadas [MMLK25]. En una red

de múltiples intersecciones, interesa examinar cómo esa adaptación puede incorporar los efectos de las decisiones locales sobre los cruces vecinos.

Las diferencias de demanda, geometría y fases disponibles entre intersecciones motivan, además, el estudio de políticas que puedan adaptarse a esas condiciones. La comparación de MAPPO y HAPPO con los esquemas independientes IPPO e IDQN permite examinar distintas formas de aprendizaje y coordinación bajo una misma formulación del problema.

1.2 Organización del documento

El presente trabajo se estructura de la siguiente manera:

- **Capítulo 1 – Introducción:** contextualiza la problemática de la congestión vehicular urbana, define los objetivos del proyecto y expone las motivaciones que impulsan esta investigación.
- **Capítulo 2 – Marco Teórico:** presenta los conceptos fundamentales de la ingeniería de tránsito, la teoría de procesos de decisión de Markov, las redes neuronales profundas y los paradigmas de aprendizaje por refuerzo multiagente.
- **Capítulo 3 – Metodología:** detalla el diseño metodológico y experimental, la calibración de la demanda vial mediante el estadístico Estadístico empírico de comparación de flujos viales de Geoffrey E. Havers (GEH), el modelado del microcentro de la Ciudad de Mendoza en Simulación de Movilidad Urbana (*Simulation of Urban MObility*) (SUMO) y la formulación algorítmica de los controladores.
- **Capítulo 4 – Resultados:** presenta la evaluación empírica y comparativa de los algoritmos de control sobre redes sintéticas y el escenario urbano restringido por datos.
- **Capítulo 5 – Conclusiones:** sintetiza los hallazgos principales, discute las limitaciones del modelo y propone direcciones de investigación futuras.
- **Apéndice A – Arquitectura de Software:** describe las especificaciones orientadas a objetos, los diagramas de clases Lenguaje unificado de modelado (*Unified Modeling Language*) (UML) y los patrones de diseño aplicados en el marco computacional `marlTLC`.

2

Marco teórico

Este capítulo presenta los fundamentos teóricos del trabajo y vincula la ingeniería de tránsito con el aprendizaje por refuerzo para la toma de decisiones secuenciales en redes viales. Estos conceptos permiten modelar una red semafórica como un entorno estocástico y formular políticas de control adaptativo con agentes homogéneos y heterogéneos.

Primero se introducen los conceptos de ingeniería de tránsito y simulación (sección 2.1). Luego se desarrollan los fundamentos del aprendizaje por refuerzo y su integración con redes neuronales (sección 2.2), seguidos de su extensión al caso multiagente y de los algoritmos MAPPO y HAPPO (sección 2.3). Finalmente, se revisan los antecedentes recientes de control semafórico multiagente según sus esquemas de coordinación, representación del estado y tratamiento de la heterogeneidad (subsección 2.3.7).

2.1 Ingeniería de tránsito y simulación

La ingeniería de tránsito es una rama de la ingeniería de transporte, tradicionalmente vinculada con la ingeniería civil. Se ocupa de la planificación, el diseño geométrico y la operación segura y eficiente del tránsito en calles, carreteras y redes viales, de acuerdo con manuales especializados como el *Traffic Engineering Handbook* [PW16] y con la literatura clásica del área [CyMR18]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa. Permiten describir el funcionamiento físico de la red, caracterizar la oferta y la demanda de circulación e identificar las variables sobre las que puede intervenir el sistema de control.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en proveer y ampliar la capacidad física vial mediante la construcción de nuevas obras o el ensanche

de calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y geoméricamente inviable [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito, priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

Estas magnitudes cumplen una función doble en el modelo computacional: pueden formar parte de las observaciones y de la señal de recompensa de los agentes, y también se utilizan para evaluar el desempeño del controlador. Su definición permite vincular la descripción física del tránsito con la formulación posterior del problema de aprendizaje.

2.1.1 Terminología de tránsito

Antes de formular el problema de control, es necesario distinguir algunos términos que se utilizarán a lo largo del capítulo. En particular, se diferencian los movimientos que realizan los vehículos, las fases que habilitan esos movimientos y los intervalos durante los cuales se mantienen las indicaciones semafóricas [Fed08].

En este contexto, se emplearán las siguientes nociones:

- **Intersección (*intersection*), acceso (*approach*) y carril (*lane*):** Una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].
- **Movimiento vehicular y movimientos en conflicto:** Un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una salida determinada, por ejemplo, un giro o un movimiento directo [W⁺19]. Dos movimientos pueden generar puntos de conflicto cuando sus trayectorias se intersectan (véase la figura 2.2). Cuando la geometría y la señalización no permiten

proteger su operación simultánea, esos movimientos deben programarse en intervalos distintos por razones de seguridad [FA11].

- **Fase (*phase*), ciclo (*cycle*) e intervalo (*interval*):** Una fase es una configuración de señales que habilita simultáneamente un conjunto de movimientos compatibles [PW16]. El ciclo corresponde a la secuencia completa de fases necesaria para atender los movimientos de la intersección. Un intervalo es el período durante el cual las indicaciones semafóricas permanecen constantes [CyMR18].

La figura 2.1 ilustra estos conceptos en una intersección de cuatro accesos. Los números del 1 al 8 identifican movimientos vehiculares individuales y las etiquetas $P2$, $P4$, $P6$ y $P8$ identifican cruces peatonales del esquema. Esta numeración sigue de manera esquemática una convención habitual de los manuales de temporización semafórica; no representa una configuración experimental ni una asignación obligatoria de Fases (Ingeniería de Tránsito) para el caso de estudio. Las flechas, los accesos y las etiquetas de calles permiten distinguir trayectorias y sentidos, mientras que los cruces peatonales muestran otros usuarios que también deben considerarse al ordenar los movimientos. En este trabajo, la figura se utiliza para separar la noción geométrica de movimiento de la regla semafórica que habilita un conjunto de movimientos compatibles [Fed08]. Por ejemplo, dos movimientos que no comparten un punto de conflicto pueden incluirse en una misma fase, mientras que los que sí lo comparten deben atenderse en fases compatibles o sucesivas.

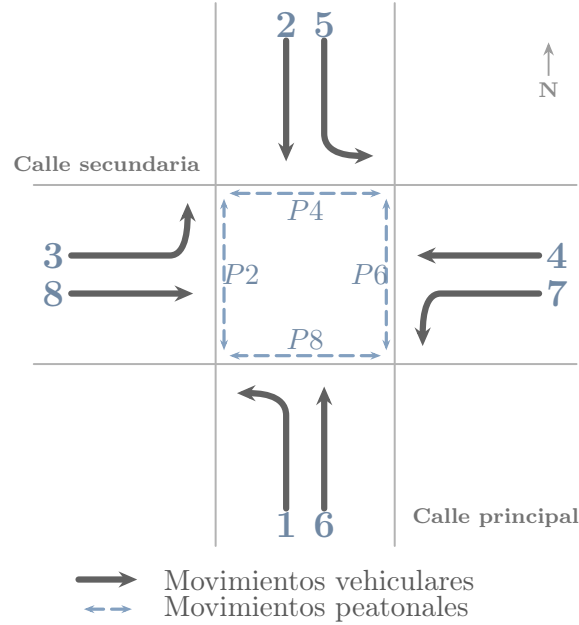


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de Administración Federal de Carreteras (*Federal Highway Administration*) (FHWA) [Fed08].

La figura 2.2 presenta tres formas esquemáticas de relación entre trayectorias. Hay

cruce cuando dos trayectorias se intersectan en un punto; *convergencia* cuando trayectorias distintas se unen en una misma corriente; y *divergencia* cuando una corriente se separa en trayectorias diferentes. Los puntos coloreados marcan el lugar de interacción que se analiza en cada caso. Cada relación puede originar un punto de conflicto según la geometría y la simultaneidad de los movimientos: por eso, la incompatibilidad no depende solo de que las líneas se relacionen, sino de que su operación conjunta comprometa la seguridad. En el modelo semafórico, estos casos se utilizan para determinar qué movimientos pueden habilitarse en una misma Fase (Ingeniería de Tránsito) y cuáles deben retenerse; la figura muestra relaciones geométricas, no una programación temporal concreta.

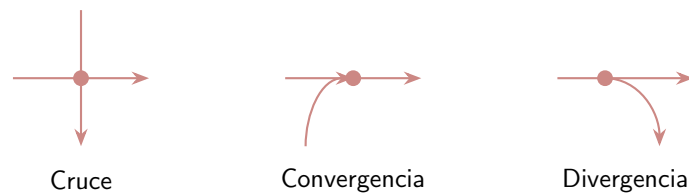


Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16]. Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí, como resume la figura 2.2. Por ello, su operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2 Fundamentos de la dinámica vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flujo vehicular,

y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópico, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son:

- **Flujo o tasa de flujo (q):** Representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora (veh/h), calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas [CyMR18].
- **Densidad o concentración (k):** Indica el número de vehículos que ocupan simultáneamente una longitud específica de vía, expresada en vehículos por kilómetro (veh/km), calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado [CyMR18]. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].
- **Velocidad media espacial (v_s):** Es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial [CyMR18].
- **Velocidad media temporal (v_t):** Representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado [CyMR18].

En el nivel microscópico, las variables asociadas describen el movimiento del vehículo individual y la separación entre vehículos consecutivos (véase la figura 2.3):

- **Velocidad puntual (v_i):** Es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.
- **Intervalo temporal (h):** Es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos,

por ejemplo, los paragolpes delanteros [CyMR18]. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.

- **Espaciamiento simple (s):** Es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo [CyMR18]. El espaciamiento promedio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

La figura 2.3 complementa estas definiciones con las magnitudes que se miden en el espacio y en el tiempo. El largo del vehículo l y la separación libre e son distancias, expresadas en metros (m); el espaciamiento s es la distancia entre puntos homólogos de vehículos consecutivos, también en m. El avance temporal β es el tiempo asociado al recorrido del largo del vehículo a la velocidad v , la brecha temporal g es el tiempo entre el paso del vehículo precedente y el siguiente respecto de puntos de referencia distintos, y el intervalo temporal h es el tiempo entre pasos de puntos homólogos; β , g y h se expresan en segundos (s). Las cotas inferiores representan relaciones espaciales y las superiores relaciones temporales; no son dos vehículos distintos ni dos mediciones independientes. En este trabajo, estas magnitudes sirven para vincular la descripción individual de los vehículos con las variables agregadas del flujo.

Tabla 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	Variable Principal	Relación / Conversión
Macroscópica	Flujo (q , veh/h)	$q = 3600/h_{\text{prom}}$
	Densidad (k , veh/km)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s , km/h)	$q = k \cdot v_s$
	Velocidad media temporal (v_t , km/h)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i , km/h)	Variable individual de cada vehículo
	Intervalo temporal (h , s)	Tiempo entre pasos de vehículos sucesivos
	Espaciamiento simple (s , m)	Distancia espacial entre paragolpes homólogos

La figura 2.3 presenta de manera conjunta estas distancias y tiempos entre vehículos, y permite interpretar cómo las magnitudes microscópicas describen la separación y el avance de una corriente.

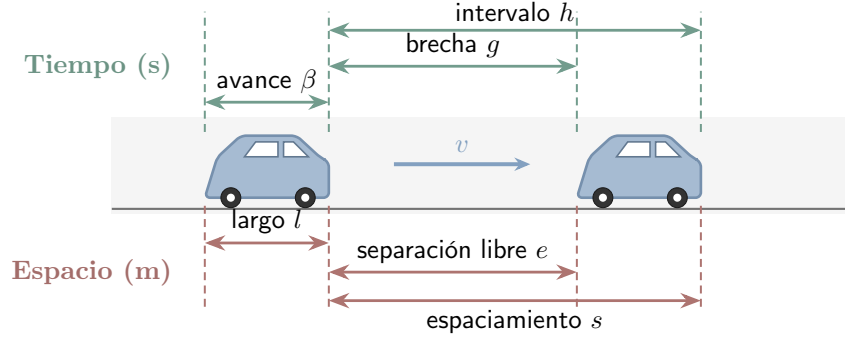


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamento (s).

Estas variables macroscópicas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \quad (2.1)$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. La ecuación (2.1) utiliza la velocidad media espacial, ya que relaciona el flujo y la cantidad de vehículos presentes en un tramo. La velocidad media temporal (v_t) es la media aritmética de las velocidades observadas en un punto durante un intervalo, mientras que v_s , asociada al promedio de las velocidades de una muestra espacial tomada sobre un tramo, se estima en la práctica como media armónica de las velocidades puntuales medidas en una sección durante un intervalo. Cuando existe variabilidad entre las velocidades, ambas magnitudes pueden diferir. La relación entre ellas se expresa mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \quad (2.2)$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. La ecuación (2.2) muestra que, dado que la varianza es no negativa ($\sigma_s^2 \geq 0$), se cumple $v_t \geq v_s$, con igualdad únicamente cuando todos los vehículos circulan a la misma velocidad. En una intersección semaforizada, una indicación roja puede reducir localmente la velocidad media y aumentar la acumulación de vehículos en los accesos. Estos efectos se manifiestan en la formación de colas y en el incremento de las demoras [CyMR18].

Métricas de ineficiencia operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** Representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo

libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.

- **Colas:** Corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de ser servidos. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección aguas arriba y propagar la congestión al resto de la red.
- **Detenciones:** Cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Estas magnitudes permiten describir el desempeño operacional de una intersección y de la red. En la formulación computacional, también pueden utilizarse para construir observaciones, señales de recompensa o criterios de evaluación del controlador.

Descripción probabilística del flujo y pelotones vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \quad (2.3)$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. La ecuación (2.3) formaliza esta probabilidad discreta. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \quad (2.4)$$

donde h es el intervalo temporal en segundos, q es el flujo en veh/h, y t es el umbral de tiempo analizado. La ecuación (2.4) expresa la probabilidad de que el intervalo supere

dicho umbral.

Esta representación probabilística supone independencia entre arribos sucesivos y resulta adecuada para flujos bajos o moderados en condiciones de flujo libre, sin influencia de semáforos cercanos aguas arriba [CyMR18]. El supuesto pierde validez en condiciones de congestión o en redes urbanas semaforizadas. En estos casos, los semáforos interrumpen el Flujo vehicular y agrupan los vehículos en pelotones (*platoons*), lo que introduce dependencia espacio-temporal entre arribos consecutivos. La figura 2.4 compara conceptualmente los intervalos temporales resultantes bajo ambas condiciones. En ella, $h_{\min}$ representa un umbral mínimo conceptual para el intervalo mostrado en la curva con pelotones; no es una estimación obtenida de datos de este trabajo. Las curvas y ese umbral tienen, por lo tanto, función ilustrativa y no constituyen una distribución empírica particular.

En este trabajo, la limitación de esos modelos analíticos motiva el uso de simulación microscópica para estudiar el control semafórico bajo condiciones variables. En SUMO, la formación, dispersión e interacción de los pelotones emergen de la dinámica simulada de los vehículos y de sus cambios de carril. El uso de distintos perfiles de demanda y semillas de simulación permite evaluar el comportamiento del controlador frente a variaciones en las condiciones de operación.

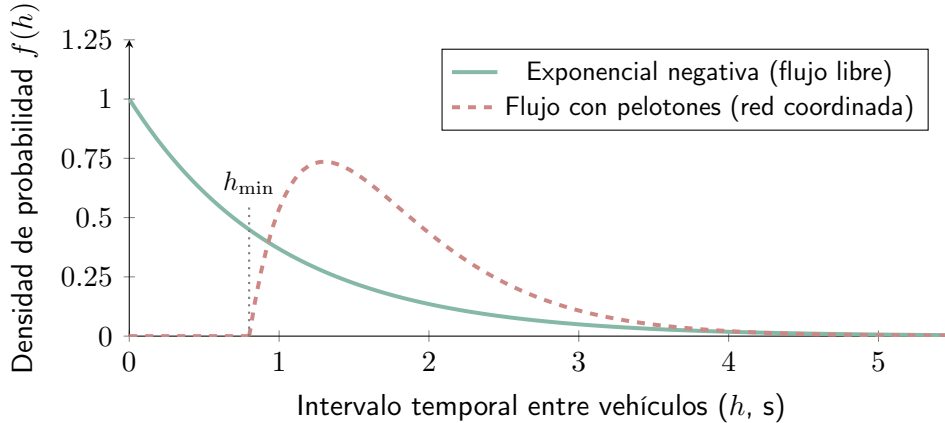


Figura 2.4: Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de pelotones. Las curvas representan situaciones conceptuales y no una distribución empírica particular.

La figura 2.4 se interpreta comparando la forma de ambas curvas. El eje horizontal representa el intervalo h entre vehículos y el eje vertical la densidad de probabilidad $f(h)$, es decir, la concentración relativa de intervalos alrededor de cada valor. En el modelo exponencial, la densidad disminuye desde el origen y asigna mayor frecuencia relativa a los intervalos breves. En la representación con pelotones, la densidad comienza después de $h_{\min}$ y alcanza un máximo alejado del origen, lo que ilustra que el arribo de un vehículo queda condicionado por la separación y la coordinación con los que lo preceden. La comparación permite reconocer por qué el supuesto de arribos independientes puede resultar

insuficiente en una red semaforizada.

2.1.3 Sistemas de control de tránsito

Para ordenar los movimientos en conflicto y regular el paso en las intersecciones urbanas se utilizan sistemas de control de tránsito [PW16]. El semáforo asigna el derecho de paso de manera alternada mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelo computacional, el semáforo actúa como el mecanismo mediante el cual el controlador interviene sobre la red. La selección de fases, la duración del verde efectivo y la coordinación entre intersecciones modifican la prioridad de paso y afectan la evolución de las colas y las demoras.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje. Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. Para el presente trabajo, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema semaforico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la capacidad y el verde efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la figura 2.5, q_{max} representa la tasa de descarga de saturación, g el intervalo de verde efectivo, y l_1 y l_2 las pérdidas inicial y final de servicio, respectivamente; el tiempo se expresa en s y la tasa de flujo en veh/h. La barra inferior distingue los intervalos de rojo, verde y amarillo, mientras que la curva superior muestra cómo la descarga alcanza y abandona el régimen de saturación. La descarga observada no depende solo de la duración nominal del verde,

sino también de pérdidas de arranque, transiciones y otras restricciones temporales del control. La figura es conceptual: estos parámetros no se estiman en este trabajo. En su lugar, los efectos se representan mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

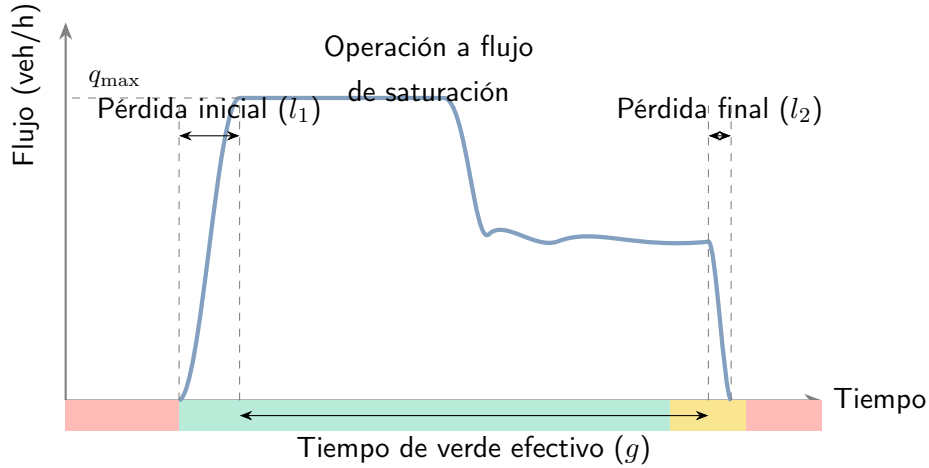


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

La figura 2.5 debe leerse como una relación temporal: las pérdidas l_1 y l_2 reducen el intervalo en que la fase puede descargar vehículos a la tasa $q_{\max}$. Por tanto, el verde efectivo g no equivale necesariamente al verde nominal programado en el controlador.

Clasificación de los controladores de semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo, la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite de seguridad y funcionamiento, como el verde mínimo ($g_{\min}$) y el verde máximo ($g_{\max}$) [PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4 Simulación de tránsito

Debido a la densidad y complejidad de las redes urbanas, la sincronización y optimización de los tiempos semafóricos requiere representar la evolución del tránsito bajo condiciones

variables. Los modelos analíticos y agregados resultan útiles bajo determinados supuestos, pero pueden ser limitados para describir la formación de colas y las interacciones locales durante la operación dinámica de una red [PW16]. De acuerdo con la taxonomía clásica [FA11], los modelos de tránsito pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los mesoscópicos representan entidades intermedias, como grupos o pelotones de vehículos; y los microscópicos modelan individualmente los vehículos, sus rutas y su evolución dentro de la red. Para el presente trabajo resulta especialmente relevante este último nivel, porque permite estudiar cómo las decisiones semafóricas afectan paso a paso la formación de colas, las detenciones y la interacción local entre vehículos.

La simulación de tránsito constituye una representación abstracta del fenómeno real. Su objetivo no es reproducir visualmente una escena particular, sino modelar los procesos que determinan la evolución de la circulación mediante reglas de comportamiento y variables observables. La calidad de los resultados depende de la representación de la red, la demanda, la flota, los modelos de comportamiento y las reglas de control. Por ello, la simulación debe entenderse como un entorno experimental sujeto a supuestos y parámetros, cuyos resultados requieren una evaluación operacional acorde con el objetivo del estudio. La correspondencia entre el fenómeno y su representación se resume en la figura 2.6: el tránsito real constituye la referencia y el simulador representa sus vehículos, red y reglas dinámicas para permitir experimentos controlados.



Figura 2.6: Correspondencia entre la circulación real y su representación microscópica en SUMO. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La figura 2.6 muestra, de forma sintética, la correspondencia entre una escena vial y

su representación microscópica en SUMO. Su función es ilustrar cómo los vehículos y la infraestructura se transforman en entidades computacionales; no constituye una validación del simulador ni una representación del escenario utilizado en este trabajo.

Para el presente trabajo resulta de mayor interés el nivel microscópico, porque representa explícitamente cada vehículo, su ruta y su movimiento dentro de la red [FA11]. SUMO pertenece a esta categoría. Es una suite libre y de código abierto iniciada en el Instituto de Sistemas de Transporte del Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*) (DLR), que permite modelar sistemas de tránsito intermodales, incluidos vehículos, transporte público y peatones. También ofrece herramientas para importar redes, calcular rutas, visualizar y evaluar simulaciones, además de interfaces para controlar su ejecución [DLR26].

En SUMO, la evolución vehicular se actualiza en pasos temporales discretos sobre una representación espacial continua. El entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR26].

La interfaz gráfica *sumo-gui* permite inspeccionar visualmente la red, los vehículos y algunos valores de la simulación durante la ejecución. Esta visualización cumple una función de apoyo para la inspección del escenario y la depuración del modelo; las métricas utilizadas para la evaluación se obtienen de las salidas de simulación y de las interfaces de control [DLR26]. La interfaz y su función de inspección se muestran en la figura 2.7; la Interfaz gráfica de usuario (*Graphical User Interface*) (GUI) no constituye la fuente de las métricas ni reemplaza su cálculo operacional.



Figura 2.7: Interfaz *sumo-gui* durante una simulación microscópica. El panel izquierdo presenta una vista general de la red; el derecho muestra información sobre un vehículo particular; el recuadro inferior grafica su velocidad y el superior muestra los valores de sus variables relacionadas. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La figura 2.7 reúne cuatro zonas de inspección. A la izquierda se observa la red completa, útil para localizar la parte del escenario que se está examinando. A la derecha se muestra una vista ampliada de la calzada y de un vehículo seleccionado, mientras que el gráfico superpuesto en la parte inferior representa la evolución temporal de su velocidad, expresada en metros por segundo. La franja inferior contiene el registro de mensajes y advertencias producidos durante la ejecución, y la barra superior agrupa los controles de avance, pausa, tiempo y escala de visualización. En conjunto, estos elementos permiten verificar visualmente la presencia de vehículos, su movimiento y posibles inconsistencias de configuración. En este trabajo la interfaz se usa con fines de inspección y depuración; la evaluación cuantitativa se realiza con las salidas de simulación y las interfaces de control [DLR26].

Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semafórico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio pueden evaluarse mediante sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles, completitud de viajes y propagación de congestión. En la formulación computacional, las variables de tránsito constituyen observaciones, las fases admisibles constituyen acciones y la evolución simulada constituye la transición del entorno. La definición operacional de estas variables y su procedimiento de cálculo se presentan en la metodología.

Representación microscópica de intersecciones en SUMO

La naturaleza microscópica de la simulación no se reduce a representar visualmente vehículos. Cada unidad modifica su velocidad y posición de acuerdo con modelos de seguimiento vehicular y cambio de carril, los parámetros de su tipo de vehículo y las restricciones de la red. De este modo, variables como la aceleración, la separación entre vehículos, la velocidad deseada y la disponibilidad de carriles intervienen en la formación y dispersión de colas. Los valores concretos utilizados para la flota y para estos parámetros pertenecen a la configuración experimental y se detallan en la metodología [DLR26].

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semafórico. Para el esquema conceptual de la figura 2.8, que adopta circulación por la derecha, se distinguen carriles, líneas de detención, conexiones entre carriles y estados de control semafórico [DLR26]. La fase representada es solo una ilustración de cómo pueden habilitarse movimientos directos opuestos; la ausencia visual de los estados *g* e *y* no implica que no existan en la lógica de SUMO ni que se excluyan de otras fases o transiciones:

- **Carriles entrantes (L_{in}) y salientes (L_{out}):** Los carriles entrantes conducen el flujo vehicular hacia la intersección, mientras que los carriles salientes reciben el flujo descargado tras cruzar el nodo.
- **Línea de detención (*stop line*):** Referencia operacional próxima al final del carril entrante, delante de la cual se detienen los vehículos cuando el movimiento no está habilitado.
- **Conexiones (*connections* o *links*):** Relaciones que conectan un carril entrante con un carril saliente y definen los movimientos posibles a través de la intersección. Cuando están controladas por un semáforo, se asocian con un índice de enlace y con un estado de señal. En SUMO, *G* representa verde con prioridad, *g* verde sin prioridad y sujeto a cesión, *y* amarillo y *r* rojo. La figura muestra únicamente una fase conceptual en la que los movimientos directos opuestos del eje horizontal están habilitados y los movimientos transversales o conflictivos permanecen retenidos.

Esta representación es compatible con la formulación algorítmica del problema, donde una fase define un conjunto de estados sobre los enlaces controlados. Una acción válida debe seleccionar una fase compatible con la geometría de la intersección y respetar las transiciones y restricciones de seguridad definidas por el entorno.

La figura 2.8 ilustra esos carriles, líneas de detención y conexiones en una fase conceptual; su función es relacionar la geometría de la red con las acciones semafóricas del modelo, no representar un programa experimental completo.

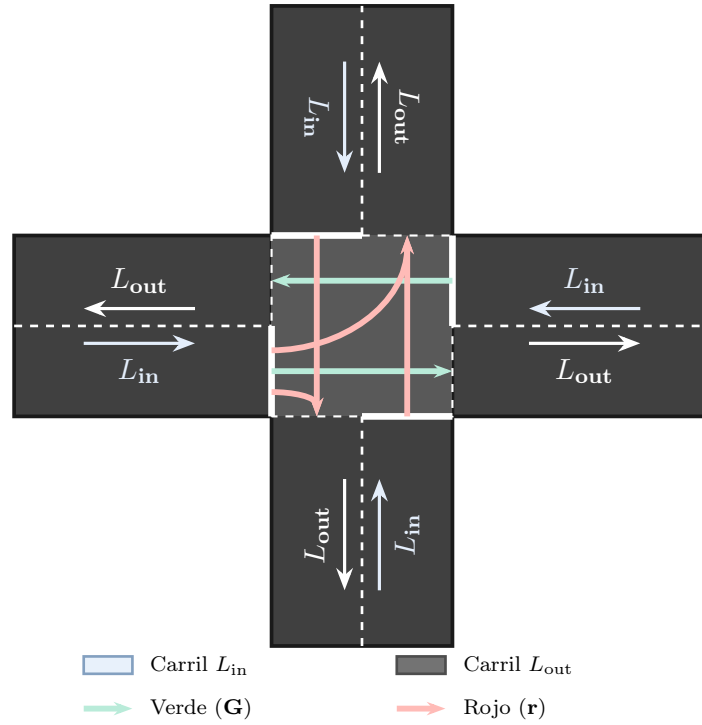


Figura 2.8: Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.

La figura 2.8 se lee desde los carriles de aproximación hacia el centro de la intersección. Las flechas blancas indican el sentido de circulación de L_{in} y L_{out} , y las líneas blancas continuas situadas antes del área de cruce representan las líneas de detención. Dentro de la intersección, las flechas verdes corresponden a conexiones habilitadas con prioridad y las rojas a conexiones retenidas. Por consiguiente, una Fase (Ingeniería de Tránsito) no modifica la geometría ni el sentido de los carriles: determina qué conexiones pueden utilizarse durante ese intervalo de control.

Componentes y configuración de SUMO

La suite SUMO reúne aplicaciones para construir redes, generar demandas, ejecutar simulaciones, visualizar sus resultados y controlar la ejecución mediante interfaces externas [DLR26]:

- **netconvert:** Aplicación que importa redes desde distintas fuentes y compila su representación en el formato nativo de SUMO. También permite reconstruir o ajustar elementos de la red, como las conexiones y los programas semafóricos.
- **netedit:** Editor gráfico para crear y modificar de forma interactiva nodos, calles, carriles, conexiones, restricciones de giro y elementos de control semafórico.
- **osmWebWizard:** Herramienta que automatiza la generación de un escenario a partir de un área seleccionada de OpenStreetMap (OSM). Se considera una alter-

nativa de importación y no reemplaza el procesamiento específico realizado con `netconvert`.

- **Interfaz de control del tránsito (*Traffic Control Interface*) (TraCI):** Interfaz que permite consultar el estado de una simulación en ejecución y modificarlo durante el avance paso a paso, incluyendo variables de vehículos, carriles, detectores y semáforos [DLR26].
- **libsumo:** Implementación embebida que ofrece la interfaz de programación de SUMO sin la comunicación mediante sockets de la modalidad TraCI estándar. Esta alternativa resulta útil para integrar el simulador con controladores externos y reducir la sobrecarga de comunicación.

Desde la perspectiva del modelado computacional, una simulación de SUMO se construye a partir de archivos Lenguaje de marcado extensible (*Extensible Markup Language*) (XML) de red, demanda, elementos adicionales y configuración [DLR26]. Los archivos Zona de análisis de tránsito (*Traffic Analysis Zone*) (TAZ) pueden utilizarse durante la estructuración espacial y la generación de demanda, mientras que las salidas de la simulación se solicitan mediante opciones de configuración específicas. Sus funciones generales se resumen en la tabla 2.2:

Tabla 2.2: Archivos de entrada y configuración empleados en una simulación de SUMO.

Archivo	Extensión	Función en el Modelo Computacional
Red Vial	<code>.net.xml</code>	Topología física, carriles, conexiones y lógicas semafóricas.
Rutas y Demanda	<code>.rou.xml</code>	Tipos de vehículos, viajes, flujos e itinerarios.
Zonas TAZ	<code>.taz.xml</code>	Zonas para estructurar espacialmente la demanda.
Elementos Adicionales	<code>.add.xml</code>	Detectores, tipos de vehículo y programas auxiliares.
Configuración	<code>.sumocfg</code>	Archivos de entrada, opciones de ejecución y salidas.

Métricas de evaluación operacional

La simulación microscópica produce salidas que permiten evaluar el desempeño del control desde la perspectiva de la ingeniería de tránsito. En este trabajo se consideran las siguientes categorías de indicadores:

- **Demora y tiempo de espera:** Cuantifican el tiempo adicional o el tiempo detenido que experimentan los vehículos durante su recorrido. Pueden analizarse de forma agregada en la red y desagregarse por intersección, acceso o agente, siempre que se mantenga una definición operacional consistente [CyMR18, DLR26].

- **Complejidad y flujo servido:** La cantidad de vehículos que completa su itinerario y su relación con la demanda generada permiten distinguir una reducción genuina de la demora de una disminución del flujo atendido.
- **Equidad espacial:** La distribución de las demoras entre intersecciones permite identificar si una política mejora el desempeño global a costa de concentrar la espera en accesos o agentes particulares.
- **Emisiones contaminantes (CO₂, CO, NO_x, HC):** SUMO estima las emisiones de dióxido de carbono (CO₂), monóxido de carbono (CO), óxidos de nitrógeno (NO_x) e hidrocarburos (HC) a partir del estado cinemático de los vehículos, su tipo y la clase de emisiones configurada [DLR26]. La masa total debe interpretarse junto con los vehículos completados y la distancia recorrida, o expresarse mediante una medida normalizada.
- **Indicadores de validez de la simulación:** Los mecanismos *teleport* de SUMO, los vehículos pendientes y otras incidencias de ejecución permiten detectar bloqueos o condiciones que comprometen la comparabilidad de una ejecución.

Las definiciones operacionales, fórmulas, unidades, ventanas de evaluación y procedimientos de agregación empleados en este trabajo se presentan en la metodología. Las recompensas y las pérdidas de optimización corresponden al entrenamiento de los algoritmos de aprendizaje y se desarrollan en las secciones dedicadas a RL y MARL.

En este marco, SUMO permite observar el estado de la red y modificar las fases semafóricas durante la ejecución mediante interfaces de control. Esta capacidad convierte el control semafórico en un problema de decisión secuencial, lo que motiva la introducción del aprendizaje por refuerzo en la siguiente sección [DLR26].

2.2 Aprendizaje por refuerzo (RL)

El Aprendizaje automático (*Machine Learning*) (ML) suele dividirse en tres paradigmas. En el Aprendizaje supervisado (*Supervised Learning*) (SL), el modelo se entrena con ejemplos etiquetados que indican la salida esperada. En el Aprendizaje no supervisado (*Unsupervised Learning*) (UL), se busca descubrir estructura en datos no etiquetados, por ejemplo mediante agrupamiento o reducción de dimensionalidad. El Aprendizaje por Refuerzo (*Reinforcement Learning*) (RL) aborda un problema distinto: una entidad aprende mediante interacción con un entorno qué acciones elegir para maximizar una recompensa acumulada [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre

se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el agente (*agent*), que aprende y toma decisiones, y el entorno (*environment*), que representa todo aquello externo al agente con lo cual este interactúa. En cada paso, el agente observa una situación, selecciona una acción y recibe una nueva observación junto con una recompensa: este ciclo constituye la interacción agente-entorno. El comportamiento está determinado por una política (*policy*), denotada por π , que especifica cómo elegir acciones a partir de los estados u observaciones disponibles. La señal de recompensa (*reward signal*) define el objetivo inmediato, mientras que la función de valor estima la conveniencia de estados o acciones en términos de recompensa futura. Sutton y Barto también distinguen el modelo del entorno, entendido como una descripción de sus transiciones y recompensas; cuando no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

En el control semafórico, el controlador de una intersección puede interpretarse como el agente y la red vial simulada en SUMO como el entorno. La selección de una fase constituye la acción, mientras que la evolución posterior del tránsito produce nuevas observaciones y una señal de recompensa que orienta el aprendizaje [DLR26].

Para fijar ideas, considérese un par de intersecciones vecinas. El agente observa localmente las colas y demoras de su intersección, selecciona una fase y modifica la corriente de vehículos que llegará a la siguiente. Esa consecuencia aguas abajo puede cambiar sus arribos y colas, y se refleja en una recompensa asociada al desempeño de la red. El retorno permite acumular esos efectos a lo largo de varias decisiones, en lugar de evaluar una fase solo por su consecuencia inmediata.

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la experiencia futura disponible para el aprendizaje [SB18].

2.2.1 Procesos de decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el Proceso de decisión de Markov (*Markov Decision Process*) (MDP). En un MDP finito, el agente y el entorno interactúan en pasos temporales discretos $t = 0, 1, 2, \dots$: en cada instante, el agente recibe $S_t \in \mathcal{S}$, selecciona $A_t \in \mathcal{A}(S_t)$ y el entorno emite $R_{t+1} \in \mathcal{R}$ y transita a S_{t+1} [LML⁺25]. La notación se ilustra en la figura 2.9.

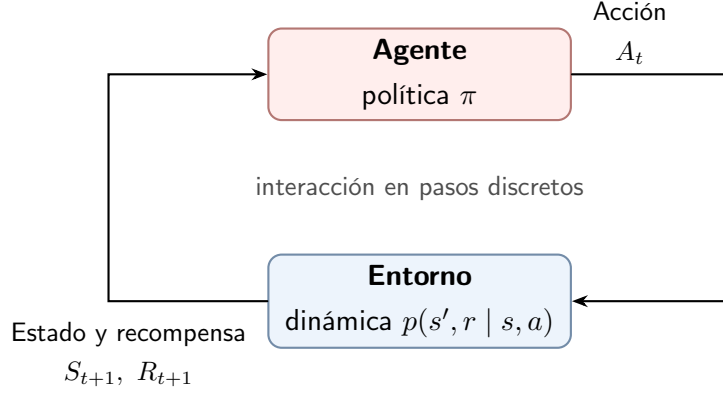


Figura 2.9: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r | s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s :

$$p(s', r | s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \quad (2.5)$$

En esta formulación, $S_t \in \mathcal{S}$ y $A_t \in \mathcal{A}$ denotan el estado y la acción en el instante t , mientras que $S_{t+1} \in \mathcal{S}$ y $R_{t+1} \in \mathbb{R}$ representan el estado sucesor y la recompensa escalar devueltos por el entorno. Si bien conceptualmente el estado compendia toda la información necesaria para determinar la dinámica, en el control semafórico el agente suele disponer de observaciones locales de tránsito que constituyen una representación parcial de dicho estado.

El proceso satisface la propiedad de Markov si la distribución condicional del próximo estado y recompensa depende exclusivamente del estado y la acción presentes, resultando condicionalmente independiente del historial de transiciones previas [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*) (POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t , que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las

recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.6)$$

En esta formulación, el retorno G_t acumula las recompensas futuras ponderadas por el factor de descuento $\gamma \in [0, 1)$. Un valor de γ cercano a cero privilegia la optimización a muy corto plazo, mientras que valores próximos a uno confieren mayor peso a las consecuencias remotas [ACS24]. En el control semafórico, esta acumulación temporal resulta indispensable para que el agente aprenda a prevenir condiciones de bloqueo o saturación en ciclos futuros, en lugar de limitarse a disipar la cola inmediata a expensas de la red.

2.2.2 Métodos basados en valor y basados en política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

Esta clasificación distingue dos formas de obtener una regla de decisión. Los métodos basados en política aprenden directamente una función que relaciona cada estado con una acción o con una distribución de probabilidad sobre las acciones. En cambio, los métodos basados en valor aprenden indirectamente la regla de decisión: estiman la conveniencia de los estados o de los pares estado–acción y, a partir de esas estimaciones, determinan qué alternativa conviene ejecutar [SB18]. En el control semafórico, las alternativas corresponden a las distintas fases admisibles de una intersección.

La figura 2.10 representa esta segunda alternativa. A partir del estado observado, la función de valor estima la conveniencia esperada de las acciones disponibles y la decisión se obtiene comparando esas estimaciones. El esquema corresponde directamente al caso de una función de valor de acción $Q(s, a)$; si se utiliza una función de valor de estado $V(s)$, la elección de la acción requiere valorar los estados sucesores que produciría cada alternativa. Por ello, $V(s)$ y $Q(s, a)$ cumplen funciones relacionadas, pero no intercambiables en la evaluación de los estados y en la selección de acciones.

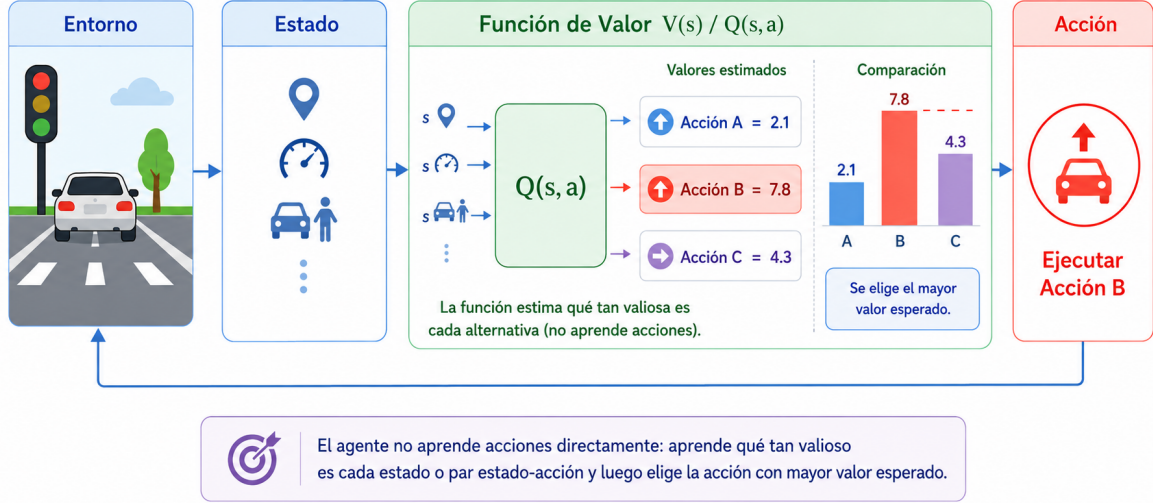


Figura 2.10: Esquema de un método basado en valor. A partir del estado observado s , la función de valor estima el retorno esperado de los pares estado-acción $Q(s, a)$. Las estimaciones para las acciones disponibles se comparan y se selecciona la alternativa con mayor valor esperado para ejecutarla en el entorno. La exploración puede incorporarse mediante estrategias como la selección ε -greedy [SB18].

En los métodos basados en valor, el objetivo es determinar la conveniencia de encontrarse en un estado o de ejecutar una acción concreta. Formalmente, esto se expresa mediante la función de valor de estado $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$ y la función de valor de acción $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$, las cuales satisfacen las relaciones recursivas de las ecuaciones de Bellman [SB18]. En la notación empleada en la figura 2.10, estas funciones se escriben habitualmente como $V(s)$ y $Q(s, a)$ cuando no es necesario hacer explícita la política respecto de la cual se calcula el valor.

Cuando el propósito es identificar la política óptima en lugar de evaluar una fija, se emplean las ecuaciones de optimalidad de Bellman:

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right]. \quad (2.7)$$

En la ecuación (2.7), la esperanza $\mathbb{E}$ promedia las transiciones estocásticas del entorno y la maximización sobre las fases admisibles a' formaliza el principio de optimalidad. En el ámbito semafórico, esta relación expresa el compromiso entre la recompensa inmediata obtenida por la fase activa y la calidad del estado vehicular resultante para las decisiones subsiguientes [SB18].

En ausencia de un modelo analítico de transición, los métodos de TD aproximan estas funciones a partir de la experiencia mediante *bootstrapping*. Un método canónico es Q-learning, cuya regla de actualización converge hacia la función óptima $q_*(s, a)$ en el caso tabular, bajo el supuesto de que todos los pares estado-acción continúen explorándose y

que la tasa de aprendizaje decrezca adecuadamente a lo largo del tiempo [SB18]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \quad (2.8)$$

donde $\alpha \in (0, 1]$ es la tasa de aprendizaje (*learning rate*) que pondera la magnitud del ajuste, R_{t+1} es la recompensa inmediata observada, y el término $\max_a Q(S_{t+1}, a)$ aproxima el retorno óptimo alcanzable desde el estado sucesor S_{t+1} .

Q-learning es un algoritmo *off-policy*: la política que genera los datos de interacción puede diferir de la política objetivo cuya función de valor se optimiza. Por el contrario, los métodos de gradiente de política operan generalmente de forma *on-policy*, utilizando transiciones producidas por la política en curso de optimización. Esta diferencia resulta decisiva al extender los algoritmos al entorno multiagente, donde las políticas de las intersecciones vecinas evolucionan simultáneamente.

En los métodos basados en política, la regla de decisión se parametriza directamente como $\pi_\theta(a | s) = \Pr\{A_t = a | S_t = s, \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables de la política. La política puede ser determinista, si asigna siempre la misma acción a un estado, o estocástica, si produce una distribución de probabilidad sobre las acciones posibles [SB18]. La figura 2.11 ilustra este último caso: para el estado observado, la política asigna probabilidades a cada fase admisible y la selección se efectúa mediante muestreo o según la alternativa más probable, de acuerdo con la regla de ejecución adoptada.

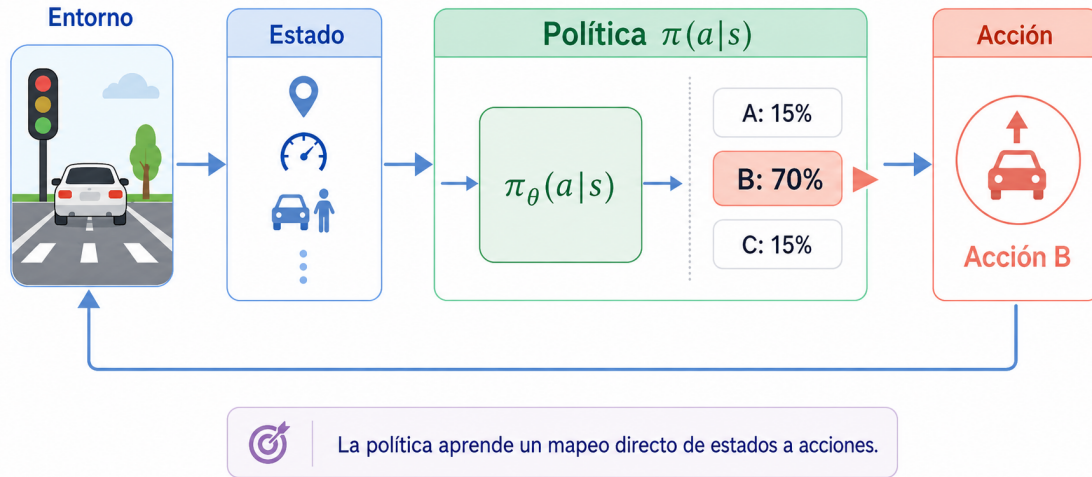


Figura 2.11: Esquema de un método basado en política. A partir del estado observado s , la política parametrizada $\pi_\theta(a | s)$ genera directamente una distribución de probabilidad sobre las acciones disponibles. La regla de ejecución puede seleccionar la alternativa de mayor probabilidad o muestrear una acción a partir de dicha distribución [SB18].

La optimización persigue maximizar el rendimiento esperado $J(\theta)$ mediante ascenso

de gradiente:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \quad (2.9)$$

donde $\nabla J(\theta_t)$ es el gradiente del rendimiento respecto de los parámetros y $\alpha > 0$ determina la magnitud del paso de actualización. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona el sustento teórico para estimar analíticamente $\nabla J(\theta_t)$ a partir de trayectorias muestreadas, ajustando la política para elevar la probabilidad de aquellas fases que conducen a menores demoras en la red [SB18].

Los métodos actor-crítico combinan una política parametrizada con una estimación de valor y actualizan ambos componentes de manera concurrente [KT03, SB18]. El actor representa la política $\pi_\theta(a \mid s)$ y determina la distribución utilizada para seleccionar la acción. El crítico no controla el entorno: estima, por ejemplo, el valor de estado $V_\phi(s)$ y produce una señal que indica si el resultado observado fue mejor o peor que lo esperado bajo la política vigente. Los parámetros θ y ϕ pueden optimizarse de forma independiente o compartir una representación común de las observaciones de tránsito.

En una formulación actor-crítico de un paso, después de ejecutar A_t en S_t , el entorno devuelve la recompensa R_{t+1} , el estado S_{t+1} y el indicador terminal d_{t+1} . El crítico calcula el error de diferencia temporal mediante:

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t), \quad (2.10)$$

En la ecuación (2.10), el factor $(1 - d_{t+1})$ anula la contribución del estado siguiente ante transiciones terminales. Cuando V_ϕ aproxima adecuadamente el valor real V^π , δ_t actúa como un estimador de un paso de la ventaja: un valor positivo refleja que la fase ejecutada reportó un desempeño superior al esperado en ese estado de tránsito, mientras que un valor negativo indica lo contrario. El crítico ajusta sus parámetros ϕ minimizando el error temporal, mientras que el actor utiliza esta misma señal δ_t para modular la actualización de la política:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t). \quad (2.11)$$

En la regla (2.11), $\alpha_\theta > 0$ es la tasa de aprendizaje del actor y $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ representa la dirección que incrementa la probabilidad de la acción elegida. Ponderar este gradiente por δ_t refuerza sistemáticamente las fases que superan la expectativa de demora y penaliza aquellas con resultados adversos. Aunque los algoritmos avanzados sustituyen δ_t por estimadores de ventaja multipaso como *Generalized Advantage Estimation* (estimación de la ventaja mediante errores temporales ponderados) (GAE) y procesan *batches* de transiciones en cada actualización, esta formulación elemental hace explícita la interacción entre ambos componentes.

La figura 2.12 sintetiza el flujo operativo entre actor y entorno, así como el circuito de evaluación mediante el cual el crítico genera la señal de retroalimentación compartida.

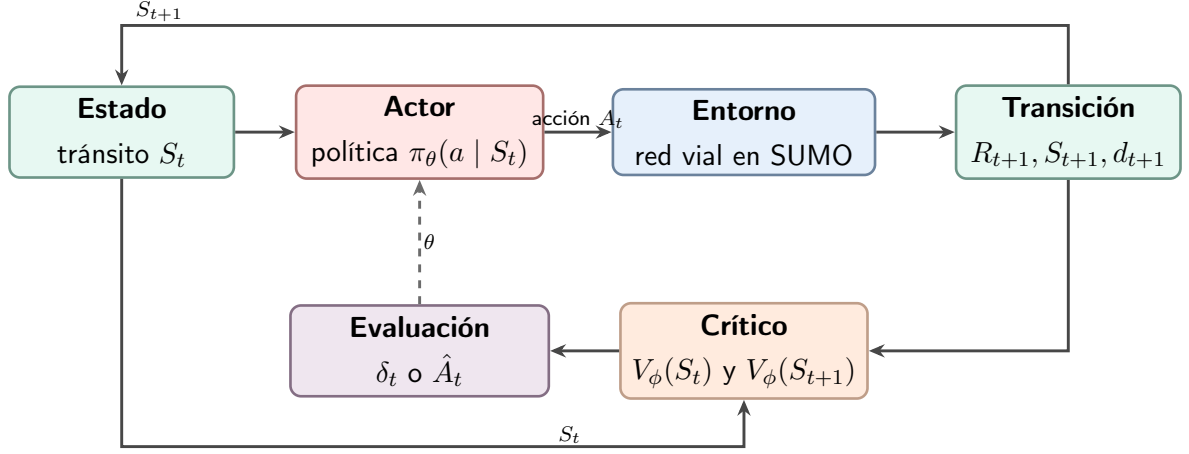


Figura 2.12: Arquitectura actor-crítico aplicada al control semafórico en la formulación de un MDP. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].

En síntesis, el estado resume las condiciones de tránsito disponibles para el agente, la acción corresponde a la selección de una fase, la recompensa refleja la consecuencia operativa inmediata y el valor estima su impacto a largo plazo. Cuando el espacio de estados es continuo o de gran escala, como ocurre en las redes viales urbanas, las funciones de valor y las políticas requieren aproximadores de alta capacidad representacional. A continuación se introducen las redes neuronales artificiales como el modelo adoptado para este fin, preservando intactos los fundamentos de RL expuestos hasta aquí.

2.2.3 Aprendizaje profundo y redes neuronales

Para estructurar este desarrollo, se expone en primer término la unidad neuronal básica y su composición en redes multicapa; luego se aborda su optimización mediante retropropagación y descenso de gradiente; finalmente, se analiza su integración en RL a través de DQN y PPO. Cabe destacar que la red neuronal actúa como un aproximador de funciones de alta capacidad, mientras que la dirección del aprendizaje proviene exclusivamente de las señales de recompensa generadas por la interacción con el entorno [SB18].

El aprendizaje profundo (*Deep Learning*) se enfoca en el entrenamiento de arquitecturas con múltiples capas de parámetros ajustables capaces de extraer representaciones jerárquicas y relaciones no lineales directamente de los datos [BB24, GBC16]. En este trabajo, esta capacidad se aprovecha para aproximar funciones de valor y políticas de control semafórico donde una formulación tabular resulta impracticable debido a la continuidad y dimensionalidad del estado del tránsito.

Aunque la denominación “neuronal” proviene de una analogía simplificada con modelos biológicos iniciales, una red artificial constituye un modelo matemático determinista cuyos parámetros se optimizan a partir de datos y gradientes de pérdida [GBC16].

La unidad neuronal y redes *feedforward*

En problemas de control con espacios de estados continuos o de gran escala, los métodos tabulares resultan impracticables [SB18]. La aproximación de funciones resuelve esta limitación mediante representaciones parametrizadas, lo que permite generalizar lo aprendido a situaciones similares no observadas. Para este fin se emplean redes neuronales artificiales [BB24].

La unidad neuronal artificial constituye el bloque computacional básico de estas arquitecturas [BB24, GBC16]. Esta procesa un vector de entradas ponderándolo mediante un vector de pesos, añade un término de sesgo y aplica una función de activación no lineal para producir una respuesta escalar. En el ámbito semafórico, las entradas corresponden típicamente a variables observables del tránsito (como tiempos de espera acumulados o longitudes de cola por carril), mientras que los pesos modulan la contribución relativa de cada acceso hacia la decisión final.

La figura 2.13 ilustra este procesamiento: la transformación afín $z = \mathbf{w}^\top \mathbf{x} + b$ integra las entradas $\mathbf{x} = (x_1, \dots, x_n)$ con los pesos $\mathbf{w}$ y el sesgo b , antes de que la activación $y = g(z)$ genere la salida. Sin la introducción de estas no linealidades $g(\cdot)$, una red multicapa se reduciría a una transformación afín simple, imposibilitando la captura de patrones de congestión complejos.

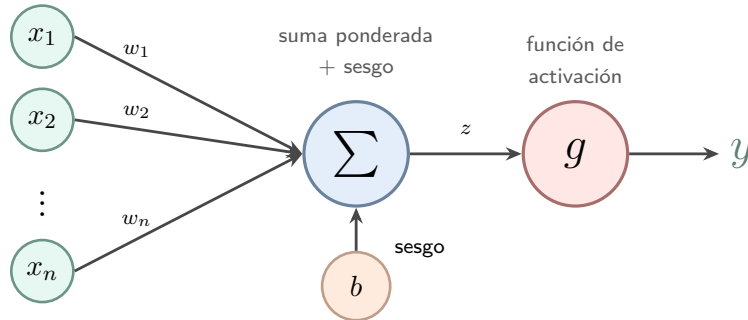


Figura 2.13: Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación.

Como ejemplo conceptual, la red puede recibir las métricas de demora de una intersección y emitir estimaciones de calidad o probabilidades sobre las fases admisibles, orientando la asignación del verde hacia las aproximaciones más críticas.

En una red multicapa, la operación para una unidad u dentro de la capa k se formaliza mediante la ecuación (2.12) [BB24, GBC16]:

$$a_{k,u} = \mathbf{w}_{k,u}^\top \mathbf{x}_{k-1} + b_{k,u}, \quad y_{k,u} = g_k(a_{k,u}) \quad (2.12)$$

donde $\mathbf{x}_{k-1}$ es el vector de activaciones proveniente de la capa anterior, $\mathbf{w}_{k,u}$ es el vector de pesos, $b_{k,u}$ es el sesgo y g_k es la función de activación. Entre las funciones más empleadas en capas intermedias destacan:

- **Unidad lineal rectificada (*Rectified Linear Unit*) (ReLU):** $g(z) = \max\{0, z\}$, que propaga linealmente las activaciones positivas y bloquea las negativas, promoviendo gradientes bien condicionados durante la optimización.
- **Leaky ReLU:** $g(z) = \max\{0, z\} + \alpha \min\{0, z\}$, con $\alpha \ll 1$, que conserva una pendiente pequeña en el semieje negativo para mitigar la inactivación irreversible de unidades.
- **Tanh:** $g(z) = \tanh(z)$, que ofrece una respuesta no lineal diferenciable acotada en $(-1, 1)$.

Las redes *feedforward* encadenan estas transformaciones en capas acíclicas sucesivas sin retroalimentación temporal. Para una arquitectura con L capas parametrizadas, la propagación completa se describe como:

$$\mathbf{h}_k = g_k(\mathbf{W}_k^\top \mathbf{h}_{k-1} + \mathbf{b}_k), \quad k = 1, \dots, L, \quad \mathbf{h}_0 = \mathbf{x}, \quad \mathbf{y} = \mathbf{h}_L \quad (2.13)$$

donde $\mathbf{W}_k$ y $\mathbf{b}_k$ denotan la matriz de pesos y el vector de sesgos de la capa k , respectivamente, y g_k se aplica componente a componente. En el controlador semafórico, el vector de entrada $\mathbf{h}_0 = \mathbf{x}$ contiene las esperas acumuladas por carril y la salida $\mathbf{y} = \mathbf{h}_L$ produce las valoraciones Q o probabilidades de selección de fase.

La figura 2.14 esquematiza esta composición con dos capas ocultas conectadas en cascada hacia la salida.

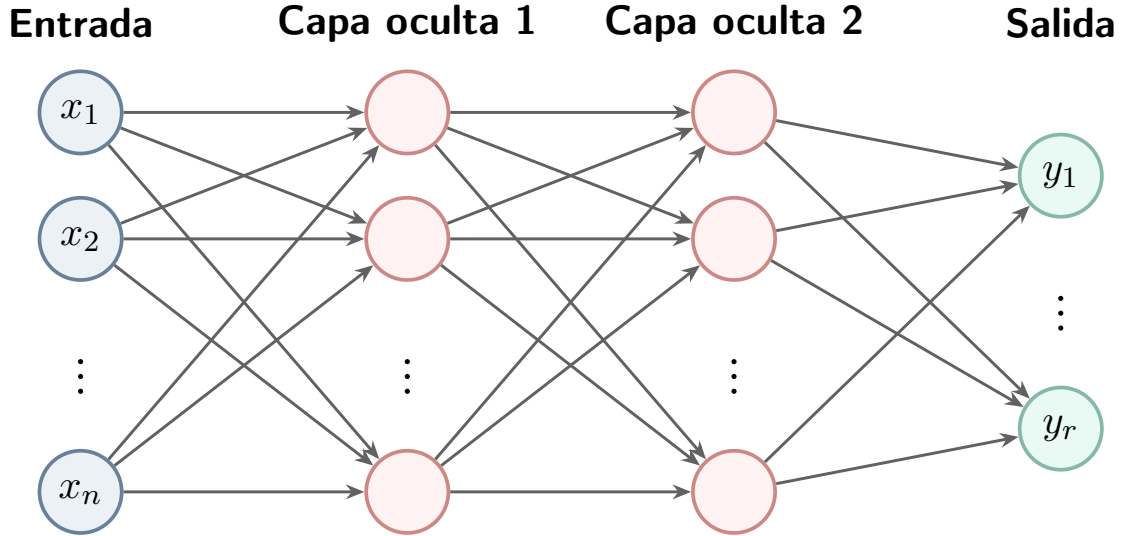


Figura 2.14: Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas.

El Teorema de Aproximación Universal es un resultado de existencia: bajo condiciones apropiadas, una red de dos capas parametrizadas, con una capa oculta suficientemente

ancha, puede aproximar una función continua sobre un dominio compacto con una precisión arbitraria. El teorema no proporciona por sí mismo un procedimiento para encontrar los parámetros, no garantiza un entrenamiento eficiente, una buena generalización ni un buen desempeño práctico sobre datos no observados. La profundidad puede ofrecer representaciones jerárquicas y una mayor eficiencia representacional para ciertas familias de funciones, pero agregar capas o neuronas no garantiza por sí solo un menor error de prueba [BB24].

El proceso de optimización

Durante el entrenamiento, la red procesa las observaciones mediante propagación hacia adelante y contrasta su respuesta frente a un objetivo mediante una función de pérdida escalar diferenciable $\mathcal{L}(\theta)$. El algoritmo de retropropagación (*backpropagation*) calcula analíticamente las derivadas parciales de dicha pérdida respecto de cada peso y sesgo aplicando de forma sistemática la regla de la cadena [BB24].

Con estos gradientes, los parámetros se actualizan iterativamente en sentido opuesto a la pendiente del error:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad (2.14)$$

donde θ reúne los parámetros entrenables de la red, $\nabla_{\theta} \mathcal{L}(\theta_t)$ representa el gradiente de la función de pérdida y $\eta > 0$ es la tasa de aprendizaje. En la práctica, calcular este gradiente sobre todo el volumen de experiencias resulta inviable, mientras que computarlo sobre transiciones aisladas induce una varianza excesiva. Por ello, la optimización se ejecuta habitualmente sobre subconjuntos estocásticos denominados *minibatches* y se complementa con algoritmos adaptativos como Adam, el cual modula individualmente la tasa de paso de cada parámetro mediante estimaciones móviles de primer y segundo momento del gradiente [BB24].

A diferencia del aprendizaje supervisado convencional —donde la pérdida compara la salida con una etiqueta externa fija—, en RL el objetivo se deriva de las recompensas y transiciones producidas por la interacción con el simulador, y cambia a medida que la política mejora su comportamiento.

2.2.4 Aprendizaje por refuerzo profundo (DRL)

El DRL combina algoritmos de RL con aproximadores neuronales. En el control semafórico, las observaciones del tránsito, como colas, ocupación o velocidades, pueden alimentar la red; sus salidas pueden representar valores de acción o parámetros de una política; y las recompensas asociadas con demoras, detenciones y colas orientan la función de pérdida del algoritmo de aprendizaje. Esta integración resulta útil cuando el espacio de estados es continuo o de alta dimensionalidad, aunque exige mecanismos de estabilización y una evaluación cuidadosa de la eficiencia de las muestras [ACS24].

El problema de la tríada mortal (*deadly triad*)

La integración de aproximadores no lineales con aprendizaje por refuerzo puede inducir inestabilidad numérica o divergencia en las estimaciones si no se controlan las condiciones de actualización. Sutton y Barto [SB18] describen este fenómeno mediante la denominada tríada mortal (*deadly triad*), originada por la concurrencia de tres elementos:

- **Aproximación de funciones no lineal:** el empleo de redes profundas para generalizar a través de estados continuos o de alta dimensionalidad.
- **Bootstrapping:** la actualización de estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (métodos TD), prescindiendo de retornos empíricos completos.
- **Entrenamiento *off-policy*:** el aprendizaje sustentado en una distribución de datos generada por una política de comportamiento distinta de la política objetivo.

Aunque prescindir del *bootstrapping* o de la aproximación de funciones comprometería la eficiencia muestral o la escalabilidad ante redes viales complejas, los algoritmos profundos modernos introducen mecanismos arquitectónicos específicos para amortiguar esta inestabilidad [SB18].

Deep Q-Network (DQN): aproximación neuronal de valores de acción

En los métodos basados en valor, DQN sustituye la tabla de estados y acciones por una red neuronal $Q(s, a; \theta)$ parametrizada por θ [MKS⁺15]. Ante una observación de tránsito, la red genera una estimación del valor Q para cada fase admisible. La función de acción-valor óptima $q_*(s, a)$ representa el máximo retorno esperado al seleccionar la mejor secuencia de decisiones futuras, objetivo que DQN persigue aproximar a partir de las recompensas del entorno.

Para mitigar la inestabilidad de la tríada mortal —en particular el problema del blanco móvil, donde actualizar el valor de un estado altera impredeciblemente las estimaciones de estados adyacentes—, DQN incorpora dos componentes estructurales [MKS⁺15]:

- **Redes objetivo (*Target Networks*):** una red secundaria idéntica cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular el término de *bootstrapping* del blanco TD, desacoplando la estimación en curso del objetivo de aprendizaje y amortiguando oscilaciones.
- **Memoria de repetición de experiencias (*replay buffer*):** almacena transiciones históricas $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en un *buffer* $\mathcal{D}$. La optimización extrae *mini-batches* aleatorios de esta memoria, lo cual mitiga la correlación temporal propia del flujo continuo de vehículos en el simulador y aproxima los datos a la hipótesis de muestras independientes.

Durante la interacción con SUMO, la red en línea evalúa la observación de tránsito y una regla ϵ_{exp} -greedy selecciona la fase a ejecutar; la transición resultante se almacena en $\mathcal{D}$. Durante el entrenamiento, la red en línea actualiza sus pesos θ a partir de los *minibatches* muestreados del *buffer* contra el blanco fijado por la red objetivo, cuyos parámetros θ^- se sincronizan solo de forma periódica (figura 2.15).

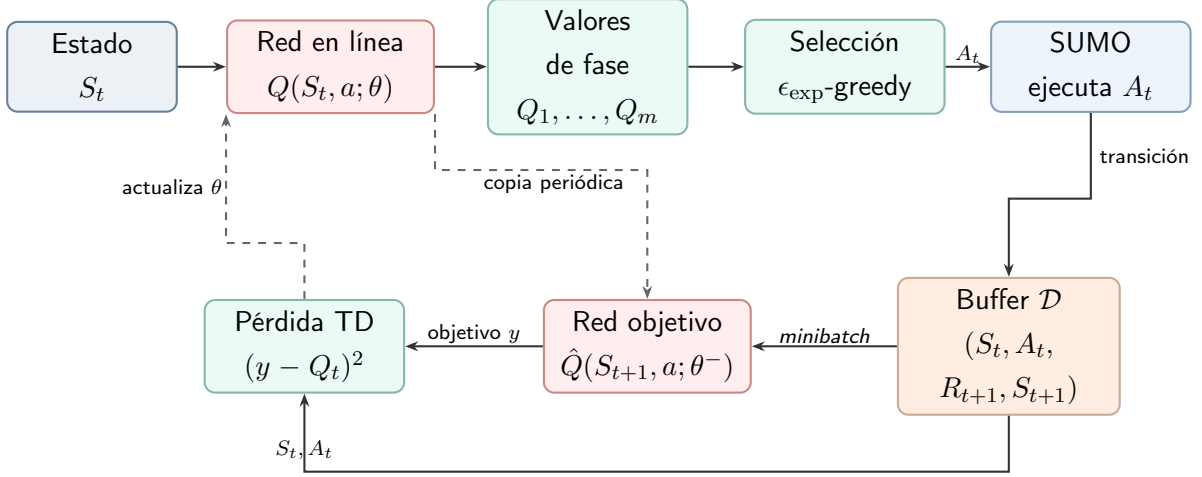


Figura 2.15: Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del blanco TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS⁺15].

La función de pérdida minimizada por DQN, representada en el bloque inferior de la figura 2.15, es la de la ecuación (2.15):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción actual}} \right)^2 \right] \quad (2.15)$$

En la ecuación (2.15), la esperanza se evalúa empíricamente promediando sobre un *mini-batch* de transiciones $(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1})$ extraídas de $\mathcal{D}$. La función de pérdida minimiza la discrepancia cuadrática entre la estimación actual $Q(s_j, a_j; \theta)$ y el blanco temporal compuesto por la recompensa inmediata observada y la mayor estimación del valor futuro sobre las fases admisibles, calculada por la red objetivo $\hat{Q}(s_{j+1}, a'; \theta^-)$. El factor $(1 - d_{j+1})$ anula la contribución del estado siguiente cuando la transición es terminal ($d_{j+1} = 1$), mientras que a' recorre las fases semafóricas admisibles.

Los cuatro bloques numerados como algoritmos en este capítulo son pseudocódigos: descripciones ordenadas e independientes de un lenguaje de programación, no fragmentos ejecutables. En ellos, una Transición (Aprendizaje por Refuerzo) registra lo ocurrido en un paso; un Episodio agrupa pasos desde el reinicio hasta la terminación; y un *Rollout* es

la secuencia de transiciones recolectada con una política. Una Iteración de entrenamiento comprende la recolección de datos y su optimización, mientras que una *Epoch* (época) es una reutilización completa de esos datos dentro de la misma iteración. Cada actualización puede calcularse sobre un *Minibatch* (minilote), es decir, un subconjunto de las muestras. Las líneas “Entrada” y “Salida” indican, respectivamente, qué información necesita el procedimiento y qué componentes entrenados produce; las líneas indentadas pertenecen al ciclo inmediatamente superior. En lo sucesivo se conservan los términos ingleses *batch*, *minibatch*, *buffer* y *epoch*, habituales en la literatura, junto con esta definición en español.

El procedimiento de DQN combina la recolección de experiencias, la selección de fases con política ϵ_{exp} -greedy y la optimización de la red neuronal a partir de *minibatches* muestreados del *Replay buffer*. El algoritmo 1 especifica este procedimiento:

Algoritmo 1 Deep Q-Network con repetición de experiencias

Entrada: entorno, fases admisibles, capacidad N , horizonte T en pasos de decisión, γ y ϵ_{exp}

Salida: parámetros entrenados θ de la red de acción-valor

- 1: Inicializar el *buffer* de experiencias $\mathcal{D}$ con capacidad N
- 2: Inicializar la red de acción-valor $Q(s, a; \theta)$ con parámetros aleatorios
- 3: Inicializar la red objetivo $\hat{Q}(s, a; \theta^-)$ con $\theta^- \leftarrow \theta$
- 4: **para** episodio = 1, 2, ... **hacer**
- 5: Inicializar el estado s_t
- 6: **para** $t = 1, 2, \dots, T$ **hacer**
- 7: Seleccionar a_t mediante una política ϵ_{exp} -greedy basada en $Q(s_t, a; \theta)$
- 8: Ejecutar a_t y observar la recompensa r_{t+1} , el estado s_{t+1} y si el episodio terminó
- 9: Almacenar $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en $\mathcal{D}$
- 10: **si** $\mathcal{D}$ contiene al menos un *minibatch* completo **entonces**
- 11: Extraer un *minibatch* de transiciones de $\mathcal{D}$
- 12: Calcular $y_j = r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$
- 13: Actualizar θ minimizando $(y_j - Q(s_j, a_j; \theta))^2$
- 14: **fin si**
- 15: Actualizar periódicamente la red objetivo: $\theta^- \leftarrow \theta$
- 16: $s_t \leftarrow s_{t+1}$
- 17: **si** $d_{t+1} = 1$ **entonces**
- 18: Terminar el ciclo interno y comenzar un nuevo episodio
- 19: **fin si**
- 20: **fin para**
- 21: **fin para**

En esta formulación, el agente desacopla la recolección de experiencias en el simulador de la optimización paramétrica de la red. La memoria $\mathcal{D}$ rompe la correlación tempo-

ral de las observaciones de tráfico, mientras que la red objetivo estabiliza el blanco de Bellman mediante la sincronización periódica de θ^- . La especificación detallada de los hiperparámetros operacionales (capacidad N , horizonte T , tasa de aprendizaje y cronograma de exploración) se desarrolla en el capítulo 3.

Aproximación de políticas, función de ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s; \theta)$ [ACS24]. Bajo la formulación MDP, en un problema de acciones discretas la red recibe el estado s , calcula un vector de puntajes continuos $z \in \mathbb{R}^m$ y los proyecta a una distribución de probabilidad mediante la función *softmax* [SB18, BB24]:

$$\pi_{\theta}(a_i | s) = \frac{e^{z_i}}{\sum_{j=1}^m e^{z_j}} \quad (2.16)$$

la cual garantiza valores estrictamente positivos que suman la unidad. En este ejemplo, las acciones corresponden a las fases semafóricas admisibles: las variables del estado de tránsito ingresan a la red y sus salidas representan probabilidades sobre dicho conjunto de acciones. Las políticas basadas en observaciones locales se formalizan posteriormente mediante Dec-POMDP. La acción puede muestrearse de esa distribución durante el entrenamiento, preservando la exploración estocástica.

La figura 2.16 muestra este recorrido para un controlador semafórico. A diferencia de DQN, las salidas no son estimaciones de valor: son probabilidades que suman uno. La señal de ventaja no entra a la red para seleccionar la acción; se utiliza durante el entrenamiento para modificar los parámetros en la dirección indicada por el gradiente de la log-probabilidad de la acción observada.

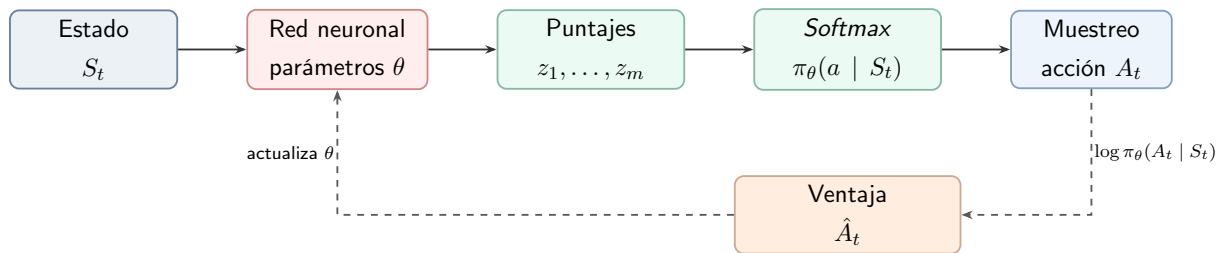


Figura 2.16: Red de política para un espacio discreto de acciones en la formulación de un MDP. El estado determina una distribución sobre las acciones disponibles; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].

La optimización de políticas se fundamenta en aislar la contribución específica de una acción respecto del rendimiento promedio del estado. Formalmente, la función de ventaja (*Advantage Function*) se define como la diferencia entre el valor de la acción y el valor

basal del estado [SB18]:

$$A^\pi(s, a) \doteq Q^\pi(s, a) - V^\pi(s) \quad (2.17)$$

donde $Q^\pi(s, a)$ es el retorno esperado al seleccionar la acción a en el estado s bajo la política π , y $V^\pi(s) = \mathbb{E}_{a \sim \pi}[Q^\pi(s, a)]$ representa el valor esperado medio en dicho estado. Un valor $A^\pi(s, a) > 0$ señala que la fase semafórica ejecutada supera el desempeño promedio de la intersección ante esas condiciones de cola, mientras que $A^\pi(s, a) < 0$ indica una decisión comparativamente deficiente.

Para estimar la ventaja sin incurrir en la alta varianza del retorno Monte Carlo completo ni en el sesgo temprano de un error temporal de un paso, Schulman et al. [SML⁺16] desarrollaron el estimador GAE (*Generalized Advantage Estimation*):

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} \doteq \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad (2.18)$$

donde $\delta_t^V = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t)$ es el error temporal del crítico parametrizado por ϕ , d_{t+1} es el indicador terminal, $\gamma \in [0, 1)$ es el factor de descuento y $\lambda \in [0, 1]$ es el parámetro de suavizado exponencial. El hiperparámetro λ interpola estimaciones de ventaja de distintos horizontes. Para $\lambda = 0$, se obtiene el residuo TD de un paso δ_t^V , que suele reducir la varianza a costa de depender de la precisión del crítico y, por tanto, puede introducir sesgo. Para $\lambda = 1$, con el retorno completo, se recupera el estimador Monte Carlo descontado menos el valor del estado; este es γ -justo incluso con un crítico aproximado, aunque normalmente presenta mayor varianza. En trayectorias truncadas, el *bootstrap* y la aproximación del crítico pueden modificar este compromiso.

El Teorema del Gradiente de Política (*Policy Gradient Theorem*) [SB18] fundamenta analíticamente la optimización directa de los parámetros de la política maximizando el retorno esperado $J(\theta)$. Empleando la función de ventaja como línea de base para mitigar la varianza, el gradiente se aproxima mediante la esperanza muestral sobre un *batch* de transiciones recolectadas:

$$\nabla_\theta J(\theta) \approx \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(A_t | S_t) \hat{A}_t \right] \quad (2.19)$$

donde $\pi_\theta(A_t | S_t)$ denota la probabilidad asignada a la fase A_t bajo los parámetros θ , y $\hat{A}_t$ es la ventaja estimada mediante GAE. Como esquematiza la figura 2.16, la dirección del vector gradiente escala proporcionalmente a $\hat{A}_t$, incrementando la probabilidad de ocurrencia de aquellas fases que superan el desempeño esperado y desincentivando aquellas que provocan demoras excesivas.

Proximal Policy Optimization (PPO) y control de las actualizaciones

Los métodos clásicos de gradiente de política exhiben una elevada sensibilidad a la magnitud de la tasa de paso: una actualización excesiva en el espacio de parámetros puede

degradar abruptamente el rendimiento de la política, desplazándola a regiones del espacio de estados donde la recolección de trayectorias queda irreparablemente sesgada.

Para acotar el tamaño de paso en el espacio de distribuciones de probabilidad, Kakade y Langford [KL02] y Schulman et al. (*Trust Region Policy Optimization* (optimización con una restricción de región de confianza) (TRPO) [SLA⁺15]) formularon métodos de región de confianza. En el análisis teórico de TRPO, bajo los supuestos del modelo y una actualización que respete la región de confianza basada en la divergencia de Kullback–Leibler (KL, una medida probabilística que cuantifica la discrepancia estadística entre la nueva política y la previa [GBC16, SLA⁺15]), la maximización exacta de la cota sustituta produce una política con retorno esperado no decreciente:

$$J(\pi_{k+1}) \geq J(\pi_k) \quad (2.20)$$

El algoritmo práctico de TRPO aproxima este procedimiento mediante ventajas estimadas, una divergencia KL promedio y optimización numérica. Sin embargo, resolver la optimización cuadrática con restricciones de TRPO en redes neuronales profundas resulta computacionalmente oneroso. Como alternativa de primer orden, PPO [SWD⁺17] reemplaza la restricción formal por un objetivo sustituto recortado (CLIP) que busca limitar actualizaciones excesivas, sin transferir automáticamente la garantía teórica de TRPO.

Definiendo la razón de probabilidad entre la política actual y la previa:

$$r_t(\theta) \doteq \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \quad (2.21)$$

donde $\pi_{\theta_{\text{old}}}$ representa la política que recolectó el *batch* de transiciones, PPO maximiza el objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t \right) \right] \quad (2.22)$$

En la ecuación (2.22), $\epsilon_{\text{clip}} > 0$ delimita el margen de recorte permitido para la razón $r_t(\theta)$. El operador $\min$ adopta una cota inferior pesimista: cuando una fase reporta una ventaja positiva ($\hat{A}_t > 0$), el recorte a $1 + \epsilon_{\text{clip}}$ desincentiva incrementos desmedidos en la probabilidad de dicha acción; análogamente, cuando $\hat{A}_t < 0$, el límite inferior $1 - \epsilon_{\text{clip}}$ frena disminuciones excesivas provocadas por muestras ruidosas.

En la arquitectura actor-crítico práctica, este objetivo de política se integra de forma conjunta con la regresión del crítico y un término de regularización entrópica:

$$J^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \hat{\mathbb{E}}_t \left[(V_\phi(s_t) - R_t^{\text{target}})^2 \right] + c_2 \hat{\mathbb{E}}_t [S[\pi_\theta](s_t)] \quad (2.23)$$

donde ϕ son los parámetros del crítico, R_t^{target} es el retorno objetivo estimado (calculado como $\hat{A}_t + V_\phi(s_t)$), $S[\pi_\theta](s_t) \doteq - \sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)$ es la entropía de Shannon de

la política en s_t , y $c_1, c_2 > 0$ son coeficientes de ponderación [SWD⁺17]. La inclusión del término entrópico favorece la exploración activa al penalizar distribuciones excesivamente concentradas, impidiendo que la política colapse prematuramente hacia un comportamiento determinista antes de examinar adecuadamente las distintas fases.

PPO opera de forma predominantemente *on-policy*: recolecta un *batch* de trayectorias con la política vigente $\pi_{\theta_{\text{old}}}$, ejecuta un número acotado de *epochs* de optimización sobre *minibatches*, y descarta inmediatamente el *batch* para evitar sesgos por desfase distribucional. El algoritmo 2 describe este ciclo de entrenamiento:

Algoritmo 2 Proximal Policy Optimization (PPO)

Entrada: entorno, horizonte de recolección, *epochs* K , γ , λ_{GAE} y ϵ_{clip}

Salida: parámetros entrenados θ del actor y ϕ del crítico

- 1: Inicializar los parámetros del actor θ y del crítico ϕ
 - 2: Inicializar la política de recolección con $\theta_{\text{old}} \leftarrow \theta$
 - 3: **para** iteración = 1, 2, ... **hacer**
 - 4: Recolectar trayectorias con la política $\pi_{\theta_{\text{old}}}$
 - 5: Calcular los retornos objetivo y las ventajas estimadas $\hat{A}_t$
 - 6: **para** *epoch* = 1 hasta K **hacer**
 - 7: Muestrear *minibatches* del conjunto de trayectorias recolectado
 - 8: Calcular $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
 - 9: Actualizar θ maximizando $L^{\text{CLIP}}(\theta)$
 - 10: Actualizar ϕ minimizando el error cuadrático del valor
 - 11: Incorporar, si corresponde, un término de entropía para favorecer la exploración
 - 12: **fin para**
 - 13: $\theta_{\text{old}} \leftarrow \theta$
 - 14: **fin para**
-

En PPO, θ_{old} representa la política recolectora y permanece fijo al calcular las razones de probabilidad. El *batch* se reutiliza durante un número acotado de *epochs*; el recorte reduce el incentivo para que las razones superen el intervalo $[1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}]$, pero no impone una restricción estricta ni garantiza una mejora monótona [SWD⁺17]. Después de la optimización, θ_{old} se actualiza con θ y se recolectan nuevas experiencias. Durante la ejecución en la red semafórica, el crítico se desacopla y el controlador evalúa únicamente su actor local π_{θ} .

2.3 Aprendizaje por refuerzo multiagente (MARL)

El MARL generaliza el marco de RL hacia sistemas donde múltiples agentes autónomos interactúan y aprenden en un entorno compartido [ACS24]. En el control de redes se-

mafóricas, la hipótesis de agente aislado deja de sostenerse: la fase habilitada en un cruce regula el flujo vehicular emitido hacia los accesos de las intersecciones adyacentes, acoplando espacialmente las dinámicas de congestión de toda la red. Cada controlador influye directamente en las tasas de arribo y longitudes de cola de sus vecinos, lo que demanda esquemas de coordinación capaces de optimizar el flujo global.

Liang et al. [LML⁺25] categorizan los problemas multiagente según la estructura de sus recompensas en completamente competitivos, mixtos o completamente cooperativos. El presente trabajo se posiciona en el régimen cooperativo de control distribuido: los controladores persiguen mitigar la demora global de la red vial. No obstante, para preservar la asignabilidad del crédito y evitar que un agente se beneficie del desempeño ajeno sin contribuir, comportamiento esperable bajo una recompensa global idéntica, cada agente recibe una señal individual compuesta que pondera el desempeño de su propia intersección junto con el estado de sus nodos inmediatamente adyacentes (sección 3.5.3).

2.3.1 Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP

La extensión matemática del MDP ante múltiples actores decisores se formaliza mediante los Juegos Estocásticos o Juegos Markovianos (*Stochastic Games*), introducidos originalmente por Shapley [Sha53] y adaptados al aprendizaje por refuerzo contemporáneo por Albrecht et al. [ACS24].

Un Juego Estocástico de n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, \{R_i\}_{i=1}^n, \gamma \rangle \quad (2.24)$$

donde $\mathcal{N} = \{1, \dots, n\}$ es el conjunto de agentes (intersecciones), $\mathcal{S}$ representa el espacio de estados globales de la red vial, $\mathcal{A}_i$ es el conjunto discreto de fases semafóricas disponibles para el agente i , y $\mathbf{a} = (a_1, \dots, a_n) \in \mathcal{A} \equiv \mathcal{A}_1 \times \dots \times \mathcal{A}_n$ denota la acción conjunta. La función de transición $P(s' \mid s, \mathbf{a})$ modela la probabilidad de evolucionar hacia el estado global s' dada la acción conjunta ejecutada en el simulador SUMO, y $R_i(s, \mathbf{a})$ es la función de recompensa asignada al agente i .

En el análisis de estabilidad de estos sistemas, el equilibrio de Nash describe una configuración donde ningún agente puede incrementar unilateralmente su retorno esperado desviándose de su política actual [ACS24]. Formalmente, una política conjunta $\boldsymbol{\pi}^* = (\pi_1^*, \dots, \pi_n^*)$ constituye un equilibrio de Nash si:

$$J(\pi_i, \boldsymbol{\pi}_{-i}^*) \leq J(\pi_i^*, \boldsymbol{\pi}_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \quad (2.25)$$

donde $J(\cdot)$ denota el retorno esperado del agente i bajo el perfil de políticas indicado, $\boldsymbol{\pi}_{-i}^*$ representa el perfil de políticas conjuntas de todos los agentes excepto el agente i , y Π_i es el espacio de políticas admisibles. Aunque en sistemas estrictamente cooperativos

este equilibrio caracteriza puntos de estabilidad mutua, no garantiza por sí mismo la optimalidad global de Pareto.

En redes urbanas reales, los controladores carecen de información instantánea y global sobre todo el trazado vial. Cada intersección opera sujeta a observabilidad parcial, percibiendo únicamente el estado de sus detectores locales. Esta restricción operacional se modela formalmente mediante un Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*) (Dec-POMDP) [AOS20], definido por la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, R, \{\Omega_i\}_{i=1}^n, O, \gamma \rangle \quad (2.26)$$

donde Ω_i es el conjunto de observaciones locales accesibles al agente i , y $O(o_1, \dots, o_n \mid s', \mathbf{a}) = \Pr\{\mathbf{o} = (o_1, \dots, o_n) \mid s', \mathbf{a}\}$ es la función de emisión estocástica de observaciones [AOS20]. El estado global s es una variable del modelo que no tiene por qué ser accesible a cada agente; en general, una política descentralizada puede depender del historial local de observaciones y acciones τ_i y expresarse como $\pi_i(a_i \mid \tau_i)$. En este trabajo, cada controlador adopta una política reactiva basada en la observación actual $\pi_i(a_i \mid o_i)$, sin identificar o_i con s (figura 2.17).

La figura 2.17 esquematiza este acoplamiento: cada agente procesa su observación local o_i y emite una fase a_i . La concurrencia de estas decisiones conforma la acción conjunta $\mathbf{a}$, la cual perturba el estado global del simulador de tráfico y devuelve a cada controlador su nueva observación y su retroalimentación individual r_i .

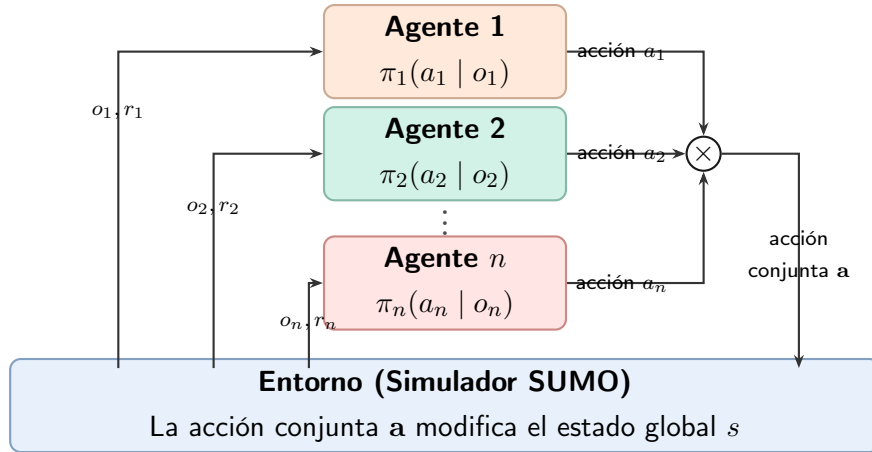


Figura 2.17: Interacción agente-entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

2.3.2 Desafíos del aprendizaje multiagente

La transición desde el aprendizaje individual hacia el dominio multiagente introduce desafíos teóricos y computacionales específicos [ACS24]:

- **Explosión combinatoria del espacio de acciones conjuntas:** Si cada controlador i dispone de un conjunto de acciones locales $\mathcal{A}_i$, la cardinalidad del espacio de acciones conjuntas es:

$$|\mathcal{A}| = \prod_{i=1}^n |\mathcal{A}_i|. \quad (2.27)$$

Cuando todos los controladores poseen A acciones, esta cardinalidad es A^n y, por tanto, crece exponencialmente con el número de agentes. Para una red de escala urbana, evaluar o enumerar exhaustivamente $\mathcal{A}$ deja de ser una alternativa viable, lo que motiva el diseño de enfoques escalables, como políticas de decisión localmente descentralizadas.

- **No estacionariedad y el problema del blanco móvil:** Desde la perspectiva local del agente i , la función de transición ambiental percibida:

$$\mathcal{P}_i(s' | s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' | s, a_i, \mathbf{a}_{-i}) \pi_{-i}(\mathbf{a}_{-i} | s) \quad (2.28)$$

varía de forma continua a medida que los restantes agentes adaptan sus respectivas políticas π_{-i} . Como formalizaron Bowling y Veloso [BV02], esta mutabilidad induce el problema del blanco móvil (*moving target*): una política óptima frente al comportamiento vigente de los vecinos puede tornarse deficiente en cuanto estos ajustan sus parámetros. Aunque el sistema global puede seguir describiéndose mediante un juego markoviano, la dinámica efectiva percibida por un agente local se vuelve no estacionaria cuando las políticas de los demás agentes cambian y no forman parte de su información. En consecuencia, las condiciones de convergencia de los métodos de agente único basados en MDPs estacionarios dejan de ser suficientes [FNF⁺17, LML⁺25].

En juegos competitivos o de suma cero como Piedra, Papel o Tijera, el aprendizaje independiente con tasas fijas genera trayectorias oscilatorias alrededor del equilibrio. Algoritmos especializados como *Win or Learn Fast Policy Hill-Climbing* (PHC con tasas distintas según el desempeño) (WoLF-PHC) (*Win or Learn Fast Policy Hill-Climbing*) modulan adaptativamente su tasa de aprendizaje, amortiguando las espirales hacia el equilibrio de Nash ($\pi^*(a) = 1/3$), como ilustra la figura 2.18. En el control semafórico, la no estacionariedad se manifiesta cuando un cruce extiende su verde para evacuar un pelotón, alterando abruptamente los arribos en los cruces receptores; si los agentes aprenden sin coordinación, corren el riesgo de generar oscilaciones destructivas en las colas de la red.

- **Observabilidad parcial:** Bajo el formalismo Dec-POMDP, cada semáforo basa su inferencia exclusivamente en detectores locales, desconociendo el estado global de la

red vial y las decisiones simultáneas de cruces lejanos, lo cual incrementa el sesgo y la incertidumbre en las estimaciones de valor.

- **Asignación de crédito multiagente (*Credit Assignment*):** Cuando múltiples agentes actúan en paralelo, determinar la contribución marginal de cada decisión local sobre el desempeño del sistema resulta complejo [ACS24]. Una acción local desfavorable puede quedar enmascarada si el resto de los semáforos operó de forma eficiente, o viceversa. El diseño de recompensas individuales compuestas (sección 3.5.3) aborda este desafío aislando la contribución de la intersección y su vecindario inmediato.

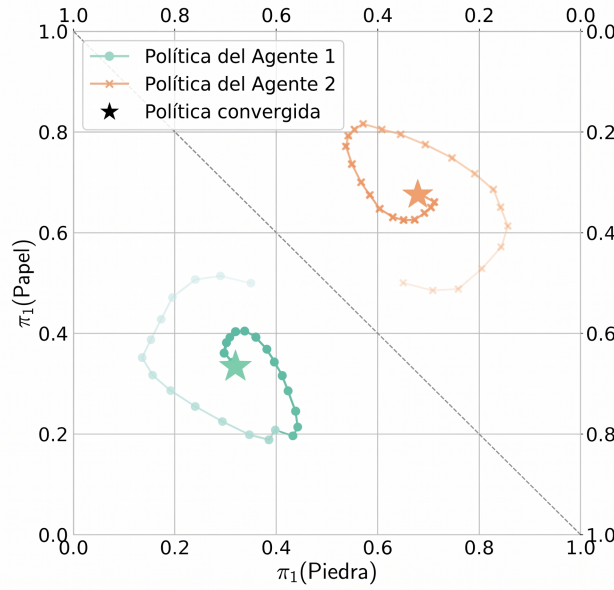


Figura 2.18: Evolución de las políticas de dos agentes con el algoritmo WoLF-PHC en el juego de Piedra, Papel o Tijera. La diagonal discontinua divide el gráfico en dos simplex de probabilidad, uno para cada agente. Cada punto en el simplex representa una distribución sobre las tres acciones disponibles; las trayectorias muestran el avance de las políticas durante el entrenamiento y los marcadores centrales destacan la política convergida en el equilibrio de Nash ($\pi^*(a) = 1/3$ para cada acción). Adaptado de [BV02, ACS24].

2.3.3 Paradigmas de entrenamiento y ejecución

Para mitigar la no estacionariedad respetando la observabilidad parcial de los controladores en despliegue, la literatura distingue dos paradigmas fundamentales según la información disponible durante la optimización [ZLYZ23]:

- **Aprendizaje Independiente (IL, *Independent Learning*):** Cada controlador se optimiza de forma totalmente descentralizada tratando a los demás agentes como componentes estocásticos del entorno. Aunque computacionalmente simple y altamente escalable, es susceptible a la inestabilidad por no estacionariedad.

- **Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*, CTDE):** Durante la etapa de entrenamiento en el simulador, se aprovecha información global privilegiada (estados conjuntos o concatenaciones de observaciones) para coordinar la evaluación y reducir la varianza de las estimaciones; durante la etapa de ejecución, el componente centralizado se desacopla y cada agente opera con su política local. La figura 2.19 ilustra esta separación entre etapas.

2.3.4 Aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*) (MADRL) y líneas de base

La convergencia entre Dec-POMDP y redes profundas sustenta el campo del MADRL [LWT⁺17]. Para evaluar empíricamente el valor agregado del entrenamiento centralizado (MAPPO y HAPPO), esta investigación establece dos líneas de base de aprendizaje independiente (IL):

- **IDQN (Línea de base basada en valor):** Extensión de DQN donde cada intersección mantiene su propia red $Q_i(o_i, a_i)$ [TMK⁺17]. Al operar *off-policy* con un *replay buffer*, almacena transiciones generadas bajo políticas que ya mutaron, lo que exacerba el desfase temporal de las experiencias y acentúa la no estacionariedad [FNF⁺17].
- **IPPO (Línea de base basada en política):** Instancia un optimizador PPO independiente por agente [dWGM⁺20]. Dado su régimen *on-policy*, descarta el *batch* tras cada iteración, mitigando la acumulación de datos obsoletos. Aunque no elimina el acoplamiento no estacionario, su mecanismo de recorte ofrece una referencia competitiva frente a los métodos coordinados.

La figura 2.19 resume esta partición operativa entre ambas etapas.

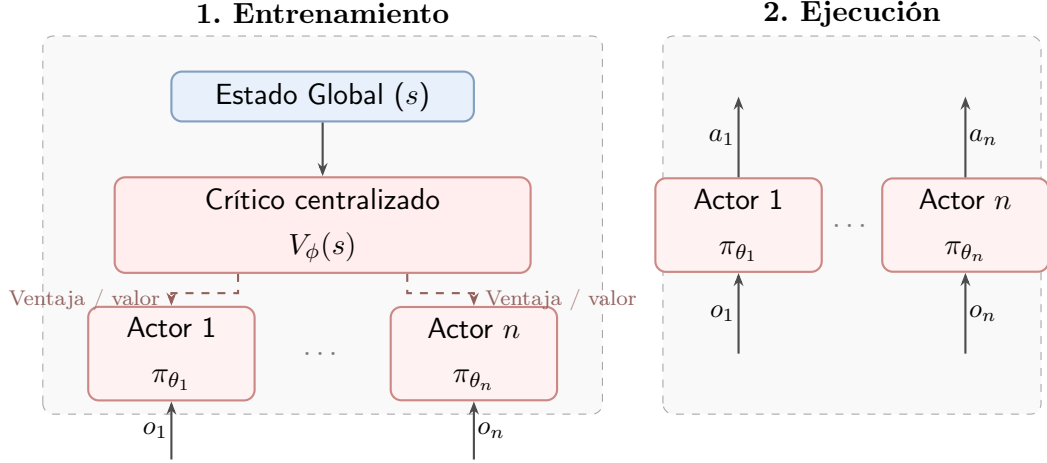


Figura 2.19: Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.

2.3.5 Multi-Agent Proximal Policy Optimization (MAPPO)

Para extender las ventajas de estabilidad de PPO al contexto multiagente bajo el paradigma CTDE, Yu et al. [YVV⁺22] estudiaron una configuración de Multi-Agent Proximal Policy Optimization (MAPPO) basada en PPO y un crítico centralizado. En lugar de rediseñar el estimador base, MAPPO preserva la simplicidad computacional de PPO mediante la factorización explícita entre políticas descentralizadas y evaluación centralizada:

- **Actores descentralizados** ($\pi_{\theta_i}(a_i | o_i)$): Cada controlador i procesa exclusivamente su vector de observación local o_i para seleccionar su fase semafórica a_i . En intersecciones con geometrías idénticas puede compartirse parámetros entre actores, aunque esto no constituye una restricción conceptual.
- **Crítico centralizado** ($V_\phi(s)$): Durante el entrenamiento, el crítico puede utilizar información global que no está disponible para los actores. Una representación global adecuada puede facilitar la estimación del valor, y una línea de base más precisa puede contribuir a reducir la varianza, sin garantía automática [YVV⁺22].

El objetivo de optimización de los actores promedia los términos individuales de recorte PPO sobre los n agentes:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t), \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \quad (2.29)$$

donde $r_t^i(\theta) = \frac{\pi_{\theta_i}(a_i | o_i)}{\pi_{\theta_{i,\text{old}}}(a_i | o_i)}$ denota la razón de probabilidad local de cada agente y ϵ_{clip} es el margen de recorte. Paralelamente, el crítico centralizado se actualiza minimizando el error cuadrático frente al retorno objetivo R_t^{target} , estimado empíricamente sobre el

batch de transiciones mediante la suma de la ventaja GAE y el valor de estado previo $\hat{A}_t(s_t, \mathbf{a}_t) + V_\phi(s_t)$ [YVV⁺22, SML⁺16]:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]. \quad (2.30)$$

Factores críticos de diseño y optimización en MAPPO

Yu et al. [YVV⁺22] analizaron empíricamente diversos factores de implementación e hiperparámetros que afectan el rendimiento de MAPPO:

1. **Normalización de retornos:** En los experimentos de Yu et al., normalizar los objetivos de valor puede favorecer el ajuste del crítico cuando cambia la escala de las recompensas.
2. **Estructuración del estado global del crítico:** En ese estudio, la entrada s se evaluó con distintas representaciones: concatenación directa de observaciones ($CL : s = (o_1, \dots, o_n)$), estado global extraído directamente del simulador (EP), o estados aumentados específicos por agente ($AS_i = EP \cup o_i$), combinados opcionalmente con filtrado de redundancias (FP).
3. **Reutilización de datos:** La cantidad de *epochs* (K) y la división en *minibatches* regulan la reutilización del *batch*. Yu et al. observaron que una reutilización excesiva puede deteriorar el rendimiento, especialmente en tareas difíciles.

El algoritmo 3 especifica el ciclo computacional de MAPPO bajo el paradigma CTDE.

Algoritmo 3 Algoritmo Multi-Agent Proximal Policy Optimization (MAPPO)

Entrada: entorno multiagente, n agentes, horizonte de recolección, $epochs$ K , γ , λ_{GAE}
y ϵ_{clip}

Salida: parámetros entrenados de los actores θ y del crítico centralizado ϕ

- 1: Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
- 2: Inicializar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
- 3: **para** iteración = 1, 2, ... **hacer**
- 4: Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta π_θ
- 5: Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
- 6: Calcular los retornos objetivos R_t^{target} normalizados para el crítico
- 7: Almacenar las transiciones $(s_t, \mathbf{o}_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
- 8: **para** $epoch = 1$ hasta K **hacer**
- 9: Muestrear *minibatches* de datos desde $\mathcal{D}$
- 10: Actualizar la política maximizando el objetivo de recorte PPO por agente:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t, \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t \right) \right]$$
- 11: Actualizar el crítico ϕ minimizando el error cuadrático medio del valor:

$$\mathcal{L}^{\text{VF}}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - R_t^{\text{target}})^2 \right]$$
- 12: **fin para**
- 13: Vaciar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
- 14: **fin para**

Durante el entrenamiento, el crítico centralizado $V_\phi(s)$ procesa el estado global de la red vial para calcular ventajas conjuntas mediante GAE, guiando la optimización paralela de los actores locales π_{θ_i} con el objetivo recortado. Al finalizar cada iteración, el *buffer* $\mathcal{D}$ se descarta por completo para preservar el régimen *on-policy* del método. En la etapa de ejecución, el crítico se desacopla y cada controlador actúa con autonomía computacional a partir de su vector local o_t^i . Los hiperparámetros de entrenamiento y la arquitectura de las redes se detallan en el capítulo 3.

La formulación anterior presenta la variante teórica con una ventaja conjunta. En la implementación de este trabajo, el crítico centralizado produce una salida por agente y los objetivos de GAE se calculan por agente a partir de la recompensa individual compuesta descrita en la sección 3.5.3; por tanto, la ecuación resume el paradigma CTDE y no debe leerse como una identidad exacta del código ejecutado.

2.3.6 Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

En sistemas de control complejos, los agentes pueden presentar asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En entornos homogéneos, compartir parámetros puede mejorar la eficiencia de aprendizaje; en entornos heterogéneos, esa práctica puede limitar la especialización. Además, las actualizaciones independientes no poseen una garantía general de convergencia debido al aprendizaje simultáneo no coordinado [KCW⁺22].

Kuba et al. y Zhong et al. [KCW⁺22, ZKF⁺24] propusieron algoritmos de aprendizaje por refuerzo multiagente para políticas heterogéneas, sin exigir que todos los agentes compartan parámetros. Esta línea se denomina aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*, HARL). Cada agente i puede mantener su propia arquitectura y parámetros θ^i , adaptados a su espacio de acciones local $\mathcal{A}_i$. En esta línea, *Heterogeneous-Agent Trust Region Policy Optimization* (optimización con región de confianza para agentes heterogéneos) (HATRPO) y HAPPO emplean una actualización secuencial de los agentes. Los resultados teóricos de mejora monótona y convergencia se refieren al procedimiento de región de confianza bajo supuestos explícitos; la variante práctica HAPPO sustituye la restricción KL por un objetivo recortado y debe interpretarse como una aproximación [KCW⁺22, ZKF⁺24].

Compartición de parámetros y políticas heterogéneas

Una distinción central entre MAPPO y HAPPO radica en la homogeneidad de las políticas y en el orden de actualización:

- **Compartición de parámetros:** En MAPPO se utiliza habitualmente cuando los agentes poseen espacios de observación y acción compatibles. Todos los agentes pueden compartir los mismos pesos $\theta_1 = \dots = \theta_n = \theta$, aunque la formulación de MAPPO también puede implementarse sin esta restricción.
- **Políticas heterogéneas y actualización secuencial:** HAPPO permite que cada agente mantenga una política independiente $\pi_{\theta^i}(a_i | o_i)$ y actualiza los actores siguiendo una permutación. El *batch* de trayectorias de la iteración actual se reutiliza de forma limitada; esto no equivale a un *buffer* histórico de *experience sharing* como el empleado por algoritmos específicos de compartición de experiencia.

Lema de descomposición de la ventaja multiagente

La actualización simultánea de todos los controladores puede generar inestabilidad porque el cambio concurrente de las políticas altera tanto las distribuciones de acción como la distribución conjunta de las trayectorias. HAPPO aborda este problema mediante una actualización secuencial: en cada iteración se optimiza un agente por vez, condicionando

cada paso a los cambios ya efectuados por los precedentes. Para sustentar este esquema, la ventaja conjunta de la red debe poder descomponerse en contribuciones marginales ordenadas [KCW⁺22].

Esta descomposición se formaliza en el lema de descomposición de la ventaja multiagente propuesto por Kuba et al. [KCW⁺22]. En cualquier juego markoviano cooperativo, para una política conjunta π y un orden arbitrario de los agentes $i_{1:n} = (i_1, \dots, i_n)$, la ventaja conjunta de la red se descompone exactamente como la suma de las ventajas marginales de cada agente, condicionadas por las decisiones de los que lo preceden en la secuencia:

$$A^\pi(s, \mathbf{a}) = \sum_{m=1}^n A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \quad (2.31)$$

donde la ventaja marginal del agente i_m se define como:

$$A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}}). \quad (2.32)$$

En la ecuación (2.32), $Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ evalúa el retorno esperado fijando las acciones de los primeros m agentes de la secuencia, mientras que las decisiones de los agentes subsecuentes ($\mathbf{a}_{i_{m+1:n}}$) se marginalizan por esperanza bajo la política vigente.

La deducción matemática canónica de este lema asume un juego markoviano estrictamente cooperativo con un retorno escalar compartido $R(s, \mathbf{a})$ para toda la red. En sistemas espaciales distribuidos de gran escala, como el control semafórico urbano, asignar una recompensa global idéntica a todas las intersecciones dificulta la asignación de crédito, extremo que Chu et al. respaldan con evidencia empírica en redes extensas [CWCL19], y encarece la estimación de las ventajas [ACS24]. Por este motivo, los modelos de tráfico descomponen la señal en recompensas locales y de vecindario inmediato.

Bajo retornos desacoplados, la identidad aditiva de la ecuación (2.31) no se traduce en una equivalencia algebraica directa entre un único valor global y los retornos individuales; no obstante, el principio de optimización secuencial por bloques (tipo Gauss-Seidel) conserva su utilidad práctica para desacoplar el aprendizaje concurrente [ACS24, WZK⁺23].

La relevancia teórica de este lema radica en que es una identidad analítica exacta para cualquier juego cooperativo sin imponer restricciones de aditividad o de monotonía sobre la función de valor en la red de mezclado (a diferencia de las condiciones de factorización IGM de arquitecturas como *Value-Decomposition Networks* (redes de descomposición del valor conjunto) (VDN) [SLG⁺18] o *Q-Mixing Network* (red que combina valores de acción individuales) (QMIX) [RSDW⁺18]).

Esquema de actualización secuencial y factor de razón compuesto M

Para explotar analíticamente esta descomposición sin incurrir en la inestabilidad del entrenamiento simultáneo, HAPPO actualiza las políticas de los agentes de manera secuencial dentro de cada iteración k . En cada ciclo se sortea una permutación de los agentes

$i_{1:n} \in \text{Sym}(n)$ (donde $\text{Sym}(n)$ representa el conjunto de todas las permutaciones posibles de los n agentes) para evitar sesgos sistemáticos de prioridad en la secuencia de optimización [KCW⁺22, WZK⁺23].

Cuando corresponde el turno del agente i_m , los agentes previos en la secuencia ($i_{1:m-1}$) ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores ($i_{m+1:n}$) conservan aún sus parámetros previos $\theta_k^{i_j}$. Para guiar al agente i_m teniendo en cuenta la alteración de la distribución conjunta producida por los agentes ya actualizados, Kuba et al. [KCW⁺22] demostraron que no se requiere entrenar un crítico separado por cada subgrupo; la ventaja marginal puede computarse reponderando la ventaja conjunta $A^{\pi_k}(s, \mathbf{a})$ mediante la productoria acumulada de las razones de probabilidad de los agentes ya actualizados:

$$M_{i_{1:m}}(s_t, \mathbf{a}_t) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}^{i_j}}^{i_j}(a_t^{i_j} | o_t^{i_j})}{\pi_{\theta_k^{i_j}}^{i_j}(a_t^{i_j} | o_t^{i_j})} \right) A^{\pi_k}(s_t, \mathbf{a}_t) \quad (2.33)$$

donde $\pi_{\theta_{k+1}^{i_j}}^{i_j}$ es la política recién optimizada del agente i_j y $\pi_{\theta_k^{i_j}}^{i_j}$ es su versión previa. Para el primer agente ($m = 1$), la productoria es vacía y $M_{i_1}(s_t, \mathbf{a}_t) \equiv A^{\pi_k}(s_t, \mathbf{a}_t)$. En la formulación algorítmica práctica, el término acumulado $M_{i_{1:m}}$ se mantiene en una variable de memoria M_{i_m} por cada agente, la cual se actualiza recursivamente conforme avanzan las optimizaciones precedentes. El factor $M_{i_{1:m}}$ actúa como un estimador de importancia que ajusta la ventaja conjunta al nuevo perfil de comportamiento sin requerir simulaciones adicionales en el entorno [KCW⁺22].

Así, la política del agente i_m se optimiza maximizando el objetivo sustituto recortado ponderado por $M_{i_{1:m}}$:

$$L^{\text{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \left[\min \left(r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \right. \right. \\ \left. \left. \text{clip}(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) M_{i_{1:m}}(s_t, \mathbf{a}_t) \right) \right] \quad (2.34)$$

donde $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente.

Concluida la optimización de θ^{i_m} , se computa la razón post-optimización:

$$\rho_{\text{post}}^{i_m} \doteq \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})} \quad (2.35)$$

y se propaga recursivamente hacia los agentes pendientes mediante:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t). \quad (2.36)$$

Garantía de mejora monótona

El principio de mejora monótona en aprendizaje por refuerzo formaliza que la secuencia de retornos esperados de la política conjunta no disminuye entre ciclos sucesivos de entrenamiento, comportándose como una función monótonamente no decreciente respecto de la iteración k :

$$J(\pi_{k+1}) \geq J(\pi_k). \quad (2.37)$$

Para políticas heterogéneas, Kuba et al. [KCW⁺22] demostraron analíticamente esta propiedad para el algoritmo HATRPO (*Heterogeneous-Agent Trust Region Policy Optimization*) bajo los supuestos del juego markoviano cooperativo con recompensa compartida y una actualización restringida por la divergencia KL. En la práctica computacional, resolver las restricciones cuadráticas de HATRPO en redes neuronales profundas resulta computacionalmente restrictivo. Por ello, HAPPO sustituye la cota formal de divergencia KL por el objetivo sustituto recortado CLIP de primer orden de la ecuación (2.34).

Esta sustitución constituye una aproximación pragmática: al igual que ocurre con PPO frente a TRPO en el caso de agente único, la garantía matemática demostrada para HATRPO no se transfiere automáticamente a la variante con recorte HAPPO, aun cuando esta última exhibe un comportamiento empírico notablemente robusto [KCW⁺22, ZKF⁺24]. Además, en redes urbanas con señales de recompensa local-vecinales, el procedimiento no garantiza una convergencia monótona analítica sobre una función objetivo global única; en cambio, opera como un mecanismo ordenado de optimización por bloques que atenúa la no estacionariedad y las colisiones de gradiente entre políticas concurrentes [WZK⁺23, ZKF⁺24]. La figura 2.20 resume este esquema de propagación secuencial.

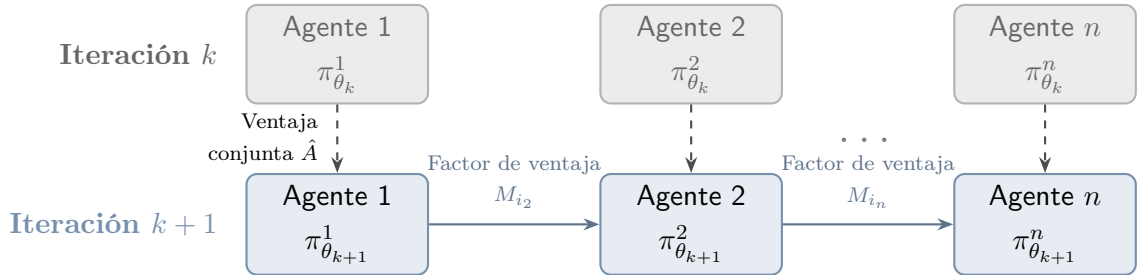


Figura 2.20: Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m} , que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.

El algoritmo 4 presenta la secuencia completa de optimización de HAPPO.

Algoritmo 4 Algoritmo Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

Entrada: entorno multiagente, conjunto $\mathcal{N}$ de n agentes, horizonte de recolección, γ , λ_{GAE} y ϵ_{clip}

Salida: políticas heterogéneas entrenadas $\{\theta^i\}_{i=1}^n$ y crítico centralizado ϕ

- 1: Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
- 2: **para** iteración = 1, 2, ... **hacer**
- 3: Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta π_θ
- 4: Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
- 5: Calcular los retornos objetivo R_t^{target} utilizados para ajustar el crítico
- 6: Sortear una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
- 7: Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \dots, n\}$
- 8: **para** $m = 1 \dots n$ **hacer**
- 9: Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
- 10: Actualizar θ^{i_m} a $\theta_{k+1}^{i_m}$ maximizando el objetivo recortado de la ecuación (2.34)
- 11: Recalcular la razón post-optimización: $\rho_{\text{post}}^{i_m} \leftarrow \frac{\pi_{\theta_{k+1}^{i_m}}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}(a_t^{i_m} | o_t^{i_m})}$
- 12: **para** $j = m + 1 \dots n$ **hacer**
- 13: Actualizar el factor modificador para agentes subsecuentes:
 $M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_j}(s_t, \mathbf{a}_t)$
- 14: **fin para**
- 15: **fin para**
- 16: Actualizar el crítico centralizado ϕ mediante descenso de gradiente:
 $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[(V_\phi(s_t) - R_t^{\text{target}})^2 \right]$
- 17: **fin para**

El procedimiento de HAPPO organiza secuencialmente la actualización de los actores locales mediante la permutación aleatoria $i_{1:n}$ y la propagación recursiva de los factores modificadores M_{i_m} . Tras optimizar sucesivamente a los n agentes sobre el *batch* de transiciones $\mathcal{D}$, el crítico centralizado V_ϕ ajusta sus parámetros mediante descenso de gradiente sobre los retornos acumulados. Durante la etapa de ejecución descentralizada, cada política local selecciona a_i a partir de o_i , sin requerir evaluar el crítico centralizado ni comunicación entre controladores para inferir su acción [KCW⁺22].

En la instanciación para control semafórico de este trabajo, la secuencia conserva esta estructura de permutación y actualización sucesiva, pero inicializa los factores M_{i_m} con las ventajas individuales $\hat{A}_i(s, \mathbf{a})$ calculadas mediante el crítico centralizado multi-salida a partir de las señales de recompensa compuestas (sección 3.5.3), propagándolos mediante la razón $\rho_{\text{post}}^{i_m}$ tras optimizar a cada actor. Esta formulación explicita la adaptación

práctica frente a la demostración teórica canónica —concebida para una recompensa escalar global compartida—, operando como un procedimiento de coordinación distribuida y estabilización frente a la no estacionariedad del entorno de simulación.

Para finalizar el estudio comparativo de los algoritmos derivados de PPO en escenarios MARL, la tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Tabla 2.3: Comparación sintética de familias algorítmicas PPO en entornos multiagente cooperativos.

Eje de análisis	PPO de agente único	IPPO independiente	MAPPO / CTDE	HAPPO / HARL
Paradigma	agente único	IL (descentralizado)	CTDE (centralizado/desc.)	CTDE; familia HARL
Actores	único $\pi_\theta(a \mid s)$	n políticas locales	actores locales; puede compartir parámetros	n políticas heterogéneas
Arquitectura del crítico	único $V_\phi(s)$	n críticos locales $V_{\phi_i}(o_i)$	un crítico centralizado $V_\phi(s)$	un crítico centralizado $V_\phi(s)$
Optimización	PPO estándar	PPO local simultáneo	PPO local con crítico global	actualización secuencial $i_{1:n}$
Señal de ventaja	ventaja local $\hat{A}(s, a)$	ventaja local $\hat{A}_i(o_i, a_i)$	ventaja conjunta $\hat{A}(s, \mathbf{a})$	factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora monótona	No estricta para PPO	Sin garantía general	Sin garantía estricta	Teórica bajo supuestos; HAPPO-clip es aproximado
Tipo de agente	No aplica	Depende de la implementación	Pensado para agentes homogéneos; la compartición de parámetros es opcional	Pensado para agentes heterogéneos

2.3.7 Antecedentes recientes del control semafórico multiagente

Para orientar la comparación, los antecedentes seleccionados se organizan según tres ejes: coordinación y entrenamiento, representación de las observaciones, y tratamiento de la heterogeneidad o la equidad entre intersecciones. Se priorizan trabajos directamente relacionados con la formulación y las variables de este trabajo.

El control coordinado de múltiples intersecciones mediante aprendizaje por refuerzo constituye un área de investigación activa. Entre los enfoques iniciales se encuentran aquellos que asignaban un agente a cada semáforo y utilizaban información local sobre colas, fases o vehículos detenidos. Sin embargo, la optimización aislada de cada intersección no contempla que una acción modifica el flujo recibido por los cruces vecinos. Por este motivo, la literatura evolucionó hacia formulaciones cooperativas que incorporan mecanismos de coordinación espacial, representaciones estructuradas del estado y esquemas de entrenamiento centralizado [ACS24, LML⁺25].

Coordinación y entrenamiento

En la consolidación del control semafórico coordinado mediante aprendizaje profundo, varios trabajos abordaron de manera temprana la asignación de crédito y la no estacionariedad en redes viales:

- **Control actor-crítico con descuento espacial (MA2C):** Chu et al. [CWCL19] propusieron una arquitectura de ventaja actor-crítico multiagente donde cada intersección incorpora a su función de valor y recompensa la información de sus vecinos inmediatos atenuada por un factor de descuento espacial. Este trabajo fundamentó conceptualmente y evidenció de manera empírica que una recompensa de equipo global en redes extensas induce severos problemas de asignación de crédito, mientras que las recompensas regionalmente estructuradas permiten coordinar corredores sin requerir estados globales inmanejables.
- **Coordinación basada en presión vehicular (PressLight):** Wei et al. [WCZ⁺19] fundamentaron su función de recompensa en el principio de *Max-Pressure* de la ingeniería de transporte, que en la teoría de control de redes garantiza estabilidad de cola bajo supuestos específicos de un esquema de servicio discreto. PressLight incorpora ese principio como recompensa de aprendizaje por refuerzo al maximizar la diferencia de vehículos detenidos entre corrientes entrantes y salientes. Esa incorporación no hereda la garantía formal de estabilidad y coordina de manera implícita las intersecciones adyacentes sin necesidad de comunicación explícita.
- **Atención en grafos para cooperación a nivel de red (CoLight):** Wei et al. [WXZ⁺19] introdujeron redes de atención en grafos para permitir que los agentes semafóricos intercambien dinámicamente representaciones de sus estados con cruces vecinos, adaptando la influencia de cada intersección adyacente según las condiciones variables de congestión.
- **Límites del aprendizaje independiente en redes viales:** Ault y Sharon [AS22] analizaron las limitaciones de los enfoques independientes frente a formulaciones cooperativas en redes de múltiples intersecciones, evidenciando un deterioro progresivo

en el desempeño y en la fluidez del tránsito a medida que la demanda se incrementa debido a la falta de coordinación y a la no estacionariedad del entorno.

A partir de estas bases fundacionales, diversas investigaciones recientes han explorado mecanismos alternativos de optimización y distribución de cómputo. Wu et al. [WWS⁺22] estudian el control de redes de gran escala mediante agentes distribuidos de aprendizaje por refuerzo profundo y una arquitectura de cómputo distribuido. Proponen dos variantes, Nash-A2C y Nash-A3C, que incorporan un equilibrio de Nash en el aprendizaje por refuerzo y se despliegan en una arquitectura distribuida para simulación y control. En sus experimentos reportan reducciones del tiempo de congestión y de la demora total de red frente a controladores de referencia. Este antecedente es pertinente como referencia de escalabilidad, pero no es equivalente a la formulación de este trabajo: aquí la ejecución también es local por intersección, mientras que la coordinación se estudia mediante la señal individual compuesta local-vecinal y el entrenamiento centralizado, sin introducir una capa de cómputo fog ni un equilibrio de Nash.

En una formulación cooperativa basada en DQN, Haddad et al. [HHA22] incorporan a la función de aprendizaje de cada intersección información sobre los estados, acciones y recompensas de sus vecinos. El estudio informa mejoras en tiempo de espera, longitud de cola y emisiones de dióxido de carbono frente a controladores de referencia. Esta evidencia respalda la necesidad de considerar el efecto de las decisiones vecinas, pero su mecanismo de coordinación es una transferencia explícita de información hacia la pérdida de DQN. En el presente trabajo, IPPO e IDQN implementan el paradigma de aprendizaje independiente puro y reciben una señal de recompensa estrictamente local ($R_i = L_i$, $\rho_{\text{coop}} = 0$); MAPPO y HAPPO incorporan la coordinación vecinal en la recompensa ($\rho_{\text{coop}} > 0$) y un crítico centralizado durante el entrenamiento. Estas alternativas no son equivalentes al mecanismo propuesto por Haddad et al., sino procedimientos de coordinación distintos que se comparan bajo una misma representación y escenario.

Fang et al. [FYX⁺23] proponen un método de coordinación de red que formula el control semafórico como un problema multiobjetivo: combina eficiencia, seguridad y coordinación espacial entre intersecciones mediante un esquema de entrenamiento centralizado con ejecución descentralizada. Además de reportar resultados en redes sintéticas y arterias reales, el trabajo destaca que las dependencias espacio-temporales entre agentes deben representarse explícitamente. Su propuesta ofrece un contraste con la señal individual compuesta y la representación interpretable utilizadas en este trabajo, que no incorporan una optimización multiobjetivo ni una representación latente basada en atención.

Representación del estado

En cuanto a la representación agregada y el aprendizaje basado en valor, Kolat et al. [KKBA23] proponen un enfoque DQN multiagente con parámetros compartidos. Los agentes observan cantidades escaladas de vehículos detenidos y reciben una recompen-

sa que penaliza el desequilibrio entre los accesos. Los resultados muestran mejoras en el tiempo de viaje y el consumo de combustible respecto de un controlador de ciclo fijo. Sirve como referencia para contrastar una representación agregada del estado y una coordinación inducida por la recompensa con la evaluación de IDQN y de métodos basados en políticas realizada en esta investigación.

Dentro de la línea centrada en la representación del estado, Yang [Yan23] propone HG-M2I, una arquitectura que combina información espacial y temporal mediante un grafo jerárquico. El modelo integra estados de los agentes y de sus vecinos en representaciones latentes utilizadas por las políticas. Posteriormente, Chen et al. [CYL⁺24] presentan CoevoMARL, que utiliza atención en grafos y una unidad recurrente para aprender cómo varía en el tiempo la dependencia entre intersecciones. Mientras HG-M2I organiza la información según una jerarquía espacial, CoevoMARL adapta las relaciones entre agentes a la evolución del tránsito. Ambos trabajos muestran la importancia de construir observaciones informativas, aunque recurren a representaciones latentes aprendidas, a diferencia de la transformación explícita e interpretable evaluada en el presente trabajo.

En el caso de una representación basada en celdas, Deng et al. [DLSZ26] introducen una partición de longitud variable. Los accesos se dividen mediante una función que combina componentes logarítmicos y lineales, y cada celda almacena la cantidad de vehículos, la velocidad media y la ocupación espacial. Esta partición asigna mayor detalle a las zonas próximas a la intersección y comprime las regiones más alejadas. Entre los trabajos seleccionados, es el antecedente más cercano al uso de una transformación logarítmica en la representación semafórica; sin embargo, el logaritmo determina la longitud espacial de las celdas y no codifica el tiempo de espera acumulado por carril.

Heterogeneidad y equidad

Ren et al. [RDZ⁺24] abordan la coordinación bajo demandas heterogéneas mediante un esquema de dos capas. Cada agente combina su ventaja local con la de las intersecciones vecinas a través de un factor de cooperación adaptable, que luego se ajusta según el rendimiento global de la red. El método considera la demora y las emisiones vehiculares, y coordina los objetivos locales y globales mediante ese factor adaptable. Este antecedente permite contrastar la coordinación adaptativa de las señales de aprendizaje con la heterogeneidad de políticas considerada por HAPPO, aunque no emplea su procedimiento de actualización secuencial.

En la coordinación mediante entrenamiento centralizado, Liu et al. [LZF⁺25] formulan el control semafórico como un juego parcialmente observable con restricciones y proponen VF-MAPPO. En este método, la equidad se expresa mediante una cota sobre el tiempo máximo de espera de los vehículos, mientras que un crítico centralizado con atención espaciotemporal estima tanto la eficiencia de la red como el cumplimiento de la restricción. Esta formulación resulta especialmente pertinente para el presente trabajo porque vincula

con una variable temporal de operación el paradigma de entrenamiento centralizado con ejecución descentralizada. No obstante, el tiempo de espera se incorpora como restricción de optimización y no mediante una codificación no lineal de la observación.

En el tratamiento de agentes heterogéneos, otro enfoque es HAPS-PPO, presentado por Lu et al. [LFYZ25]. El método completa con relleno las observaciones de distinta dimensión y agrupa a los agentes según sus espacios de acción, asignando cabezales de política específicos a cada grupo. Esto permite coordinar intersecciones con geometrías y conjuntos de fases diferentes. A pesar de la similitud entre las siglas, HAPS-PPO no corresponde a HAPPO, ya que no emplea la actualización secuencial ni la descomposición de la ventaja explicadas en la sección 2.3.6.

Posicionamiento del presente trabajo

La selección analizada sugiere que el desempeño del control semafórico multiagente depende de tres decisiones relacionadas: el algoritmo utilizado para aprender y coordinar las políticas, la representación de las condiciones de operación y el tratamiento de intersecciones con demandas, geometrías o espacios de acción diferentes. Los enfoques basados en DQN proporcionan una referencia para el aprendizaje por valor. Los modelos de grafos construyen representaciones latentes de las relaciones espaciales, mientras que las variantes de MAPPO incorporan información global durante el entrenamiento. Los esquemas de cooperación adaptable o con múltiples cabezales atienden distintas formas de heterogeneidad. La codificación por carril y la discretización de variables de tránsito también cuentan con antecedentes en control semafórico [GR18]; aquí se emplean para representar la espera acumulada por carril mediante la transformación especificada en la metodología.

Entre los antecedentes considerados, HeteroLight aborda la heterogeneidad mediante una arquitectura de parámetros compartidos con módulos que extraen características generales y específicas de cada intersección [ZLFS24], mientras que MATLight se apoya en un marco de aprendizaje especular para agentes heterogéneos (*heterogeneous-agent mirror learning*) y emplea un Transformer para procesar secuencias de estados [ZWL24]. Son mecanismos distintos de la adaptación HAPPO implementada aquí: no se los presenta como equivalentes ni como evidencia de que HAPPO sea superior. En las fuentes consultadas hasta septiembre de 2026 no se localizó una evaluación conjunta de IDQN, IPPO, MAPPO y HAPPO bajo una misma formulación, ni una aplicación directa del HAPPO original al control de fases semafóricas urbanas. Esta observación delimita la búsqueda realizada y no permite concluir que tales estudios no existan.

En particular, este trabajo explora una codificación híbrida logarítmica-lineal del tiempo de espera acumulado por carril. Al asignar valores continuos a un conjunto finito de intervalos, la cuantización agrupa esperas dentro de una misma categoría y limita los valores representables; la saturación también reúne los valores que alcanzan la cota. En aprendizaje por refuerzo, una abstracción de estados puede simplificar el problema y con-

servar comportamiento cercano al óptimo cuando los estados agrupados son suficientemente similares, bajo las condiciones teóricas correspondientes [AHL16]. Ese resultado no constituye una garantía para esta codificación: aquí se transforman observaciones locales en un problema parcialmente observable y no se demuestra que los valores agrupados sean equivalentes para la política óptima. La resolución no uniforme del codificador asigna más niveles a esperas bajas, lo que permite distinguirlas con mayor detalle; es una hipótesis de diseño, pues la cuantización también puede descartar distinciones relevantes, en particular cerca de los umbrales entre intervalos. Su efecto se examina como parte del sistema completo, sin una ablación que lo aisle frente a otras representaciones. Esta propuesta se diferencia de las observaciones agregadas utilizadas por los enfoques DQN, de las proyecciones latentes basadas en grafos y de la partición espacial de Deng et al. [DLSZ26], que divide los accesos y no codifica la espera acumulada por carril.

A su vez, la evaluación bajo IDQN, IPPO, MAPPO y HAPPO examina el desempeño de cuatro procedimientos completos de aprendizaje y coordinación, cada uno con su receta ajustada, sobre una misma representación de observación. En MAPPO se utiliza un crítico centralizado compartido de múltiples salidas, condicionado por la concatenación de observaciones locales, con valores y ventajas calculados por agente. En HAPPO, las ventajas individuales estimadas mediante GAE se reponderan en las actualizaciones secuenciales de los actores; esta adaptación no aplica directamente el lema canónico de descomposición de ventajas. Los críticos estiman valores durante el entrenamiento en ambos procedimientos, pero no se atribuye al crítico la diferencia entre ellos ni se postula una actualización secuencial del crítico. La comparación estima diferencias entre procedimientos integrados y no aísla el efecto causal de un componente individual —como el crítico centralizado o la actualización secuencial—, lo cual requeriría ablaciones con el resto de los factores constantes. De este modo, la solución puede contrastarse con los antecedentes recientes en términos de representación, paradigma de entrenamiento y capacidad para tratar agentes heterogéneos. El capítulo 3 describe cómo se concreta esta propuesta en el diseño experimental.

Metodología y diseño experimental

Este capítulo describe el diseño experimental, la operacionalización de las variables, la construcción de los escenarios de simulación y la arquitectura computacional utilizada. También presenta los protocolos de entrenamiento y evaluación empleados para comparar MAPPO y HAPPO con las líneas de base IPPO e IDQN. El estudio adopta un enfoque cuantitativo y un diseño comparativo con replicaciones independientes y comparaciones pareadas por semilla: cada ejecución de entrenamiento produce una política y cada política se evalúa bajo semillas y condiciones de demanda previamente definidas. La simulación microscópica del tránsito funciona como un entorno controlado; el escenario de la Ciudad de Mendoza se interpreta como una construcción sintética restringida por datos parciales, no como un gemelo digital completo.

3.1 Enfoque y diseño de investigación

Este trabajo adopta un **diseño experimental cuantitativo comparativo con replicaciones independientes y comparaciones pareadas por semilla**. Los factores principales son la familia algorítmica, la arquitectura de la red neuronal, el grado de cooperación vecinal ρ_{coop} y el escenario de tránsito. Las políticas se comparan dentro de cada escenario y, de forma complementaria, se estudia cómo cambia su comportamiento al aumentar la escala o modificar la topología. Dado que también cambian la demanda y el número de agentes, las comparaciones entre escenarios se expresan mediante métricas absolutas y mediante variaciones relativas respecto del controlador de tiempo fijo del mismo escenario.

La simulación microscópica se utiliza como plataforma experimental por tres razones. El simulador utilizado es SUMO [DLR26]:

1. **Seguridad y viabilidad operacional:** evaluar algoritmos estocásticos de aprendizaje en tiempo real sobre una red urbana real implicaría riesgos para la seguridad vial y la fluidez del tránsito.
2. **Aislamiento de variables exógenas:** la simulación permite controlar perturbaciones que no forman parte del modelo, como condiciones meteorológicas, siniestros viales o fluctuaciones externas.
3. **Replicabilidad:** la fijación de semillas pseudoaleatorias permite someter los distintos controladores a las mismas condiciones de demanda.

Para evaluar las políticas se emplean métricas físicas extraídas de la telemetría microscópica de SUMO, principalmente del archivo `TripInfo`. La métrica primaria de la búsqueda de hiperparámetros es la espera media por vehículo insertado; las demás métricas se utilizan para interpretar eficiencia, atención de la demanda, equidad y estabilidad:

- **Espera media de los vehículos insertados ($\bar{w}_{\text{ins}}$):** métrica operacional obtenida de `TripInfo`:

$$\bar{w}_{\text{ins}} = \frac{\sum_{v \in \mathcal{V}_{\text{ins}}} w_v}{|\mathcal{V}_{\text{ins}}|}, \quad w_v = \text{waitingTime}_v. \quad (3.1)$$

En la ecuación (3.1), $\mathcal{V}_{\text{ins}}$ es el conjunto de vehículos insertados en la ventana, w_v es el tiempo de espera del vehículo v en segundos y $|\mathcal{V}_{\text{ins}}|$ es su cardinalidad. SUMO define `waitingTime` como el tiempo con velocidad menor o igual a 0,1 m/s; por tanto, la métrica es un promedio en segundos por vehículo y no una suma en vehículo-segundo. El entorno conserva además la espera integrada de los vehículos expuestos durante la ventana, pero esta magnitud no constituye el objetivo principal.

- **Tasa de completitud de viajes (*Completion Rate*) (CR):** proporción de la demanda cargada que completa su itinerario dentro del horizonte de evaluación. La métrica también contabiliza los vehículos retenidos en los accesos de origen por sobresaturación:

$$\text{CR} = \frac{\text{throughput}}{\text{departed} + \text{pending_vehicles_end}} \quad (3.2)$$

La ecuación (3.2) relaciona los vehículos completados con la demanda que ingresó o quedó retenida. Todos sus términos son conteos del mismo horizonte: `throughput` corresponde a los vehículos que completaron su itinerario, `departed` a los que ingresaron a la calzada y `pending_vehicles_end` a los que permanecieron en origen sin poder insertarse al cierre de la ventana. Cuando el simulador no descarta vehículos, se cumple la conservación $\text{loaded} \approx \text{departed} + \text{pending_vehicles_end}$ y el denominador de CR coincide con la carga de la ventana [DLR26]. También se registran la Tasa de inserción (*Insertion Rate*) (IR) ($\text{IR} = \frac{\text{inserted}}{\text{loaded}}$) y la Tasa de completitud de los

vehículos insertados (*Completion of Inserted Vehicles*) (CI) ($CI = \frac{\text{arrived_inserted}}{\text{inserted}}$). Esta última tasa se restringe a los vehículos insertados durante la ventana que también completan su itinerario en ella; evita mezclar en el numerador vehículos presentes desde el calentamiento. Las tres tasas se informan conjuntamente con la métrica primaria porque un promedio calculado sobre los vehículos que circulan puede mejorar si un controlador restringe la inserción de demanda sin aliviar la congestión, efecto documentado en la evaluación de control semafórico por aprendizaje por refuerzo [JLW⁺24].

- **Equidad espacial de demoras (coeficiente de Gini, G_{wait}):** cuantifica la desigualdad en la distribución del tiempo de espera medio entre las intersecciones semaforizadas de la red:

$$G_{\text{wait}} = \frac{2 \sum_{i=1}^N i \cdot \bar{w}_{(i)} - (N+1) \sum_{i=1}^N \bar{w}_{(i)}}{N \sum_{i=1}^N \bar{w}_{(i)}} \quad (3.3)$$

La ecuación (3.3) se calcula después de ordenar los tiempos de espera medios. En ella, $\bar{w}_{(1)} \leq \bar{w}_{(2)} \leq \dots \leq \bar{w}_{(N)}$ son los tiempos de espera medios ordenados de cada intersección y N es el total de intersecciones semaforizadas. Cada $\bar{w}_i$ es el promedio temporal, sobre los pasos de decisión de la ventana controlada, de la espera media de los vehículos presentes en los carriles controlados por esa intersección. Las intersecciones entran con peso uniforme, sin ponderar por demanda: el índice describe la distribución interseccional del servicio, no la distribución entre vehículos individuales. Esta métrica se complementa con la desviación estándar inter-intersección (σ_{wait}) para evaluar si las políticas aprendidas distribuyen equitativamente el servicio o si sacrifican accesos secundarios en favor de arterias dominantes.

- **Pérdida temporal ($\bar{\ell}$):** promedio de `timeLoss` por vehículo insertado, que cuantifica el tiempo perdido respecto de la velocidad ideal del vehículo. Se registra como métrica secundaria porque incluye desaceleraciones y conflictos de circulación que no siempre se reflejan en la espera.

El archivo `TripInfo` solo incluye viajes finalizados cuando se escribe durante la ejecución; por eso el entorno activa `-tripinfo-output.write-unfinished` y cierra la conexión al terminar el episodio. Las métricas se calculan entonces sobre todos los registros válidos cuyo atributo de partida pertenece a la ventana, incluidos los viajes que aún no llegaron a destino. La documentación de SUMO define de forma independiente `waitingTime` y `timeLoss`; no se los intercambia en el análisis.

La figura 3.1 resume las siete etapas de la investigación. En este contexto, *etapa* designa una parte del procedimiento metodológico y no una fase del ciclo semafórico.

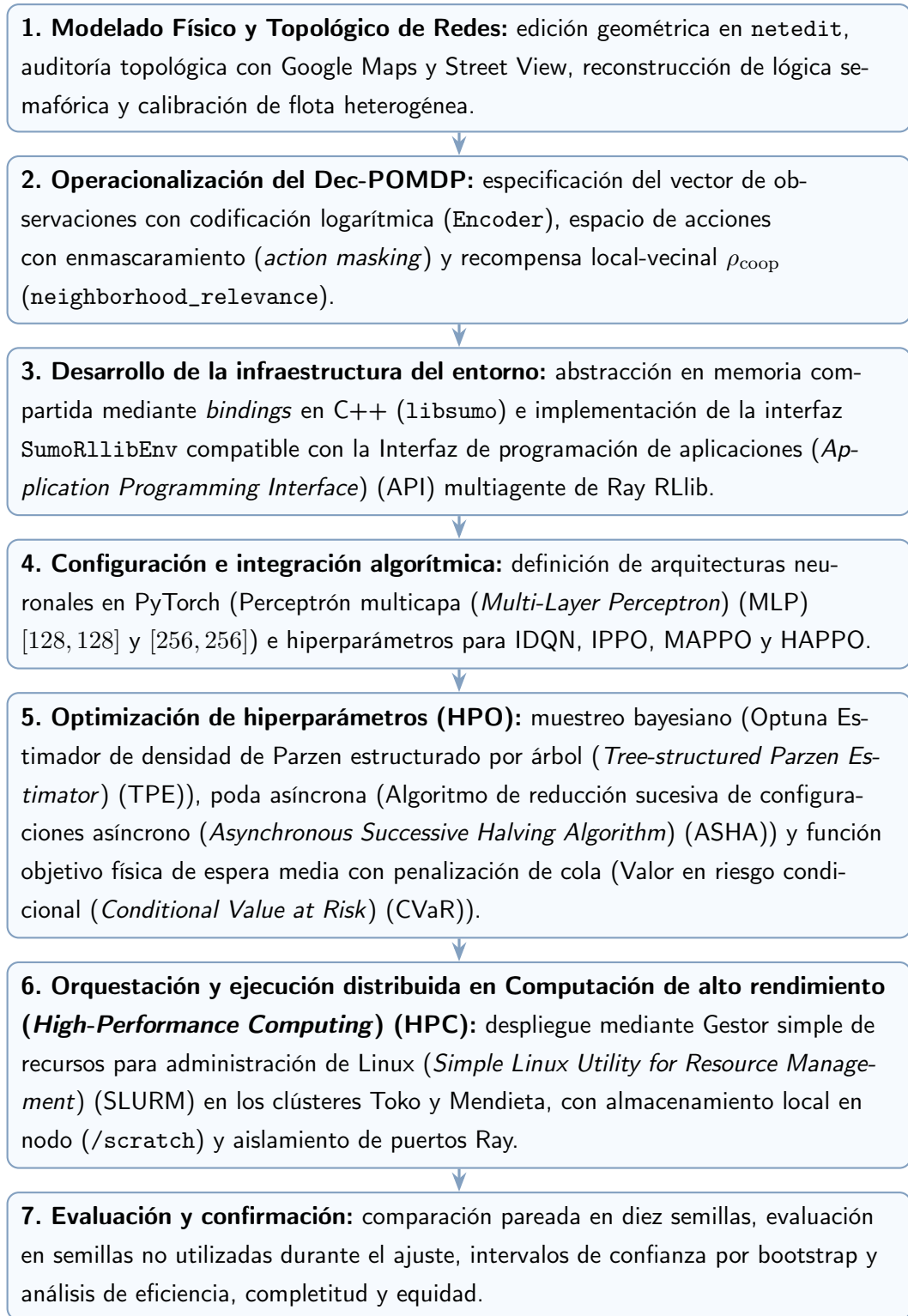


Figura 3.1: Flujo metodológico de la investigación experimental. Los siete recuadros representan, de arriba hacia abajo, las etapas de modelado de redes, formulación del problema, desarrollo del entorno, integración algorítmica, optimización de hiperparámetros, ejecución distribuida y evaluación estadística; las flechas indican la dependencia secuencial entre ellas.

La figura 3.1 comienza con la construcción física y topológica de las redes que sirven

como escenarios experimentales. Sobre esos escenarios se define el problema de decisión: qué observa cada agente, qué acciones puede ejecutar y qué recompensa recibe. La tercera etapa implementa esas definiciones en el entorno de simulación, y la cuarta incorpora los algoritmos y sus redes neuronales. Una vez que el sistema puede entrenarse, la quinta etapa busca configuraciones de hiperparámetros. La sexta distribuye las ejecuciones en la infraestructura de cómputo, y la séptima compara las políticas mediante métricas físicas y análisis de variabilidad. Las flechas verticales expresan la dependencia metodológica y no representan ejecución simultánea.

3.2 Especificación del ecosistema tecnológico

Para garantizar la reproducibilidad científica, la tabla 3.1 detalla las librerías, marcos de trabajo y plataformas de cómputo sobre las cuales se desarrolló y ejecutó el entorno `marlTLC`. En esta sección, la Telemetría es el conjunto de mediciones que el simulador registra durante la ejecución; un Marco de trabajo (*framework*) proporciona interfaces y componentes reutilizables para construir el sistema; un *Binding* conecta una biblioteca escrita en un lenguaje con otro, y la Serialización convierte un objeto o mensaje en una representación que puede almacenarse o transmitirse. Los procesos denominados *env runners* generan trayectorias al interactuar con el entorno y los *learners* actualizan los parámetros de las redes a partir de esas trayectorias. Estas definiciones permiten interpretar la tabla sin suponer familiaridad previa con la infraestructura de software.

Tabla 3.1: Especificación de las tecnologías y herramientas utilizadas en la investigación.

Componente	Herramienta / Versión	Función Metodológica
Lenguaje de Programación	Python 3.10+	Núcleo del sistema, tipado estático (<code>typing.Protocol</code>) y scripts de orquestación.
Simulador de Tránsito	SUMO 1.20+ / TraCI	Simulación microscópica, dinámica cinemática y telemetría de red.
<i>Bindings</i> de Simulación	<code>libsumo</code> (C++)	Ejecución de la simulación en memoria compartida mediante <i>bindings</i> en C++ (<code>LIBSUMO_AS_TRACI=1</code>).
Marco de trabajo MARL (<i>framework</i>)	Ray RLlib 2.x [LLN ⁺ 18]	Abstracción de entornos multiagente (<code>MultiAgentEnv</code> , <code>RLModuleSpec</code> , <code>LearnerGroup</code>).
Aprendizaje Profundo	PyTorch 2.x	Modelado neuronal y optimización (Actor-Crítico MLP, <i>action masking</i>).
Optimización Automática	Optuna 3.x [ASY ⁺ 19]	Muestreo Bayesiano TPE y poda asíncrona de pruebas (ASHA [LJR ⁺ 20]).
Infraestructura HPC	Clústeres Toko y Mendieta	Cómputo distribuido multinúcleo gestionado vía SLURM con Entrada/salida (<i>Input/Output</i>) (E/S) local (<i>/scratch</i>).

Estas capas se integran de la siguiente manera. Python coordina la ejecución y Ray RLlib gestiona los procesos concurrentes de recolección de trayectorias (*env runners*). PyTorch calcula las derivadas y optimiza las redes neuronales en los módulos de aprendizaje (*learners*). `SumoRllibEnv` encapsula la lógica episódica y `libsumo` ejecuta los cálculos de cinemática vehicular en la memoria del proceso, sin comunicación por red ni serialización Protocolo de control de transmisión (*Transmission Control Protocol*) (TCP).

3.3 Diseño y calibración de escenarios de tránsito en SUMO

La evaluación empírica se realiza sobre cuatro escenarios de red en SUMO. Las redes se construyen con la herramienta gráfica `netedit`, los utilitarios de conversión `netconvert` y scripts de generación en Python. En conjunto, los escenarios representan distintas complejidades topológicas y condiciones de flujo.

3.3.1 Modelado geométrico y sanitización topológica

La construcción de las redes de simulación sigue un procedimiento de diseño y validación en SUMO:

1. **Diseño geométrico y accesos:** definición de calzadas, cantidad de carriles por aproximación y longitudes de almacenamiento en `netedit` o a partir de datos cartográficos de OSM.
2. **Auditoría y corrección topológica:** verificación de los sentidos de circulación, la cantidad de carriles, las conexiones de giro y las maniobras prohibidas. La revisión se contrasta con imágenes satelitales y vistas de calle de Google Maps y Google Street View.
3. **Configuración semafórica:** las configuraciones de los semáforos estáticos corresponden a los planes generados por defecto por SUMO (`netconvert -tls.rebuild`) al compilar la red. Los programas de fases y los tiempos de ciclo originales se mantienen sin cambios. La correspondencia entre los enlaces físicos y los índices de control semafórico (`linkIndex`) se recalcula para conservar la consistencia topológica.

3.3.2 Escenarios sintéticos: 3×1 , 3×3 y 4×4 $_H$

Todas las redes sintéticas comparten las siguientes reglas operacionales:

- **Límite de velocidad:** 13.89 m/s (50 km/h) en todas las arterias urbanas.
- **Flota vehicular heterogénea:** se definieron tres perfiles cinemáticos en el archivo `urban_vehicle_mix.add.xml`, a partir de los modelos disponibles en el simulador [DLR26]:
 1. **SLOWVEHTYPE** (25%): perfil de menor aceleración y velocidad máxima ($a = 1.9 \text{ m/s}^2$, $d = 4.0 \text{ m/s}^2$, $v_{\max} = 8.33 \text{ m/s} = 30 \text{ km/h}$).
 2. **MEDVEHTYPE** (60%): perfil de referencia. Parámetros: $a = 2.6 \text{ m/s}^2$ y $d = 4.5 \text{ m/s}^2$; $v_{\max} = 13.89 \text{ m/s} = 50 \text{ km/h}$.
 3. **FASTVEHTYPE** (15%): perfil de mayor aceleración, $a = 3.1 \text{ m/s}^2$ y $d = 5.0 \text{ m/s}^2$; $v_{\max} = 19.44 \text{ m/s} = 70 \text{ km/h}$.

Parámetros compartidos: longitud 4.5 m, brecha mínima de seguridad (`minGap`) 2.5 m e imperfección del conductor de Krauss $\sigma = 0.5$.

- **Generación de demanda por zonas (TAZ):** las rutas se generan mediante `random_routes_taz.py` a partir de matrices de zonas de asignación de tránsito. Los tres escenarios sintéticos almacenan flujos con el atributo `vehsPerHour`; la demanda nominal se expresa en vehículos por hora y SUMO inserta los vehículos conforme al flujo solicitado. No se atribuye a estas redes un proceso de llegadas exponencial.

- **Calibración por bisección:** los flujos totales de los tres escenarios sintéticos se ajustaron mediante bisección bajo control semafórico de tiempo fijo, con el objetivo de obtener una tasa de completitud sobre los vehículos insertados (CI) próxima a 0,70. El horizonte de referencia fue de 4200 s, compuesto por 600 s de calentamiento y 3600 s de control.

La tabla 3.2 resume las características verificadas y los flujos nominales obtenidos para los tres escenarios sintéticos.

Tabla 3.2: Características verificadas de los escenarios sintéticos.

Escenario	Agentes	Tramos	Carriles	Flujo nominal (veh/h)
3×1	3	20	40	3.260
3×3	9	48	96	7.087
$4 \times 4_H$	16	71	166	7.202

Los flujos finales de 3.260 veh/h para 3×1 , 7.087 veh/h para 3×3 y 7.202 veh/h para $4 \times 4_H$ fueron los obtenidos con esa calibración. El objetivo de completitud cercano a 0,70 fija condiciones de congestión comparables para el estudio sintético; no establece una equivalencia exacta de demanda entre topologías ni valida una demanda urbana.

Las tres topologías sintéticas cumplen funciones analíticas complementarias:

- **Corredor arterial 3×1 (3.260 veh/h):** tres intersecciones dispuestas en línea. El perfil canónico asigna explícitamente 40% al par $3 \rightarrow 4$ y 35% al par $4 \rightarrow 3$; el remanente se distribuye de forma reproducible entre los pares válidos restantes. Su disposición permite estudiar la propagación longitudinal de colas y la coordinación de fases sobre las corrientes predominantes.
- **Malla simétrica 3×3 (7.087 veh/h):** nueve intersecciones organizadas en una cuadrícula cerrada. El perfil canónico asigna 20%, 17,5%, 17,5% y 20% a los pares $5 \rightarrow 6$, $6 \rightarrow 5$, $10 \rightarrow 1$ y $1 \rightarrow 10$, respectivamente; el 25% restante se asigna entre los pares válidos. La interconexión permite estudiar el desbordamiento de colas (*spillback*) y la estabilidad de MARL bajo congestión severa.
- **Cuadrícula $4 \times 4_H$ (7.202 veh/h):** dieciséis intersecciones con geometría heterogénea. El perfil canónico asigna 20%, 17,5%, 17,5% y 20% a los pares $5 \rightarrow 9$, $6 \rightarrow 5$, $13 \rightarrow 1$ y $1 \rightarrow 13$, respectivamente; el 25% restante se distribuye entre los pares válidos.

3.3.3 Escenario urbano de la Ciudad de Mendoza

Para estudiar el comportamiento de los controladores en una red urbana de mayor escala, se construyó un escenario del microcentro de la Ciudad de Mendoza a partir de

su infraestructura topológica y de una serie temporal parcial de aforo vehicular captada mediante una cámara termográfica. La red representa la geometría urbana, pero su demanda completa se aproxima mediante supuestos espaciales y un volumen nominal; por eso el escenario no se presenta como una validación del tránsito total de la ciudad. El diseño experimental prevé un ajuste de hiperparámetros, entrenamiento y evaluación propios para cada uno de los cuatro algoritmos en este escenario; no se transfieren pesos ni configuraciones finales desde las redes sintéticas.

Posición metodológica: escenario restringido por datos

El escenario de Mendoza se considera un **escenario sintético restringido por datos** (*data-constrained synthetic scenario*), y no un gemelo digital completo.

La inferencia de una matriz Origen-destino (*Origin-Destination*) (OD) completa a partir de volúmenes medidos en pocos puntos de control es un problema inverso subdeterminado y mal condicionado [CyMR18, PW16]. Para una red con $|\mathcal{Z}| = 37$ zonas de asignación de tránsito (TAZ), existen $|\mathcal{Z}|(|\mathcal{Z}| - 1) = 1.332$ pares direccionales potenciales, de los cuales 520 son físicamente alcanzables. Este número contrasta con la cantidad limitada de enlaces instrumentados en el acceso oriental. Por lo tanto, las observaciones disponibles no determinan una única asignación OD.

Por consiguiente, este trabajo utiliza dos criterios para construir la demanda:

1. **Restricción dinámica empírica:** las mediciones continuas de aforo determinan el perfil horario de la demanda total y fijan el flujo observado en el corredor principal de ingreso.
2. **Estructuración espacial híbrida:** la distribución espacial de los viajes combina un conocimiento previo (*prior*) determinista, basado en la jerarquía vial, con un remanente estocástico controlado sobre los pares alcanzables.

Adquisición, auditoría y preprocesamiento de telemetría empírica

La base empírica de aforo proviene de registros continuos de sensores de conteo y clasificación vehicular, correspondientes a cámaras térmicas Tecnotrans / Tecnología infrarroja con barrido hacia adelante (*Forward Looking Infrared*) (FLIR) ThermICam. Los registros tienen una resolución de 15 minutos y corresponden al período octubre de 2025 – abril de 2026.

En las etapas preliminares se dispuso de información de cuatro dispositivos ubicados en el entorno del nudo de Costanera y Acceso Este. El relevamiento geoespacial mostró que las cámaras 192.168.1.167 (egreso hacia el este), 192.168.1.168 (circulación hacia el norte por Costanera) y 192.168.1.170 (circulación hacia el sur por Costanera) se encuentran entre 600 m y 800 m al este del perímetro de la red modelada. Por lo tanto, miden flujos de la autopista que quedan fuera del modelo vial urbano. En la generación y calibración definitiva se utiliza exclusivamente la Cámara 169 (192.168.1.169), ubicada

en el viaducto sobre el canal Cacique Guaymallén, que registra el ingreso hacia el oeste por la Avenida José Vicente Zapata.

Para controlar la calidad de los datos de la Cámara 169, se aplicó un procedimiento de auditoría y saneamiento estructurado en cinco etapas:

1. **Filtrado temporal y estructural:** descarte de marcas temporales corruptas, incluida la fecha anómala 1970-01-01, o fuera del rango válido [2020–2035].
2. **Validación de zonas y clases vehiculares:** admisión de zonas válidas ($\text{ZoneID} \in [1, 19]$) y clasificación física en vehículos livianos ($l \leq 5$ m), medianos ($5 \text{ m} < l \leq 10$ m) y pesados ($l > 10$ m), según $\text{ClassID} \in \{1, 2, 3\}$.
3. **Eliminación de duplicados:** eliminación de registros redundantes mediante la clave ($\text{DateUTC}, \text{ZoneID}, \text{ClassID}$), conservando el último registro (`keep='last'`).
4. **Alineación temporal:** reasignación de marcas con desviaciones $|\Delta t| \leq 5$ s al cuarto de hora más cercano y descarte de registros desalineados (*off-grid*).
5. **Filtro de días hábiles completos:** selección de días hábiles, de lunes a viernes, con cobertura del 100% de los intervalos diarios de 15 minutos. El 95% se utilizó como umbral operativo de auditoría antes de aplicar el criterio más estricto de inclusión.

Este protocolo seleccionó 33 días hábiles completos no perturbados, arrojando en la Cámara 169 un volumen empírico diario medio de $\mu_{\text{obs}} = 25.283,85$ veh/día con una desviación estándar interdiaria de $\sigma_{\text{obs}} = 2.316,54$ veh/día.

Maapeo geoespacial, corrección topológica y señalización

El trazado geométrico base se obtuvo mediante la exportación de datos de OSM y su conversión al formato de red de SUMO (`.net.xml`). El dominio modelado delimita el microcentro de la Ciudad de Mendoza entre Av. José Vicente Zapata / Colón al sur, Av. General San Martín como eje central norte-sur, Av. Manuel Belgrano al oeste y Av. Las Heras al norte. La red se proyecta en coordenadas Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*) (UTM) Zona 19S (Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*) (WGS84)).

Punto de inyección este (TAZ 28): dado que el nudo de Costanera se ubica al este del perímetro del modelo, la Cámara 169 se representa mediante el primer tramo interno disponible de la calzada de ingreso. Se utiliza el enlace (*edge*) 93904440#0, asignado al origen de la zona TAZ 28.

Auditoría y corrección topológica con Google Maps y Street View: tras la conversión inicial desde OSM, se detectaron discrepancias entre la red y la infraestructura real. La red se revisó con imágenes satelitales de Google Maps y vistas de calle de Google Street View:

- **Restricciones de giro y maniobras prohibidas:** se implementaron las restricciones observadas en terreno, como la prohibición de giro a la izquierda desde Av. General San Martín hacia el sur por Av. Colón. También se conservaron los giros semaforizados permitidos, como el de calle Perú hacia Av. Las Heras.
- **Sentidos de circulación y conectividad vial:** se corrigieron los sentidos de circulación de calzadas secundarias que figuraban como bidireccionales o invertidas en OSM. También se restituyó la continuidad de los cuatro carriles de ingreso de la Av. José Vicente Zapata hacia la trama interior (93904440#0 a 93904440#1).
- **Cantidad de carriles y accesos laterales:** se ajustó el número de carriles de aproximación por calzada y se habilitaron las conexiones de inserción en las calles transversales Catamarca, Buenos Aires y Leandro N. Alem.
- **Configuración de semáforos estáticos y señalización:** las configuraciones de los semáforos de tiempo fijo corresponden a los programas generados por defecto por SUMO (`-tls.rebuild`) al compilar la red. Las señalizaciones, las secuencias de fases y las duraciones de ciclo de la red modelada se mantuvieron sin cambios. Estos programas constituyen la línea base computacional del escenario; no se interpretan como una reconstrucción de los planes semafóricos municipales.

Perfil temporal y factor de expansión territorial

El flujo vehicular en cada paso temporal se modela a partir del perfil horario representativo $\bar{q}_{\text{obs},169}(h)$ ($h \in [0, 23]$), promediado sobre los 33 días hábiles completos. El análisis de distribución temporal revela que el período diurno de 12 horas (06 : 00–18 : 00) concentra el 69,29% del flujo diario total, mientras que la ventana matutina de hora pico (06 : 30–08 : 30) concentra el 12,53%.

Para proyectar el aforo del enlace observado a la escala de la red, se aplica un **factor de expansión territorial constante** al perfil horario [CyMR18, PW16]. El factor se basa en el volumen diario nominal $V_{\text{diario}} = 260.000$ veh/día, adoptado tras una exploración de sensibilidad en el rango 250.000–290.000 veh/día:

$$f_{\text{exp}} = \frac{V_{\text{diario}}}{\sum_{h=0}^{23} \bar{q}_{\text{obs},169}(h)} = \frac{260.000}{25.283,85} \approx 10,2832 \quad (3.4)$$

La ecuación (3.4) compara el volumen diario objetivo con el volumen diario observado y, por ello, produce un factor adimensional. La demanda total de la red en la hora h queda determinada por:

$$Q_{\text{red}}(h) = \bar{q}_{\text{obs},169}(h) \cdot f_{\text{exp}} \quad (3.5)$$

La ecuación (3.5) conserva el perfil horario observado y lo escala al volumen nominal de la red mediante f_{exp} . Se adopta $\text{camera_retention} = 1,0$ como hipótesis de trabajo: la

cámara mide un acceso parcial y no permite determinar qué fracción de cada viaje alcanza el dominio modelado. Por ello, este parámetro debe interpretarse como una decisión de construcción del escenario y no como una tasa de cobertura validada.

Estructuración espacial de la demanda (60/40)

A partir de las 37 zonas TAZ, se calcularon 520 pares OD físicamente alcanzables mediante Búsqueda en anchura (*Breadth-First Search*) (BFS) sobre el grafo de conectividad vial. La distribución de la demanda horaria $Q_{\text{red}}(h)$ se rige por el archivo OD canónico utilizado en la campaña (versión interna v14), formulado como una partición híbrida:

$$p_{od} = \begin{cases} p_{od}^{\text{calib}} & \text{si } (o, d) \in \mathcal{P}_{\text{exp}} \quad \text{con} \quad \sum_{(o,d) \in \mathcal{P}_{\text{exp}}} p_{od}^{\text{calib}} = 0,60 \\ 0 & \text{si } (o, d) \in \mathcal{Z}_{\text{struct}} \\ \frac{0,40}{|\mathcal{P}_{\text{libre}}|} \cdot \xi_{od} & \text{si } (o, d) \in \mathcal{P}_{\text{libre}} = \mathcal{P}_{\text{reach}} \setminus (\mathcal{P}_{\text{exp}} \cup \mathcal{Z}_{\text{struct}}) \end{cases} \quad (3.6)$$

La ecuación (3.6) asigna a cada par origen-destino una única de las tres reglas de la partición. En ella, $\mathcal{P}_{\text{exp}}$ es el conjunto de pares explícitos, $\mathcal{Z}_{\text{struct}}$ los ceros estructurales, $\mathcal{P}_{\text{libre}}$ los pares libres alcanzables y $\xi_{od} \geq 0$ es un factor de asignación estocástica. Para que el remanente represente exactamente el 40% de la demanda, estos factores se normalizan mediante $\sum_{(o,d) \in \mathcal{P}_{\text{libre}}} \xi_{od} = |\mathcal{P}_{\text{libre}}|$; si todos los pares reciben el mismo peso, entonces $\xi_{od} = 1$. En consecuencia, la suma de las probabilidades de los pares explícitos positivos y del remanente libre es uno, mientras que los ceros estructurales no reciben demanda.

Componente fijo calibrado (60%): estructurado en 476 entradas de `mendoza_od_proportions.json` (473 pares con cuotas positivas y 3 ceros directos), para incorporar hipótesis de jerarquía vial informadas por la inspección cartográfica y por el aforo disponible:

- **Jerarquía de arterias principales:** priorización de los corredores Av. Vicente Zapata (TAZ 28), Av. General San Martín (TAZ 19/29), Av. Colón / Emilio Civit (TAZ 7), Av. Las Heras (TAZ 37/38) y Av. Manuel Belgrano (TAZ 0).
- **Intervención en TAZ 12 (Av. Perú):** reducción teórica del tráfico de origen en -20% , con un desvío realizado de $-0,47\%$ respecto de la cuota objetivo de 0,03657. La intervención buscó reducir la sobrecarga sobre calle San Lorenzo (*edge* 249618899), identificada durante la construcción del escenario como cuello de botella de distribución en la hora pico.
- **Incremento de recorridos pasantes en Patricias Mendocinas:** aumento de $+9,3\%$ en la cuota de pares longitudinales respecto de la línea base histórica, para representar la función de drenaje norte-sur de esta vía comercial.

Ceros estructurales (Z_{struct}): tres pares alcanzables, $0 \rightarrow 3$, $10 \rightarrow 5$ y $21 \rightarrow 17$, reciben una probabilidad de asignación $p_{od} = 0,0$ en el archivo canónico. Así, el remanente aleatorio no asigna vehículos a esas maniobras, consideradas frágiles durante la construcción del escenario. Esta decisión reduce una fuente conocida de bloqueo en el escenario generado, pero no constituye una garantía general de ausencia de *gridlock*.

Remanente estocástico (40%) y selección de semilla: el remanente se distribuye entre los pares libres mediante `random_routes_taz.py`. Durante el diseño del escenario se evaluaron 18 semillas pseudoaleatorias; 10 completaron la simulación continua de 12 horas sin teleports reportados. Se seleccionó la semilla 104773 por su cercanía a la cuota objetivo de la TAZ 12 entre las realizaciones estables, y se fijó la semilla de simulación de SUMO en 104729 para la validación operativa de la red. La selección no buscó maximizar la completitud ni obtener una semilla estadísticamente representativa. Una vez validada, la especificación de demanda se congeló en `mendoza_12h.rou.xml`: el archivo fija los pares origen-destino, las tasas horarias y la mezcla de flota, mientras que los instantes de inserción se sortean durante la simulación conforme al proceso exponencial descrito más abajo. Las comparaciones algorítmicas usan esa misma especificación de demanda y, dentro de cada réplica pareada, la misma semilla de SUMO para todos los controladores. Las semillas reservadas producen realizaciones adicionales de llegadas y dinámica; por ello, cuantifican la variabilidad de la simulación bajo el escenario definido y no la incertidumbre de la demanda urbana real.

Dinámica microscópica y heterogeneidad de flota: las inserciones se programan como procesos de Poisson. Para cada par origen-destino, si Q_{od} es el flujo en vehículos por hora, la tasa de arribos es $\lambda_{od} = \frac{Q_{od}}{3600}$ veh/s y cada intervalo se obtiene como $H_{od} = -\ln(U)/\lambda_{od}$, con $U \sim \mathcal{U}(0,1)$. Así, el intervalo tiene unidad de segundos y valor esperado $\mathbb{E}[H_{od}] = 1/\lambda_{od} = 3600/Q_{od}$ s. En el archivo de rutas, `period="exp(rate)"` designa este muestreo exponencial; no significa calcular $e^{\lambda_{od}}$. La distribución de flota urbana heterogénea `td_urban_mix` está compuesta por 25% de vehículos lentos, 60% estándar y 15% dinámicos.

Protocolo de validación operacional y criterio GEH

La validación operacional se definió sobre dos horizontes temporales:

1. **Escenario base canónico (12 horas, 06 : 00–18 : 00):** horizonte experimental completo de 43.200 s, que reproduce la evolución continua del tránsito diurno.
2. **Ventana operacional (2 horas, 06 : 30–08 : 30):** recorte temporal de la hora pico ($t = [1800, 9000]$ s), utilizado para la exploración iterativa de controladores y las evaluaciones de alta demanda.

En ambos horizontes se registran los vehículos cargados, insertados y completados, junto con CR, inserción, completitud y activaciones del mecanismo *teleport*. Estas mediciones

permiten verificar que el escenario se mantiene operable antes de evaluar controladores; sus valores se presentan en el capítulo 4.

Validación de calidad del flujo (estadístico GEH): conforme a las recomendaciones para la calibración de demanda en simulación microscópica [DLR26, PW16], la consistencia de la inyección en el corredor José Vicente Zapata (TAZ 28) respecto de los aforos de la Cámara 169 se evalúa mediante el estadístico GEH:

$$\text{GEH} = \sqrt{\frac{2(V_{\text{mod}} - V_{\text{obs}})^2}{V_{\text{mod}} + V_{\text{obs}}}} \quad (3.7)$$

donde V_{mod} es el volumen horario modelado y V_{obs} el conteo horario empírico observado, ambos expresados en vehículos por hora. La ecuación (3.7) resume la discrepancia entre ambos volúmenes en una magnitud adimensional. En este trabajo, el estadístico se interpreta como una comprobación de consistencia entre el flujo solicitado en el acceso representado por la cámara y el aforo observado, no como evidencia de que la matriz OD completa sea verdadera: los flujos del corredor se construyeron a partir de ese mismo aforo, por lo que la comparación verifica la implementación del perfil horario, no la demanda total. Al informar este cálculo se declaran el archivo de rutas, la ventana temporal y la hipótesis de retención; las versiones anteriores del escenario se documentan como antecedentes de construcción y no se combinan con la evaluación del escenario canónico.

3.4 Arquitectura de integración entorno-agentes

El marco computacional `marlTLC` integra el simulador SUMO con la librería Ray `RLlib` a través de la interfaz personalizada `SumoRllibEnv`. La especificación detallada de las clases y patrones de diseño orientados a objetos que sustentan esta arquitectura se expone en el Apéndice A.

3.4.1 Ciclo episódico e interfaz de simulación

Para mejorar la recolección de muestras durante el entrenamiento distribuido, las simulaciones se ejecutan sin interfaz gráfica. En los trabajos por lote se activa la variable de entorno `LIBSUMO_AS_TRACI=1`, con lo que Python utiliza el *binding* nativo `libsumo`; cuando la variable no está presente, el mismo código puede utilizar una conexión TraCI estándar. La primera opción evita la comunicación por sockets entre procesos del simulador y el entorno, mientras que ambas exponen la misma interfaz de control.

La conexión se configura en `sumo_env.py` con un único objeto activo por entorno. Las opciones comunes fijan `-waiting-time-memory`, `-end` y la semilla de SUMO; cuando se solicita telemetría, se añade `-tripinfo-output` junto con `--tripinfo-output.write-unfinished`. En las ejecuciones por lote se desactivan `-no-step-log` y `-no-warnings` para reducir E/S innecesaria. La conexión TraCI convencional se inicia con una

etiqueta propia y hasta diez reintentos, mientras que `libsumo` reutiliza la misma interfaz sin abrir un puerto. Al reiniciar un episodio, el entorno elimina el archivo de telemetría anterior, carga el estado de calentamiento con `simulation.loadState` y limpia los contadores de vehículos, teletransportes, espera y equidad; así se evita repetir el calentamiento y se impide que una ejecución contamine a la siguiente. El mapa de vecinos se construye una sola vez a partir de los carriles controlados y sus enlaces, con un recorrido de complejidad $O(TLC)$ en lugar de comparar todos los pares de semáforos, donde T es el número de agentes, L el número de carriles controlados y C el número de conexiones inspeccionadas.

El ciclo de vida episódico opera bajo el siguiente protocolo de inicialización y control:

1. **Período de calentamiento (*warmup*):** los escenarios sintéticos utilizan 600 s bajo control semafórico de tiempo fijo. En la configuración MARL de Mendoza para dos horas, el archivo `.sumocfg` comienza en $t = 1500$ s (06:25), el calentamiento dura 300 s y la ventana controlada queda así en $[1800, 9000]$ s (06:30–08:30), coincidente con el recorte de la hora pico. Esta fase puebla la red y evita que los agentes aprendan únicamente sobre una red vacía.
2. **Almacenamiento y restauración de puntos de control (*checkpoint*):** al finalizar el calentamiento, el estado microscópico de la red se almacena en un archivo XML (`state_pid...xml`). Cada invocación de `reset()` restaura este estado, de modo que el entrenamiento comienza en condiciones equivalentes sin repetir la simulación preliminar. El tiempo controlado se interpreta como un intervalo absoluto cuando el archivo `.sumocfg` tiene un valor de inicio distinto de cero.
3. **Paso de decisión y transiciones semafóricas:** las decisiones se ejecutan en pasos de $\Delta t = 5.0$ s. Cuando un agente cambia la fase activa, el entorno intercala una fase amarilla de `yellow_time = 4` s y exige un tiempo mínimo de verde de `min_green_time = 5` s antes de habilitar la nueva transición.

3.5 Instanciación del modelo Dec-POMDP

La interacción distribuida entre las intersecciones semaforizadas se formaliza como un juego parcialmente observable con observaciones locales. La notación de Dec-POMDP del marco teórico se utiliza para describir la información disponible y la acción conjunta. En la implementación, cada controlador recibe una recompensa individual compuesta; la coordinación se induce mediante el componente vecinal de esa señal y el crítico centralizado durante el entrenamiento de MAPPO y HAPPO, no mediante la suma de todas las recompensas en un escalar global. La formulación difiere así del Dec-POMDP cooperativo canónico en la señal de retorno: el objetivo que optimiza cada actor es su retorno individual local-vecinal. Las comparaciones entre familias algorítmicas miden, en consecuencia, el desempeño de procedimientos completos de aprendizaje y coordinación, y no aíslan causalmente cada componente.

3.5.1 Observaciones y codificación logarítmico-lineal

La observación local $o_i^{(t)}$ recibida por el agente i en el paso t combina el índice de la fase activa y una representación por carril de la espera acumulada:

$$o_i^{(t)} = (p_i^{(t)}, e(w_{i,1}^{(t)}), \dots, e(w_{i,m_i}^{(t)})), \quad (3.8)$$

donde $p_i^{(t)}$ es el índice de la fase, m_i es el número de carriles de entrada controlados por la intersección y $w_{i,l}^{(t)}$ es la espera acumulada atribuida al carril l en el paso t : la suma de los tiempos de espera acumulados de los vehículos presentes en ese carril, expresada en vehículo-segundo. La atribución recorre los vehículos y asigna a cada carril la contribución no registrada en los carriles anteriores, de modo que un vehículo que cambia de carril no duplica automáticamente su espera. En la configuración de las campañas de ajuste, la observación vecinal no se concatena al actor mediante `neighbor_info=False`; la información de los vecinos se utiliza en la señal de recompensa y en la observación conjunta del crítico centralizado.

Para representar esta información se aplica la clase `Encoder` de `src/marITLC/utlis/encoder.py`. La transformación cuantiza cada espera en uno de I niveles y conserva más resolución en el rango bajo. Así, valores de espera próximos que caen en el mismo intervalo quedan representados por una misma categoría, mientras que la saturación agrupa los valores iguales o superiores a M_{lane} . Esta reducción de la variedad de valores codificados puede ofrecer una representación más manejable para el aprendizaje; el nivel de detalle adicional en esperas bajas busca retener distinciones relevantes en ese rango. Son motivaciones de diseño, no efectos de aprendizaje o rendimiento demostrados por separado: el redondeo puede perder distinciones relevantes y generar cambios de categoría al cruzar un umbral. La evaluación estudia el codificador integrado al sistema y no incluye una ablación frente a otras representaciones. Analíticamente, la función se construye como una combinación convexa en el dominio normalizado adimensional. Sea $x \geq 0$ la espera acumulada del carril y $M_{\text{lane}} > 0$ su cota de saturación, ambas expresadas en la misma unidad física (vehículo-segundo). Se define la variable adimensional normalizada $\hat{x} = \min(1, x/M_{\text{lane}}) \in [0, 1]$. En dicho dominio unitario, se combinan un mapeo logarítmico cóncavo y un mapeo lineal mediante una mezcla convexa $g(\hat{x}) \in [0, 1]$:

$$g(\hat{x}) = (1 - L) \frac{\log_2 \left(\frac{\hat{x}}{L} + 1 \right)}{\log_2 \left(\frac{1}{L} + 1 \right)} + L\hat{x}, \quad (3.9)$$

donde $I \geq 2$ es el número de intervalos discretos y $L = (I-1)/M_{\text{lane}}$ actúa como parámetro numérico de curvatura y ponderación lineal. Con los rangos de I y M_{lane} empleados en las configuraciones, $0 < L < 1$, por lo que los pesos de la combinación son positivos. Dado que $g(0) = 0$ y $g(1) = 1$, la curva mapea continuamente $[0, 1]$ sobre sí mismo. Para

cuantizar la señal en el conjunto discreto $\{0, \dots, I - 1\}$, la salida se escala por el índice máximo $(I - 1)$ y se aplica la función techo (ceil): $e(x) = \min\{I - 1, \text{ceil}((I - 1)g(\hat{x}))\}$.

Al expandir algebraicamente $(I - 1)g(\hat{x})$ mediante las relaciones $(I - 1) = L \cdot M_{\text{lane}}$, $\hat{x}/L = x/(M_{\text{lane}}L)$ y $(I - 1)L\hat{x} = L^2x$, se obtiene de forma exacta la expresión evaluada computacionalmente en el código:

$$e(x) = \text{ceil} \left(F \cdot \log_2 \left(\frac{x}{M_{\text{lane}} \cdot L} + 1 \right) + L^2 \cdot x \right), \quad (3.10)$$

con $F = (1 - L)(I - 1)/\log_2(1/L + 1)$. Para $0 \leq x < M_{\text{lane}}$, la ecuación (3.10) es la factorización compacta de la combinación convexa continua escalada por $(I - 1)$; para $x \geq M_{\text{lane}}$, la implementación satura directamente la salida en $I - 1$. La función ceil redondea hacia arriba al entero más cercano. La letra M_{lane} se utiliza aquí exclusivamente para el codificador y no debe confundirse con el factor de ventaja M de HAPPO.

En esta formulación:

- **Número de intervalos (I):** corresponde al hiperparámetro `encoder_intervals`, explorado en el rango entero $[4, 64]$. La clase almacena internamente $I - 1$ como el índice máximo, por lo que la salida pertenece a $\{0, \dots, I - 1\}$.
- **Cota de saturación del carril (M_{lane}):** corresponde a `max_lane_value`, determinado empíricamente calculando el percentil 99 (P_{99}) de las esperas acumuladas por carril en simulaciones preliminares de cada red: $M_{\text{lane}} = 1184$ vehículo·segundo para 3×1 , $M_{\text{lane}} = 1136$ vehículo·segundo para 3×3 , $M_{\text{lane}} = 712$ vehículo·segundo para $4 \times 4_H$ y $M_{\text{lane}} = 1494$ vehículo·segundo para el microcentro de la Ciudad de Mendoza.

La variable `lane_metric` determina qué cantidad se extrae antes de codificar: la configuración utilizada en el ajuste es `acc_waiting_time`, definida arriba, mientras que el entorno también admite la espera del último paso y la cantidad de vehículos detenidos.

La observación ubica primero el índice de fase y luego los valores codificados de los carriles, en el orden definido por el entorno. La figura 3.2 muestra la discretización escalonada de una espera, mientras que la figura 3.3 ilustra la mezcla logarítmica-lineal. Estas figuras son auxiliares: la definición reproducible de la observación es la ecuación (3.8), y la función de codificación se implementa en `src/mar1TLC/utils/encoder.py`.

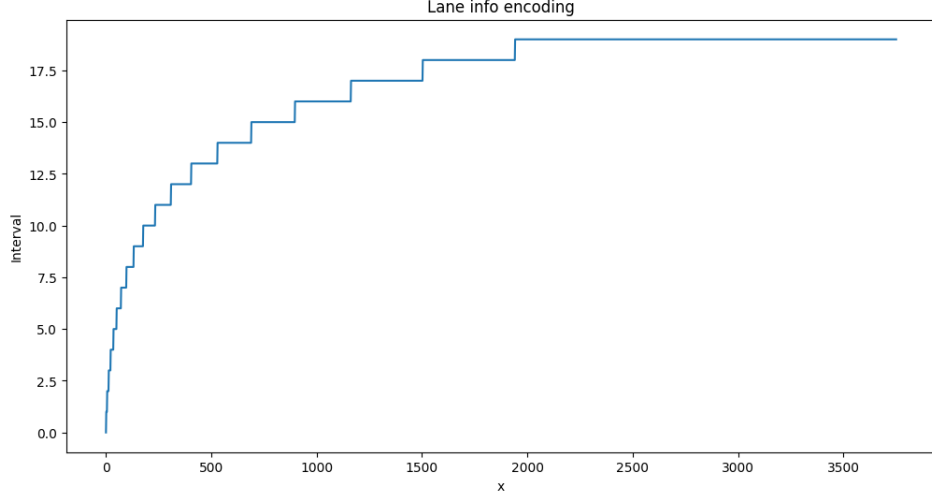


Figura 3.2: Codificación por carril empleada para construir la observación local. Cada entrada de espera se transforma y conserva su correspondencia espacial con el carril de origen.

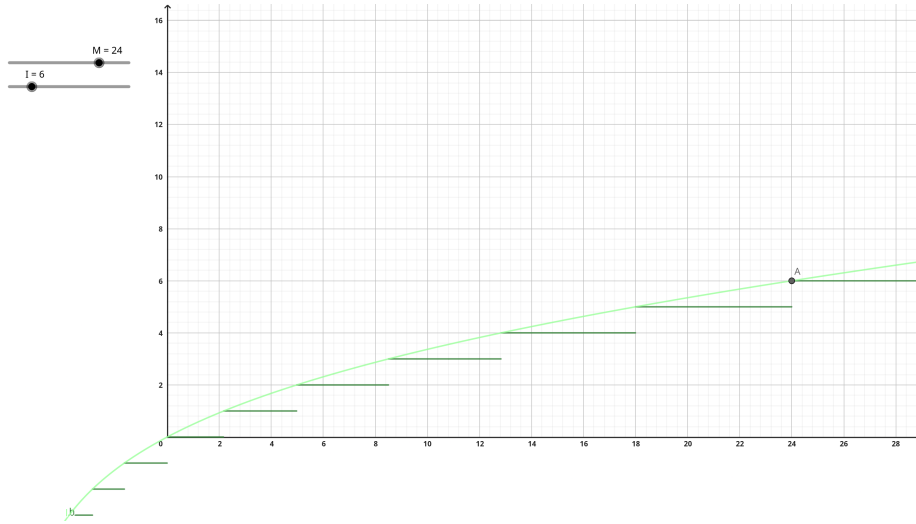


Figura 3.3: Ejemplo de la mezcla logarítmica-lineal utilizada por `Encoder`. La curva se interpreta como una transformación de la espera acumulada, no como una trayectoria de entrenamiento.

3.5.2 Espacio de acciones y enmascaramiento dinámico

El espacio de acciones $\mathcal{A}_i = \{0, 1, \dots, K_i - 1\}$ representa el conjunto de fases verdes seleccionables por la intersección i . Los valores m_i y K_i difieren entre intersecciones según la geometría del cruce y las restricciones de maniobra, y cada agente dispone de un módulo de política propio sin compartición de parámetros; en ese sentido el conjunto de agentes es heterogéneo y la evaluación de HAPPO se realiza sobre los espacios de observación y acción efectivos de cada escenario. En cruces con geometrías asimétricas o restricciones de maniobra, el entorno construye el conjunto de acciones admisibles y la capa personalizada `ActionMaskingTorchRLModule` aplica un enmascaramiento dinámico sobre los *Logits* $z_i(a)$

antes del cómputo de la distribución *Softmax*. Los *Logits* son puntuaciones reales sin normalizar producidas por la red; la función *Softmax* las transforma en probabilidades cuya suma es uno:

$$\tilde{z}_i(a, t) = z_i(a) + \log m_i(a, t), \quad m_i(a, t) = \begin{cases} 1 & \text{si } a \in \mathcal{A}_i^{\text{adm}}(t) \\ 0 & \text{si } a \notin \mathcal{A}_i^{\text{adm}}(t) \end{cases} \quad (3.11)$$

donde $\mathcal{A}_i^{\text{adm}}(t) \subseteq \mathcal{A}_i$ es el conjunto de acciones admisibles en el instante t , $m_i(a, t)$ es la máscara binaria correspondiente y $z_i(a)$ es la puntuación producida por la política. El entorno define $\mathcal{A}_i^{\text{adm}}(t) = \{a_i^{\text{actual}}\}$ durante una transición amarilla o mientras el tiempo de la fase corriente sea menor que $\max(t_{\min}, t_{\text{yellow}})$. Una vez satisfecha esa condición, quedan habilitadas las fases compatibles con la configuración del cruce. El término $\log m_i(a, t)$ vale 0 para acciones admisibles y se limita a un entero negativo de magnitud máxima en aritmética de precisión finita para las inadmisibles, de modo que su contribución es despreciable tras la normalización. En IPPO, MAPPO y HAPPO la suma se aplica a los *logits*; en IDQN se aplica a los valores Q y también a la acción siguiente utilizada para el objetivo temporal, tanto en la red principal como en la red objetivo. La máscara evita acciones incompatibles, pero no reemplaza la lógica de transición amarilla del entorno, que se ejecuta después de aceptar un cambio de fase.

3.5.3 Función de recompensa compuesta y coordinación vecinal

La función de recompensa representa el compromiso entre el desempeño operacional local y la cooperación distribuida. El código permite evaluar varias señales de tránsito; la selección de una señal no se interpreta como una superioridad universal, sino como una decisión experimental que se mantiene fija al comparar algoritmos dentro de un escenario. La tabla 3.3 resume las alternativas implementadas.

Tabla 3.3: Señales de recompensa disponibles en el entorno.

Identificador	Señal utilizada	Unidad o interpretación
<code>diff_halted</code>	Variación del número de vehículos detenidos entre pasos.	vehículos
<code>diff_waitingTime</code>	Variación de la espera del último paso.	s
<code>diff_cumulativeWaitingTime</code>	Variación de la espera acumulada en los carriles de entrada.	vehículo-s
<code>negative_waitingTime</code> <code>negative_cumulativeWaitingTime</code>	Penalización directa de la espera instantánea o acumulada.	s o vehículo-s
<code>diff_cumulativeWaitingTime_normalized</code>	Variación acumulada dividida por la cantidad de vehículos considerada.	s
<code>pressure</code>	Negativo de la diferencia absoluta entre vehículos detenidos en entradas y salidas.	vehículos
<code>negative_queue_length</code>	Negativo de la cantidad de vehículos detenidos en los carriles de entrada.	vehículos

La tabla documenta las señales implementadas en el entorno; la disponibilidad de una clase no implica que haya sido comparada experimentalmente. En las campañas de ajuste de este capítulo se fija `reward_key = diff_cumulative_waiting_time_with_neighbors`. Esta clave combina la señal diferencial de espera acumulada con una media sobre los vecinos mediante `NeighborhoodRelevanceReward`. En las líneas de base independientes (IPPO e IDQN), se fija de forma estricta $\rho_{\text{coop}} = 0$, reduciendo la señal a la recompensa puramente local $R_i^{(t)} = L_i^{(t)}$. Para los métodos cooperativos (MAPPO y HAPPO), el parámetro $\rho_{\text{coop}} \in [0.0, 0.95]$ se optimiza mediante búsqueda bayesiana para determinar el grado óptimo de ponderación vecinal en cada escenario y arquitectura; una vez seleccionada la receta, su valor permanece fijo durante la confirmación y la evaluación reservada. No se agregan términos heurísticos adicionales. Las demás filas de la tabla se presentan como alternativas implementadas, no como funciones evaluadas, porque no existe para ellas una ejecución identificable por escenario, semilla, configuración y métrica.

1. **Recompensa local diferencial (L_i):** cuantifica la variación del tiempo de espera acumulado entre pasos consecutivos. En esta sección, $W_{\text{acc},i}(t)$ representa la suma de los tiempos de espera de los vehículos presentes en los carriles de entrada controlados por la intersección i en el paso t , expresada en vehículo-segundo. La implementación activa selecciona esta señal mediante la opción de configuración `reward_key = diff_cumulative_waiting_time_with_neighbors`; su componente

local utiliza esta diferencia sin dividirla por el número de vehículos. La ecuación (3.12) asigna una señal positiva cuando la espera disminuye:

$$L_i^{(t)} = W_{\text{acc},i}(t-1) - W_{\text{acc},i}(t) \quad (3.12)$$

2. **Señal vecinal (N_i):** corresponde al promedio de las señales locales obtenidas por las intersecciones del vecindario topológico directo $\mathcal{N}(i)$. Esta construcción incorpora al retorno del agente la evolución de los cruces inmediatamente conectados, pero no equivale a observar sus estados ni a optimizar una recompensa global. La ecuación (3.13) supone $|\mathcal{N}(i)| > 0$:

$$N_i^{(t)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} L_j^{(t)} \quad (3.13)$$

3. **Mezcla convexa:** se adopta una combinación convexa gobernada por el hiperparámetro $\rho_{\text{coop}} \in [0, 0.95]$ para ponderar el componente local y el vecinal. La ecuación (3.14) conserva las unidades de L_i y N_i :

$$R_i^{(t)} = (1 - \rho_{\text{coop}})L_i^{(t)} + \rho_{\text{coop}}N_i^{(t)} \quad \text{con} \quad \rho_{\text{coop}} \in [0, 0.95] \quad (3.14)$$

Si el vecindario es vacío, $|\mathcal{N}(i)| = 0$, la implementación omite el promedio de la ecuación (3.13) y define $R_i^{(t)} = L_i^{(t)}$. En IPPO e IDQN, $\rho_{\text{coop}} = 0$ opera de forma fija como ancla puramente local descentralizada, garantizando que los agentes independientes no reciban retroalimentación de desempeño ajena. En MAPPO y HAPPO, $\rho_{\text{coop}} > 0$ modula la cooperación vecinal. En cada paso, el entorno forma el vector de recompensas $\mathbf{r}_t = (R_1^{(t)}, \dots, R_N^{(t)})$ y entrega cada componente individual al módulo correspondiente; no se agregan sus componentes en una recompensa escalar global.

3.6 Implementación algorítmica e hiperparámetros

Los cuatro algoritmos evaluados se integran en la API de Ray RLlib mediante el módulo de fábrica `algo_factory.py`. MAPPO y HAPPO utilizan *learners* propios sobre PyTorch, mientras que IPPO e IDQN reutilizan las implementaciones de PPO y DQN de RLlib junto con módulos propios para el enmascaramiento de acciones.

3.6.1 Mecanismos de aprendizaje independiente y centralizado

- **IDQN e IPPO (Aprendizaje Independiente):** Cada intersección aprende una política local $\pi_{\theta_i}(a_i|o_i)$ tratando a los demás controladores como parte del entorno no estacionario. En IPPO, cada agente cuenta con su propia red de valor local $V_{\phi_i}(o_i)$.
- **MAPPO (PPO multiagente con crítico centralizado [YVV⁺22]):** Emplea el

paradigma CTDE. Durante la ejecución, cada actor selecciona acciones de forma descentralizada a partir de su observación local o_i . Durante el entrenamiento, el módulo `SharedCriticTorchRLModule` actúa como un crítico centralizado compartido multi-salida: recibe el vector de observaciones locales concatenadas $s = (o_1, o_2, \dots, o_N)$, que aporta contexto conjunto al crítico sin equivaler al estado Markoviano completo de la simulación, y produce $\mathbf{V}_\phi(s) = (V_{\phi,1}(s), \dots, V_{\phi,N}(s))$. Las recompensas, valores, objetivos y ventajas de GAE (λ_{GAE}) se calculan por agente.

- **HAPPO (PPO para Agentes Heterogéneos [KCW⁺22]):** Su marco teórico canónico se apoya en el **lema de descomposición de ventajas multiagente**, cuya formulación utiliza una ventaja conjunta. La implementación de este trabajo constituye una adaptación: no aplica directamente ese lema, sino que inicializa los factores de actualización con ventajas individuales estandarizadas obtenidas mediante GAE y los repondera secuencialmente. El crítico centralizado proporciona estimaciones de valor durante el entrenamiento, mientras que la reponderación secuencial descrita corresponde a la actualización de los actores; no se atribuye a esta secuencia una actualización secuencial del crítico. En cada *epoch* de entrenamiento, los agentes se actualizan secuencialmente, sobre el lote completo y según un orden $(i_1, i_2, \dots, i_N)$ re-sortado, donde N es el número total de agentes y i_m es el agente ubicado en la posición m de ese orden; cada actor recibe un único paso de gradiente por *epoch*. Si $\overline{|A|}_i$ denota la media del valor absoluto de la ventaja individual ya estandarizada del agente i en la muestra —el conector de entrenamiento normaliza las ventajas por módulo antes de la actualización—, el hiperparámetro p_{greedy} determina cómo se construye el orden: 0.0 produce un orden aleatorio; 1.0, un orden *greedy* según $\overline{|A|}_i$; y un valor en $(0, 1)$, un orden *semi-greedy* que combina ambos criterios. Sea $\pi_b^{(i_m)}$ la política que recolectó la muestra y $\pi_+^{(i_m)}$ el actor de i_m después de su actualización dentro del *epoch* corriente. Para una observación o y una acción a de la muestra, la razón que se propaga a los agentes restantes es:

$$\rho_{\text{post}}^{(i_m)} = \frac{\pi_+^{(i_m)}(a|o)}{\pi_b^{(i_m)}(a|o)} \quad (3.15)$$

La ecuación (3.15) compara la política posterior a la actualización con la política de comportamiento que generó la muestra, el mismo denominador que utiliza el objetivo tipo PPO de la actualización. Para cada agente aún no actualizado i_j , con $j > m$, $M^{(i_j)}$ es el factor de ventaja reponderada del *epoch* corriente, inicializado con su ventaja individual estandarizada al comienzo de ese *epoch*. El factor se actualiza como

$$M^{(i_j)} \leftarrow M^{(i_j)} \cdot \rho_{\text{post}}^{(i_m)}, \quad j > m. \quad (3.16)$$

La ecuación (3.16) incorpora al factor la razón de cada actor ya optimizado: la

actualización de i_1 afecta a todos los agentes restantes del *epoch*, la de i_2 a los que siguen en el orden, y así sucesivamente. Puesto que M se reinicializa en cada *epoch*, la propagación cubre únicamente a los agentes previos en el orden de ese *epoch*, y no la secuencia completa de actualizaciones acumuladas desde la recolección.

El procedimiento canónico de HAPPO agrupa, en cambio, los pases internos de cada actor bajo un único factor secuencial; la diferencia se mantiene explícita porque afecta a la interpretación de las garantías: la mejora monótona corresponde al procedimiento teórico de región de confianza bajo sus supuestos explícitos y no se transfiere a esta variante con recorte. En la implementación evaluada, el coeficiente de la penalización de KL permanece fijo durante la actualización de HAPPO.

La reponderación secuencial de ventajas individuales ofrece una forma de considerar políticas propias de intersecciones con acciones y condiciones de operación distintas, sin garantizar por sí misma un mejor control que MAPPO.

En los cuatro algoritmos, el entrenamiento consume observaciones del tránsito, fases ejecutadas, recompensas y observaciones siguientes para ajustar los parámetros de las redes. IDQN conserva transiciones anteriores en un *replay buffer* de episodios priorizado y multiagente; las variantes de PPO recolectan un *rollout* con la política vigente y lo reutilizan durante una cantidad acotada de *epochs*. Cada *epoch* recorre el conjunto recolectado; en IPPO y MAPPO el lote se subdivide en *minibatches* y cada uno produce una actualización, mientras que en HAPPO el lote no se subdivide y cada actor realiza un único paso de gradiente completo por *epoch*. El optimizador, los búferes y los críticos pertenecen al proceso de aprendizaje: durante la ejecución del control, cada intersección solo necesita su observación local y la red entrenada que asigna valores o probabilidades a las fases admisibles. Por tanto, la salida del entrenamiento es una política parametrizada, no un plan semafórico fijo.

3.7 Búsqueda de hiperparámetros (HPO)

La búsqueda de hiperparámetros se ejecutó con Optuna [ASY⁺19], Ray Tune y el algoritmo de poda asíncrona ASHA [LJR⁺20] para las combinaciones sintéticas de algoritmo, escenario y arquitectura. La Ciudad de Mendoza se definió como un estudio separado, cuya búsqueda se inicia después del cierre de los estudios y la confirmación sintéticos. La receta sintética no se transfiere automáticamente: sus resultados pueden utilizarse, a lo sumo, para acotar el espacio de exploración de Mendoza.

Antes del muestreo se fijaron los parámetros que definen la semántica de la interacción. En particular, $\gamma = 0,99$ se mantuvo constante en MAPPO, HAPPO, IPPO e IDQN como decisión de diseño informada por las configuraciones de referencia del proyecto, y no como un óptimo universal. También se fijaron la métrica de carril, la señal de recompensa y el calibrado M_{lane} por escenario. Las arquitecturas **h128** y **h256** no se mezclan dentro de un

mismo estudio: cada una recibe un estudio independiente y se comparan después mediante métricas físicas. La métrica de carril, la recompensa, M_{lane} y el modo de interacción se mantienen fijos por escenario. En cambio, el número de intervalos del codificador es un hiperparámetro: modifica la resolución con que se representa cada carril y, por ello, se explora explícitamente junto con los parámetros de aprendizaje.

1. **Etapa 1 (exploración bayesiana amplia):** muestreo mediante TPE. El presupuesto nominal es de 100 pruebas por combinación; el valor de 150 corresponde al límite máximo previsto para una eventual ampliación y no a una segunda etapa de búsqueda. ASHA aplica poda temprana con período de gracia de 10 iteraciones y factor de reducción 4; las pruebas podadas no integran el conjunto de candidatos finales. Se evalúan las arquitecturas compacta (h128: [128, 128]) y ancha (h256: [256, 256]) como estudios independientes de Ray Tune. El espacio registrado comprende 13 dimensiones para MAPPO y 13 parámetros nominales para IPPO, de los cuales 12 son efectivos porque $\rho_{\text{coop}} = 0$ se fija. Para HAPPO se registran 14 parámetros, aunque dos no modifican la actualización y el espacio efectivo comprende 12 dimensiones:

- $\rho_{\text{coop}} \in [0.0, 0.95]$ (solo MAPPO y HAPPO, **uniform**)
- $\alpha_{\text{lr}} \in [3 \times 10^{-5}, 10^{-3}]$ (**loguniform**)
- $\epsilon_{\text{clip}} \in [0.10, 0.22]$ (**uniform**)
- $\lambda_{\text{GAE}} \in [0.88, 0.99]$ (**uniform**)
- $c_{\text{entropy}} \in [0.0, 0.01]$ (**uniform**)
- $\text{grad_clip} \in [0.25, 5.0]$ (**loguniform**)
- $c_{\text{KL}} \in [0.05, 0.5]$ (**loguniform**)
- $d_{\text{KL_target}} \in [0.005, 0.03]$ (**loguniform**)
- $\text{vf_clip} \in [5.0, 20.0]$ (**loguniform**)
- $K \in [5, 15]$ para MAPPO e IPPO, y $K \in [5, 18]$ para HAPPO (**randint**)
- $N_{\text{episodes}} \in [2, 12]$ (**randint**)
- $I \in [4, 64]$ (**randint**)
- $B_{\text{mini}} \in \{128, 256, 512\}$ (**choice**)
- $p_{\text{greedy}} \in [0.20, 0.90]$ (solo HAPPO, **uniform**)

Parámetros fijos: `lane_metric = acc_waiting_time`, `use_kl_loss = True`, $\gamma = 0.99$ y, para IPPO, $\rho_{\text{coop}} = 0$ fijo. Para IDQN, el espacio específico de diez dimensiones reemplaza los parámetros de PPO por tasa de aprendizaje $[10^{-5}, 10^{-3}]$, lotes $\{64, 128, 256, 512\}$, `grad_clip` $\in \{0, 5; 1; 2; 5; 10\}$, actualización de la red objetivo $\{500, 1000, 2000, 5000\}$ pasos, decaimiento de ϵ en $\{5000, 10000, 20000\}$ pasos, $\epsilon_{\text{final}} \in [0, 0.2, 0.5, 1.0]$, retorno de n pasos $\{1, 3, 5\}$ y comienzo del aprendizaje

$\{1000, 2000, 5000\}$ pasos, también con recompensa estrictamente local ($R_i = L_i$). En todos los algoritmos se explora $I \in [4, 64]$ mediante `tune.randint(4, 65)`, cuyo extremo superior no se incluye, y se fija el valor de saturación M_{lane} calibrado para cada escenario. En HAPPO, las dimensiones asociadas al *minibatch* y al objetivo de KL se muestrean y registran, pero no modifican su actualización, que opera con lote completo por actor y coeficiente de KL fijo; la búsqueda efectiva sobre HAPPO comprende doce dimensiones. En los escenarios sintéticos, las semillas de entrenamiento de HPO son $\{101, 137, 179\}$ para 3×1 , 3×3 y $4 \times 4_H$, respectivamente, y las de evaluación son $\{100101, 100137, 100179\}$. La búsqueda en cada escenario sintético dispuso de una única semilla de entrenamiento, de modo que el ordenamiento de candidatos queda expuesto a esa realización pseudoaleatoria; la confirmación sobre diez semillas se definió para controlar ese riesgo de selección. Para Mendoza, el presupuesto y los conjuntos de semillas deben fijarse antes de iniciar su campaña y mantenerse separados de los utilizados en la evaluación reservada.

2. **Etapla 2 (confirmación pareada):** se previó un reentrenamiento a presupuesto completo sobre las semillas de confirmación definidas por escenario. En MAPPO y HAPPO, el diseño contrasta el ρ_{coop} seleccionado con la línea base puramente local ($\rho_{\text{coop}} = 0$). IPPO e IDQN se mantienen como líneas de base de aprendizaje independiente con recompensa puramente local, con sus propias recetas seleccionadas y las mismas reglas de evaluación reservada. Para Mendoza se definió una secuencia propia de ajuste, congelamiento y confirmación, posterior al cierre sintético. Dado que el tamaño del lote de aprendizaje forma parte del espacio de ajuste, las cuarenta iteraciones fijan el número de actualizaciones y no la experiencia total consumida; por ello, cada ejecución registra los pasos de entorno muestreados de forma acumulada (`num_env_steps_sampled`) para informar, junto con el desempeño, la experiencia requerida para alcanzarlo. Las comparaciones entre familias se formulan sobre las políticas finales evaluadas bajo el mismo horizonte, la misma demanda y las mismas semillas, y no como afirmaciones de eficiencia muestral entre algoritmos.

La selección de candidatos minimiza la métrica física `metric_objective`, que en la campaña corresponde a la espera media estable por vehículo insertado. Sea w_r la espera media de los vehículos insertados en la iteración de entrenamiento r . Para diagnosticar inestabilidad se calcula el incremento positivo

$$X_r = \max\{0, w_r - w_{r-1}\}. \quad (3.17)$$

La colección de realizaciones $\{X_r\}$ corresponde a `positive_interiteration_increases`.

Para la fracción superior $\alpha_{\text{CVaR}} = 0,2$ se utiliza el estimador empírico de cola:

$$\widehat{\text{CVaR}}_{\alpha_{\text{CVaR}}}(X) = \frac{1}{k} \sum_{j=1}^k X_{(j)}, \quad k = \max\{1, \lceil m\alpha_{\text{CVaR}} \rceil\}, \quad (3.18)$$

La ecuación (3.18) reproduce el cálculo implementado: m es la cantidad de incrementos finitos disponibles, $X_{(j)}$ son esos incrementos ordenados de mayor a menor y k garantiza que siempre se utilice al menos una observación. Por tanto, la penalización es un promedio empírico del 20% superior, no una garantía de ausencia de *gridlock*. La espera w_r se obtiene de **TripInfo** y se expresa en segundos por vehículo insertado.

$$\text{metric_objective}_r = w_r + 1,0 \cdot \widehat{\text{CVaR}}_{0,2}(X) \quad (3.19)$$

La ecuación (3.19) se informa sobre las últimas 10 iteraciones, después de excluir las primeras 5 del historial de estabilidad. La primera parte prioriza una espera media baja y el segundo término penaliza aumentos recientes; el coeficiente 1,0 es adimensional. Los candidatos se ordenan de menor a mayor por **metric_objective**; cuando existe una métrica de evaluación válida, la espera media de evaluación se utiliza como segunda clave y, si no existe, se utiliza la espera media de entrenamiento. La ejecución debe alcanzar las 40 iteraciones y aportar métricas derivadas de **TripInfo** aptas para el ordenamiento. La completitud, la inserción, los teletransportes y la pérdida temporal se conservan para la confirmación.

3.7.1 Selección y congelamiento de las configuraciones finales

La selección se realiza por separado para cada combinación de algoritmo, escenario y arquitectura. Primero se consideran elegibles las pruebas que alcanzan las 40 iteraciones y aportan las métricas requeridas, derivadas de **TripInfo**: el selector exige que la métrica primaria y la secundaria estén presentes, sean convertibles a número y no sean NaN. La finitud de las magnitudes físicas se garantiza aguas arriba, en la extracción de métricas: el lector de **TripInfo** aborta ante atributos infinitos o negativos, valores no numéricos e identificadores de vehículo duplicados, de modo que una ejecución con telemetría inválida no produce métrica elegible.

El selector ordena a continuación por **metric_objective** en sentido ascendente. Cuando está disponible, **tripinfo/eval_mean_waiting_time_inserted** constituye la segunda clave; en su ausencia se utiliza **tripinfo/mean_waiting_time_inserted**. Esta regla mantiene un ordenamiento reproducible, aunque la segunda clave puede provenir de evaluación o de entrenamiento; esa circunstancia se registra en el manifiesto de cada receta.

Las arquitecturas **h128** y **h256** se exploran en estudios independientes y se comparan después mediante las mismas métricas físicas. La receta final conserva el identificador del estudio y del ensayo seleccionado, junto con la referencia al checkpoint generado al cierre

del reentrenamiento, lo que asegura la trazabilidad de la configuración y de la política evaluada.

La receta congelada conserva todos los parámetros que determinan la interacción —`reward_key`, `lane_metric`, M_{lane} y γ — junto con los hiperparámetros de optimización seleccionados. No se modifican al evaluar las semillas reservadas del escenario correspondiente. Para Mendoza se definió una selección de recetas propia para MAPPO, HAPPO, IPPO e IDQN, sin transferencia automática desde las redes sintéticas; la limitación del aforo se trata como una restricción de validez externa y no como una razón para excluir el ajuste del controlador. La política de cada evaluación reservada proviene del reentrenamiento de la receta congelada y sus métricas no alimentan ninguna nueva decisión de ajuste.

3.7.2 Infraestructura de cómputo y orquestación

Dado el tamaño del espacio de búsqueda y el costo computacional de la simulación microscópica en SUMO, la exploración y el entrenamiento final se desplegaron en los clústeres de cómputo **Toko** (Centro de Cómputo de Alto Rendimiento (CCAR) - Facultad de Ciencias Exactas, Físicas y Naturales (FCEFN) / UNCuyo / Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET)) y **Mendieta** (Centro de Cómputo de Alto Desempeño (CCAD) - Universidad Nacional de Córdoba).

La ejecución distribuida se gestionó mediante el sistema de colas SLURM. Se aplicaron las siguientes medidas:

- **Aislamiento de procesos Ray en un nodo:** configuración de instancias Ray de nodo único (`setup_ray_single_node`) con asignación dinámica de puertos basada en el identificador de trabajo:

$$\text{ray_port} = 20000 + (\text{SLURM_JOB_ID} \pmod{20000}) \quad (3.20)$$

La ecuación (3.20) transforma el identificador del trabajo en un puerto dentro de un intervalo acotado. En ella, `SLURM_JOB_ID` es el identificador entero del trabajo y $\pmod{20000}$ devuelve su resto al dividirlo por 20.000. Por tanto, el puerto calculado pertenece al intervalo entero $[20000, 39999]$; la fórmula reduce la probabilidad de colisión entre trabajos concurrentes, pero no constituye una reserva de puertos del sistema. Se utiliza junto con la autodetección de dirección IP mediante `hostname -ip-address`.

- **Gestión de E/S en almacenamiento local (`/scratch`):** para evitar saturar el sistema de archivos distribuido en red (Sistema de archivos de red (*Network File System*) (NFS)) con la telemetría microscópica de SUMO (`tripinfo.xml`, `summary.xml`), las simulaciones y la recolección de trayectorias se ejecutaron en el almacenamiento local de cada nodo (`/scratch/$USER/ray_tmp/`).

- **Persistencia ante preempción:** los scripts de envío incorporan trampas de señales POSIX (`trap ... EXIT USR1 TERM INT`).

Estas trampas sincronizan mediante `rsync` los resultados y las bases de datos SQLite de Optuna con el almacenamiento persistente del usuario (`$HOME`), en el directorio `ray\_results/`. Esto permite reanudar los trabajos mediante `Tuner.restore()`.

3.8 Evaluación de rendimiento

La evaluación de los modelos con hiperparámetros congelados se definió por escenario y bajo tres niveles de comparación. Cada entrenamiento independiente produce una política asociada a una semilla de entrenamiento; sus episodios de evaluación se agregan primero dentro de esa ejecución. Luego se comparan las ejecuciones independientes, que constituyen la unidad experimental.

Para los escenarios sintéticos se definieron las semillas de confirmación de entrenamiento $\{211, 223, 227, 229, 233, 239, 241, 251, 257, 263\}$ y sus semillas de evaluación correspondientes $\{200211, \dots, 200263\}$. La Ciudad de Mendoza se reservó como un estudio separado, con sus propias semillas y configuración de campaña, posteriores al cierre sintético. Cada receta se evalúa sin nuevas decisiones de ajuste con las semillas reservadas del escenario. Estas semillas son diferentes de las utilizadas para seleccionar candidatos y no se las denomina OOD, porque la demanda y la topología permanecen dentro de la familia definida para cada escenario.

Las partidas de los cuatro escenarios provienen de flujos congelados en archivos de rutas, que SUMO convierte en vehículos durante la ejecución: en Mendoza los instantes de inserción se sortean según el proceso exponencial descrito en la sección 3.3.3, y en las redes sintéticas los flujos siguen intervalos regulares por `vehsPerHour`. En ambos casos la semilla de SUMO gobierna los componentes pseudoaleatorios del simulador —asignación de tipos vehiculares, empates de ruteo y evolución microscópica— además de las llegadas cuando estas son estocásticas. La semilla del generador de demanda de cada escenario queda registrada en su manifiesto.

La confirmación pareada de cooperación se definió para contrastar el valor seleccionado de ρ_{coop} con el ancla $\rho_{\text{coop}} = 0$ en MAPPO y HAPPO, que son las familias para las que se implementa este contraste. IPPO e IDQN se utilizan como líneas de base de aprendizaje independiente: sus recetas se seleccionan con el mismo criterio físico de HPO y sus políticas congeladas se evalúan bajo las mismas semillas reservadas para la comparación final. A diferencia de MAPPO y HAPPO, que incorporan la recompensa cooperativa local-vecinal ($\rho_{\text{coop}} > 0$) y entrenamiento centralizado, IPPO e IDQN operan exclusivamente con la señal de recompensa local ($R_i = L_i$, fijando $\rho_{\text{coop}} = 0$) para representar el paradigma descentralizado e independiente.

El análisis experimental evalúa la hipótesis de si HAPPO logra superar a las líneas de base de aprendizaje multiagente (MAPPO, IPPO e IDQN) en escenarios con disparidad

de fases y heterogeneidad de flujos. De forma complementaria, cada política aprendida se contrasta contra el controlador semafórico de tiempo fijo del mismo escenario, que actúa como ancla operacional no aprendida de referencia. Para una métrica M cuyo valor menor es preferible, se informa la diferencia relativa

$$\Delta_M = 100 \frac{M_{\text{politica}} - M_{\text{fijo}}}{M_{\text{fijo}}}. \quad (3.21)$$

Para tasas de inserción, completitud o finalización, cuyo valor mayor es preferible, se informa además el cambio porcentual con el signo interpretado según esa dirección. La ecuación (3.21) permite comparar redes con distinta cantidad de intersecciones sin confundir una reducción absoluta de segundos con una mejora relativa.

Las métricas se presentan en tablas por escenario y algoritmo, con media, mediana, desviación estándar e intervalo de confianza del 95% por bootstrap percentil. La unidad de remuestreo es la ejecución completa asociada a una semilla, no cada vehículo individual ni cada episodio de una misma política. Para la confirmación pareada se utiliza el procedimiento `paired_bootstrap`: calcula la diferencia candidato menos línea base dentro de cada semilla, realiza 20.000 remuestreos con semilla 910001 y devuelve el intervalo percentil, la diferencia media, la diferencia relativa y la proporción de remuestreos cuya media de diferencias es favorable. Esta proporción no se interpreta como una probabilidad posterior de superioridad.

El remuestreo refleja la incertidumbre de las diez ejecuciones disponibles sin aumentar el tamaño muestral: la precisión de los intervalos descansa en esa muestra. Las métricas de conteo se informan también de forma absoluta para evitar que una razón inestable o un denominador pequeño oculte el comportamiento de la red.

Cada métrica se calcula sobre una población y una ventana declaradas. La espera media de vehículos insertados utiliza como denominador los registros cuyo instante de partida pertenece a la ventana controlada; los viajes inconclusos aportan la espera observada hasta el cierre, que no equivale a su tiempo total de viaje. Si el denominador de una tasa o diferencia relativa es cero, el cambio relativo se informa como no definido y se conserva la diferencia absoluta. Para las tasas de completitud, inserción y finalización se reportan también los puntos porcentuales de diferencia. Cuando todos los tiempos de espera intersección son cero, el Gini se informa como cero por definición operacional.

La tabla 3.4 fija qué magnitudes se consideran primarias, secundarias o de control.

Tabla 3.4: Métricas utilizadas para presentar y controlar la evaluación final.

Métrica	Papel	Justificación
Espera media de vehículos insertados	Primaria	Resume la demora experimentada por la demanda que efectivamente ingresa en la ventana controlada; es el objetivo de HPO.
Pérdida temporal media (<code>timeLoss</code>)	Secundaria	Captura desaceleraciones y circulación por debajo de la velocidad ideal que no siempre se reflejan en detenciones completas.
CR, IR y CI	Eficiencia y atención de demanda	Distinguen una política que reduce la espera de una que simplemente impide insertar vehículos o deja viajes pendientes. La CI mide específicamente la fracción de vehículos insertados en la ventana que completa su itinerario en ella.
Teletransportes iniciados/finalizados	Control de estabilidad	Detecta bloqueos o colapsos que SUMO resuelve mediante su mecanismo de <i>teleport</i> .
Media, desviación, máximo, rango y Gini de espera por intersección	Equidad espacial	Indican si la mejora media se obtiene sacrificando accesos secundarios o concentrando la demora en pocos nodos.

El objetivo de HPO se utiliza exclusivamente para ordenar configuraciones; la evaluación final no reemplaza la espera media por su término de estabilidad. Se reportan por separado la espera estable, la espera media sin penalización y el incremento CVaR. No se aplican umbrales de descarte definidos después de observar los resultados: cualquier reducción de completitud, aumento de teletransportes o deterioro de equidad se muestra como resultado de control y se discute junto con la métrica primaria. Así se evita convertir una decisión posterior en un criterio de selección encubierto.

En síntesis, el diseño experimental estima el cambio de la espera media por vehículo insertado contrastando prioritariamente a HAPPO frente a MAPPO, IPPO e IDQN, utilizando el control de tiempo fijo del mismo escenario como ancla operacional de referencia, con la ejecución completa de entrenamiento como unidad experimental y con CR, IR, CI, teletransportes y equidad espacial como controles de atención a la demanda y de estabilidad. Sus principales restricciones de validez externa son el carácter restringido por datos del escenario de la Ciudad de Mendoza y la línea base de tiempo fijo generada por

SUMO —que no reconstruye los planes semafóricos municipales—. El capítulo 4 presenta los resultados obtenidos con este protocolo.

Análisis y discusión de resultados

En este capítulo se presentan y analizan los resultados de las ejecuciones de entrenamiento y evaluación de los algoritmos estudiados. Con el objetivo de distinguir con claridad las etapas metodológicas del entrenamiento de los agentes, la sección se organiza en tres partes.

En primer lugar, se reportan los resultados del proceso de ajuste de hiperparámetros. En segundo lugar, se evalúan los modelos finales seleccionados en los escenarios planteados, desde redes sintéticas hasta el escenario ubicado en Ciudad de Mendoza. Finalmente, se desarrolla una discusión integradora de los hallazgos, sus implicancias y las principales limitaciones del estudio.

4.1 Resultados del ajuste de hiperparámetros

En esta sección se resume el proceso de ajuste de hiperparámetros y se justifica la configuración utilizada en la evaluación posterior de los modelos.

4.1.1 Diseño del proceso de ajuste

4.1.2 Resultados del ajuste y análisis de sensibilidad

4.1.3 Configuración seleccionada para la evaluación final

4.2 Resultados de los modelos finales

Esta sección reúne los resultados obtenidos con los modelos finales una vez fijada la configuración experimental. La exposición se organiza según el tipo de escenario analizado.

4.2.1 Criterios de evaluación y métricas de comparación

4.2.2 Desempeño en escenarios sintéticos: 3×1 , 3×3 , 4×4 heterogéneo

4.2.3 Desempeño en la red de carreteras de la Ciudad de Mendoza

Antes de evaluar los controladores, se verificó la operación del escenario canónico v14 de la Ciudad de Mendoza en los dos horizontes definidos en la sección 3.3.3. La tabla 4.1 presenta los conteos y las tasas obtenidas con el controlador de tiempo fijo; estos valores caracterizan la condición inicial del escenario y no el desempeño de las políticas aprendidas.

Tabla 4.1: Métricas operacionales del escenario canónico v14 de la Ciudad de Mendoza bajo control de tiempo fijo.

Ventana temporal	Cargados	Insertados	Completados	CR	Inserción	Compleitud	<i>teleport</i>
2 h (06 : 30–08 : 30)	30.629	23.735	22.702	0,7412	0,7749	0,9565	0
12 h (06 : 00–18 : 00)	180.163	143.819	143.022	0,7938	0,7983	0,9945	0

4.3 Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo.

5

Conclusiones

Este capítulo sintetiza los hallazgos del estudio, relaciona la evidencia experimental con los objetivos planteados y delimita el alcance de las conclusiones. También identifica las limitaciones del modelo y propone líneas de trabajo futuro. Las afirmaciones finales deben interpretarse dentro de los escenarios, métricas y protocolos descritos en la metodología.



Arquitectura de software y patrones de diseño

En este apéndice se documenta la especificación arquitectónica del marco computacional `marlTLC`, su organización orientada a objetos, los diagramas de clases UML y los patrones de diseño previstos para desacoplar los componentes de simulación, aprendizaje por refuerzo y cálculo de recompensas. El repositorio que acompaña a este documento no contiene el código fuente completo del marco; por ello, las clases, métodos y relaciones que se presentan deben interpretarse como una especificación de diseño y no como una auditoría de archivos ejecutables.

A.1 Diseño orientado a objetos del entorno de simulación

El diseño del entorno separa las responsabilidades de sus componentes. En la especificación, la clase abstracta `BaseSumoEnvironment` encapsula la inicialización de SUMO, el control temporal y la interacción de bajo nivel con la API `libsumo` / `traci`. A partir de ella, `SumoRllibEnv` se define como el adaptador de la simulación a la interfaz `MultiAgentEnv` de Ray `RLLib`.

La figura A.1 ilustra el diagrama de clases UML correspondiente a las entidades del entorno y su relación con los controladores semafóricos. Un diagrama de clases representa los tipos de objetos que componen un programa. Cada recuadro se divide en tres partes: el nombre de la clase, sus atributos o datos almacenados y sus métodos u operaciones. El signo “-” indica un miembro de uso interno o privado; “+”, uno público; y “#”, uno

protegido, destinado a la propia clase y a sus derivadas. El texto situado después de los dos puntos indica el tipo de dato esperado. Los nombres en cursiva corresponden a clases u operaciones abstractas: definen una interfaz que debe concretarse en otra clase. Una flecha con la leyenda “hereda de” apunta desde la clase especializada hacia la clase general, mientras que la multiplicidad 1..* significa “uno o más” objetos relacionados.

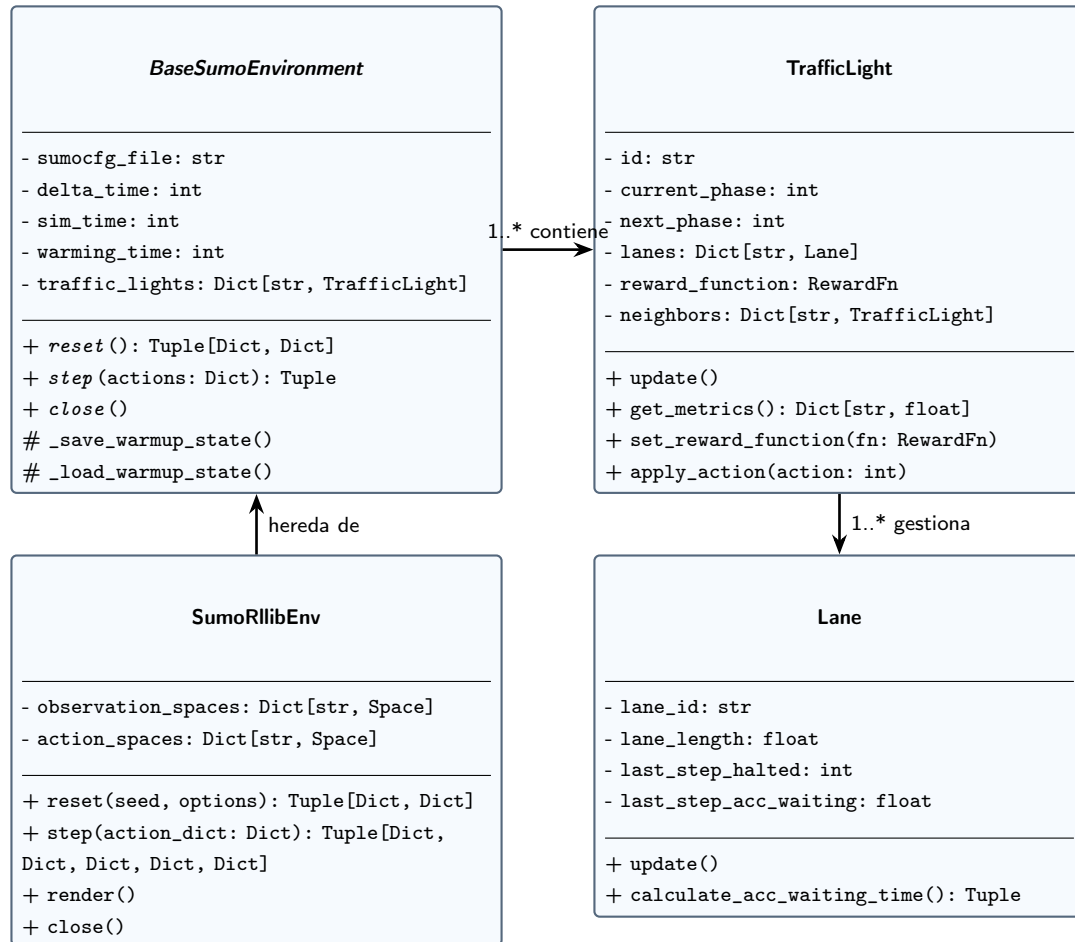


Figura A.1: Diagrama de clases UML del entorno. **SumoRllibEnv** hereda la interfaz de **BaseSumoEnvironment**; el entorno contiene uno o más objetos **TrafficLight**, y cada controlador semafórico gestiona uno o más objetos **Lane**. En cada clase se muestran sus principales atributos y métodos.

La figura A.1 se organiza desde la abstracción general hacia las entidades de la red. En la columna izquierda, **BaseSumoEnvironment** concentra el archivo de configuración, los parámetros temporales, los controladores y las operaciones generales de reinicio, avance y cierre. Debajo, **SumoRllibEnv** especializa esa clase para recibir un diccionario de acciones de los agentes y devolver las observaciones, recompensas y estados definidos por la interfaz de Ray Rllib. A la derecha, el diseño asigna a **TrafficLight** la fase actual, la siguiente fase, sus carriles, su función de recompensa y sus vecinos; también le asigna la actualización de métricas y la aplicación de acciones. Finalmente, **Lane** representa la longitud y las medidas recientes de detención y espera. Las flechas muestran las dependencias previstas

entre la interfaz de aprendizaje, el entorno, los controladores y sus carriles.

A.2 Jerarquía de funciones de recompensa y patrón Strategy

En la especificación, el cálculo de la recompensa se desacopla mediante el patrón **Strategy**. Esta organización permite sustituir la métrica de optimización sin modificar el bucle principal de simulación.

La clase base abstracta **RewardFn** define la interfaz común `compute_reward(tl_id)`. En el diseño se contemplan formulaciones locales basadas en tres señales: demora acumulada, longitud de cola y presión de red. La coordinación distribuida se representa mediante el patrón **Composite**: una clase de recompensa compuesta combina la señal local y la señal promedio de los semáforos adyacentes.

La figura A.2 detalla las clases que intervienen directamente en la recompensa cooperativa. Las demás estrategias locales registradas en el sistema no se dibujan para conservar la legibilidad. Se mantienen las convenciones explicadas para la figura A.1; en este segundo diagrama, la flecha discontinua rotulada “compone” indica que una clase utiliza otra como parte de su cálculo, no que herede sus atributos y métodos.

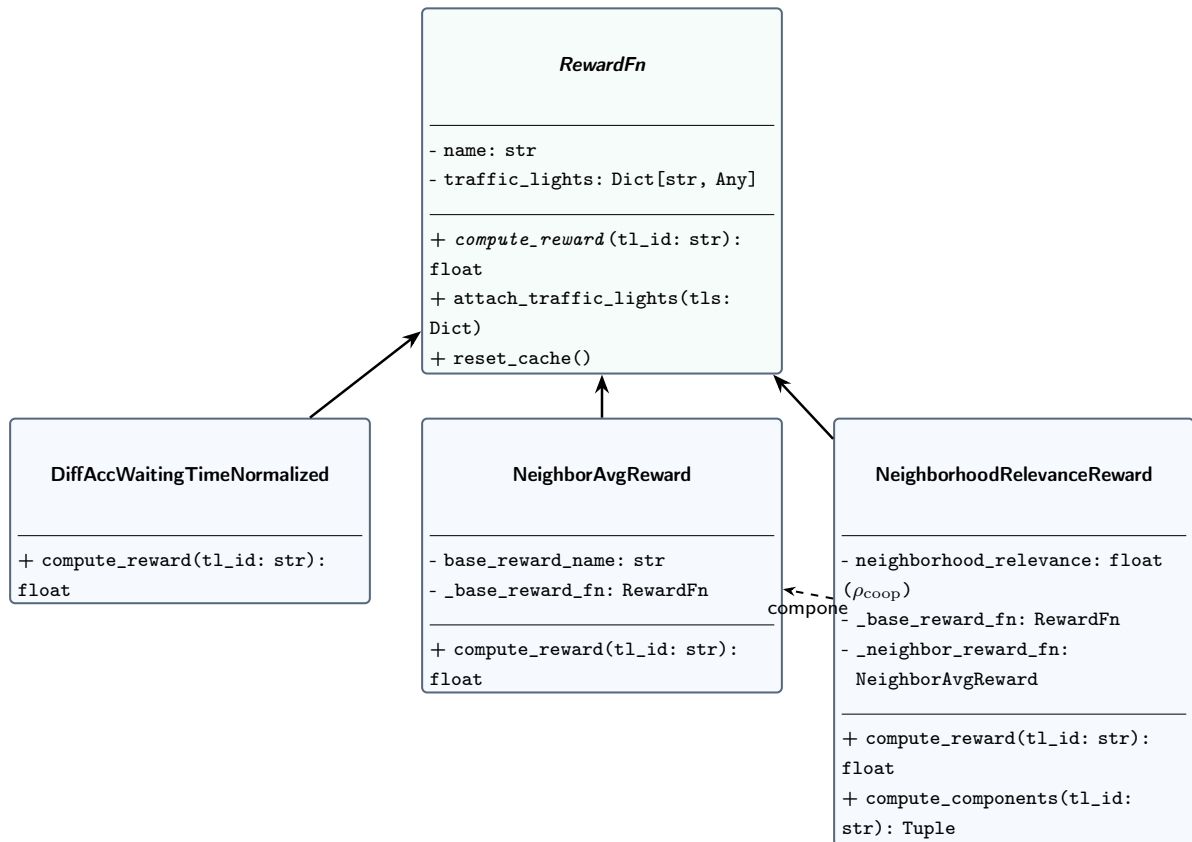


Figura A.2: Especificación UML del módulo de recompensas. Las clases inferiores se definen como realizaciones de **RewardFn**; la clase compuesta incorpora la señal promedio de los controladores vecinos.

La figura A.2 sitúa a `RewardFn` en la parte superior porque define la operación común `compute_reward`, que recibe el identificador de un controlador semafórico y devuelve un valor numérico. Las tres flechas ascendentes indican que las clases inferiores se especializan para proporcionar variantes de esa operación. `DiffAccWaitingTimeNormalized` representa una señal local normalizada a partir de la variación del tiempo de espera acumulado, mientras que `NeighborAvgReward` promedia la señal base de los controladores vecinos. `NeighborhoodRelevanceReward` conserva ambas funciones y las combina según ρ_{coop} , el peso de cooperación vecinal. La flecha discontinua entre las dos últimas clases muestra esa relación de composición. Esta separación permite cambiar la formulación de la recompensa sin modificar el bucle que avanza la simulación.

A.3 Patrones de diseño implementados

A continuación se resumen los patrones contemplados en la especificación de `marlTLC`. La lista describe responsabilidades y relaciones de diseño; no certifica la presencia de una implementación ejecutable de cada componente en el repositorio.

1. Patrón Strategy (Estrategia):

- **Propósito:** encapsular los algoritmos de recompensa para intercambiarlos sin modificar los controladores semafóricos.
- **Rol en el diseño:** interfaz base `RewardFn` y variantes concretas de recompensa.

2. Patrón Factory Method y Registry (Fábrica con Registro Dinámico):

- **Propósito:** instanciar funciones de recompensa y módulos neuronales a partir de cadenas de configuración o especificaciones compuestas (`RewardSpec`). También permite registrar nuevas funciones sin modificar la fábrica, de acuerdo con el principio abierto/cerrado.
- **Rol en el diseño:** registro mediante `RewardFactory` y `@register_reward`, que identifica el mecanismo previsto para asociar nombres de configuración con funciones de recompensa.

3. Patrón Composite (Objeto Compuesto):

- **Propósito:** combinar recompensas locales y vecinales mediante una interfaz uniforme.
- **Rol en el diseño:** clases de recompensa compuesta que agregan señales locales y vecinales.

4. Patrón Protocol / Structural Subtyping (Tipado Estructural):

- **Propósito:** desacoplar el proveedor de métricas vecinales del tipo concreto `TrafficLight`, facilitando las pruebas unitarias con objetos simulados (*mocks*) y evitando dependencias circulares.
- **Rol en el diseño:** protocolo `NeighborInfoProvider` para describir la información vecinal requerida sin depender de una clase concreta. El detalle del decorador de comprobación en tiempo de ejecución no forma parte de la evidencia disponible en este repositorio.

5. Patrón Adapter (Adaptador):

- **Propósito:** traducir la API procedural y basada en comandos de TraCI / libsumo a la interfaz orientada a objetos de `MultiAgentEnv` (Ray RLlib).
- **Rol en el diseño:** `SumoRllibEnv` como punto de adaptación entre el simulador y la interfaz de aprendizaje.

6. Patrón Decorator (Decorador):

- **Propósito:** extender dinámicamente las capacidades de los módulos sin alterar su jerarquía de herencia.
- **Rol en el diseño:** decoradores de registro y una envoltura para los módulos de acción en PyTorch; los nombres concretos de esas piezas se mantienen como identificadores de la especificación.

Bibliografía

- [ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. <https://www.marl-book.com/>, Consultado en 2025.
- [AHL16] Abel, David, D. Ellis Hershkowitz y Michael L. Littman: *Near Optimal Behavior via Approximate State Abstraction*. En *Proceedings of the 33rd International Conference on Machine Learning*, volumen 48 de *Proceedings of Machine Learning Research*, páginas 2915–2923. PMLR, 2016.
- [AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.
- [AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the Third Workshop on Data-driven Intelligent Transportation, co-located with CIKM '22*, 2022. <https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf>, Fuente: [7, 8] Destaca el "éxito limitado" de SARL y los enfoques independientes en redes.
- [Aso18] Asociación de Fábricas Argentinas de Componentes: *Flota circulante en Argentina 2017*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, mayo 2018. <https://cdn.motor1.com/pdf-files/afac-informe-parque-circulante-2017pdf.pdf>, Informe elaborado con datos de AFAC y Promotive.
- [Aso26] Asociación de Fábricas Argentinas de Componentes: *Flota vehicular circulante en Argentina*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, 2026. https://www.automotrix.com.ar/adm/enviosmasivos/Plantillas/PRENSA/2026/AFAC-PRENSA_FLOTACIRCULANTE_2025.pdf, Datos al cierre de 2025; informe elaborado en el marco del convenio entre AFAC y Promotive S.A.

- [ASY⁺19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.
- [BB24] Bishop, Christopher M. y Hugh Bishop: *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.
- [BV02] Bowling, Michael y Manuela Veloso: *Multiagent learning using a variable learning rate*. *Artificial Intelligence*, 136(2):215–250, 2002.
- [CWCL19] Chu, Tianshu, Jie Wang, Lara Codecà y Zhaojian Li: *Multi-agent deep reinforcement learning for large-scale traffic signal control*. *IEEE Transactions on Intelligent Transportation Systems*, 21(3):1086–1095, 2019.
- [CYL⁺24] Chen, Wubing, Shangdong Yang, Wenbin Li, Yujing Hu, Xiao Liu y Yang Gao: *Learning Multi-Intersection Traffic Signal Control via Coevolutionary Multi-Agent Reinforcement Learning*. *IEEE Transactions on Intelligent Transportation Systems*, 25(11):15947–15963, 2024.
- [CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.
- [DLR26] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2026. <https://sumo.dlr.de/docs/>, Documentación web consultada en 2026.
- [DLSZ26] Deng, Maojiang, Shoufeng Lu, Jiazhao Shi y Wen Zhang: *Adaptive Traffic Signal Control Optimization Using a Novel Road Partition and Multi-Channel State Representation Method*. *Urban Lifeline*, 4(1):9, 2026.
- [dWGM⁺20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr y cols.: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.
- [FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.
- [Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Capítulo 4: Traffic Signal Design. Disponible en: <https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter4.htm>.

- [FNF⁺17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.
- [FYX⁺23] Fang, Jie, Ya You, Mengyun Xu, Juanmeizi Wang y Sibin Cai: *Multi-Objective Traffic Signal Control Using Network-Wide Agent Coordinated Reinforcement Learning*. *Expert Systems with Applications*, 229:120535, 2023.
- [GBC16] Goodfellow, Ian, Yoshua Bengio y Aaron Courville: *Deep Learning*. MIT Press, Cambridge, MA, 2016. <https://www.deeplearningbook.org/>.
- [GR18] Genders, Wade y Saiedeh Razavi: *Evaluating Reinforcement Learning State Representations for Adaptive Traffic Signal Control*. *Procedia Computer Science*, 130:26–33, 2018.
- [HHA22] Haddad, Tarek Amine, Djalal Hedjazi y Sofiane Aouag: *A Deep Reinforcement Learning-Based Cooperative Approach for Multi-Intersection Traffic Signal Control*. *Engineering Applications of Artificial Intelligence*, 114:105019, 2022.
- [JLW⁺24] Jiang, Haoyuan, Ziyue Li, Hua Wei, Xuantang Xiong, Jingqing Ruan, Jiaming Lu, Hangyu Mao y Rui Zhao: *X-Light: Cross-City Traffic Signal Control Using Transformer on Transformer as Meta Multi-Agent Reinforcement Learner*. En *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, páginas 94–102, 2024.
- [KCW⁺22] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fuchun Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. En *International Conference on Learning Representations (ICLR)*, 2022. <https://arxiv.org/abs/2109.11251>.
- [KKBA23] Kolat, Máté, Bálint Kővári, Tamás Bécsi y Szilárd Aradi: *Multi-Agent Reinforcement Learning for Traffic Signal Control: A Cooperative Approach*. *Sustainability*, 15(4):3479, 2023.
- [KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.
- [KT03] Konda, Vijay R. y John N. Tsitsiklis: *On Actor-Critic Algorithms*. *SIAM Journal on Control and Optimization*, 42(4):1143–1166, 2003.

- [LFYZ25] Lu, Qiong, Haoda Fang, Zhangcheng Yin y Guliang Zhu: *HAPS-PPO: A Multi-Agent Reinforcement Learning Architecture for Coordinated Regional Control of Traffic Signals in Heterogeneous Road Networks*. Applied Sciences, 15(20):10945, 2025.
- [LJR⁺20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-Tzur, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning*. En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.
- [LLN⁺18] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstractions for distributed reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.
- [LML⁺25] Liang, Jiaxin, Haotian Miao, Kai Li, Jianheng Tan, Xi Wang, Rui Luo y Yueqiu Jiang: *A Review of Multi-Agent Reinforcement Learning Algorithms*. Electronics, 14(4):820, 2025. <https://doi.org/10.3390/electronics14040820>.
- [LWT⁺17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.
- [LZF⁺25] Liu, Wanting, Chengwei Zhang, Wanqing Fang, Kailing Zhou, Yihong Li, Furui Zhan, Qi Wang, Wanli Xue y Rong Chen: *Vehicle-Level Fairness-Oriented Constrained Multi-Agent Reinforcement Learning for Adaptive Traffic Signal Control*. IEEE Transactions on Intelligent Transportation Systems, 26(4):4878–4890, 2025.
- [MKS⁺15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning*. Nature, 518(7540):529–533, 2015. <https://doi.org/10.1038/nature14236>.
- [MMLK25] Michailidis, Panagiotis, Iakovos Michailidis, Charalampos Rafail Lazaridis y Elias Kosmatopoulos: *Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations*. Infrastructures, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.

- [PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.
- [RDZ⁺24] Ren, Fuyue, Wei Dong, Xiaodong Zhao, Fan Zhang, Yaguang Kong y Qiang Yang: *Two-Layer Coordinated Reinforcement Learning for Traffic Signal Control in Traffic Network*. *Expert Systems with Applications*, 235:121111, 2024.
- [RSDW⁺18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, páginas 4295–4304. PMLR, 2018.
- [SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction*. The MIT Press, Cambridge, MA, 2nd edición, 2018.
- [Sha53] Shapley, Lloyd S: *Stochastic games*. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.
- [SLA⁺15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization*. En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.
- [SLG⁺18] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls y Thore Graepel: *Value-Decomposition Networks For Cooperative Multi-Agent Learning*. En *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, páginas 2085–2087. International Foundation for Autonomous Agents and Multiagent Systems, 2018.
- [SML⁺16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation*. En *International Conference on Learning Representations (ICLR)*, 2016.
- [SWD⁺17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.
- [TMK⁺17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning*. *PloS one*, 12(4):e0172395, 2017.

- [W⁺19] Wei, H. y cols.: *A survey on traffic signal control methods*, 2019. Available at: <https://arxiv.org/abs/1904.08117>.
- [WCZ⁺19] Wei, Hua, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu y Zhenhui Li: *PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 1290–1298, 2019.
- [Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.
- [WWS⁺22] Wu, Qiang, Jianqing Wu, Jun Shen, Bo Du, Akbar Telikani, Mahdi Fahmideh y Chao Liang: *Distributed Agent-Based Deep Reinforcement Learning for Large Scale Traffic Signal Control*. Knowledge-Based Systems, 241:108304, 2022.
- [WXZ⁺19] Wei, Hua, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu y Zhenhui Li: *CoLight: Learning Network-level Cooperation for Traffic Signal Control*. En *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, páginas 1913–1922, 2019.
- [WZK⁺23] Wang, Xihuai, Lingpan Zheng, Jakub Grudzien Kuba, Ruiqing Chen, Li Shen, Yidong Cao, Jun Wang y Yaodong Yang: *Order matters: Agent-by-agent policy optimization*. En *International Conference on Learning Representations (ICLR)*, 2023.
- [Yan23] Yang, Shantian: *Hierarchical Graph Multi-Agent Reinforcement Learning for Traffic Signal Control*. Information Sciences, 634:55–72, 2023.
- [YVV⁺22] Yu, Chao, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.
- [ZKF⁺24] Zhong, Yifan, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji y Yaodong Yang: *Heterogeneous-Agent Reinforcement Learning*. Journal of Machine Learning Research, 25:1–67, 2024. <https://www.jmlr.org/papers/v25/23-0488.html>.
- [ZLFS24] Zhang, Yifeng, Peizhuo Li, Mingfeng Fan y Guillaume Sartoretti: *Hetero-Light: A General and Efficient Learning Approach for Heterogeneous Traffic*

- Signal Control*. En *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, páginas 1010–1017, 2024.
- [ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.
- [ZWL24] Zhang, Haipeng, Zhiwen Wang y Na Li: *MATLight: Traffic Signal Coordinated Control Algorithm Based on Heterogeneous-Agent Mirror Learning With Transformer*. En *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, páginas 2582–2584, 2024.